Chapter 1 — Vision & Philosophy

Purpose

IRON BOLT শুধুমাত্র আরেকটি AI Chatbot নয়।
এটি একটি modular AI operating framework, যেখানে বিভিন্ন Large Language Model (LLM), Tools, Memory System এবং Intelligent Agent একই architecture-এর মধ্যে একসাথে কাজ করতে পারে।

Project-এর মূল লক্ষ্য হলো এমন একটি production-ready foundation তৈরি করা, যার উপর ভবিষ্যতে Personal AI Assistant, Autonomous Agent, Coding Assistant, Research Assistant, Automation System অথবা সম্পূর্ণ AI Operating System তৈরি করা সম্ভব হবে।

IRON BOLT-এর প্রতিটি component loosely coupled রাখা হবে, যাতে একটি component পরিবর্তন করলেও পুরো system পরিবর্তন করতে না হয়।

Vision

IRON BOLT-এর দীর্ঘমেয়াদী লক্ষ্য হলো—
একটি unified AI platform তৈরি করা যেখানে

- একাধিক AI Provider ব্যবহার করা যাবে।
- Runtime-এ Model পরিবর্তন করা যাবে।
- Tool execution নিরাপদভাবে পরিচালিত হবে।
- AI নিজের Memory ব্যবহার করতে পারবে।
- বিভিন্ন Agent একই Framework ব্যবহার করবে।
- ভবিষ্যতে Multi-Agent Collaboration সম্ভব হবে।
- Production-scale application তৈরি করা যাবে।

অর্থাৎ IRON BOLT একটি single chatbot নয়, বরং AI applications তৈরির একটি complete foundation।

Design Philosophy

IRON BOLT-এর প্রতিটি feature নিচের design principles অনুসরণ করবে।

Modular

প্রতিটি module আলাদা থাকবে।

Provider, Memory, Tool, Agent, API—সবগুলো independent component হিসেবে তৈরি হবে।

এর ফলে maintenance সহজ হবে এবং নতুন feature যোগ করা অনেক সহজ হবে।

Provider Independent

System কখনও একটি নির্দিষ্ট AI Provider-এর উপর নির্ভর করবে না।

OpenAI, Gemini, Groq, Ollama, OpenRouter অথবা ভবিষ্যতের নতুন Provider—সব একই interface ব্যবহার করবে।

এই architecture vendor lock-in এড়ায়।

Scalable

ছোট project থেকে enterprise-level application পর্যন্ত একই architecture ব্যবহার করা যাবে।
ভবিষ্যতে component সংখ্যা অনেক বেড়ে গেলেও system maintain করা সহজ থাকবে।

Extensible

IRON BOLT-এর প্রতিটি layer এমনভাবে তৈরি হবে যাতে নতুন module যোগ করতে existing code খুব কম পরিবর্তন করতে হয়।

নতুন Provider, নতুন Tool, নতুন Memory Backend অথবা নতুন Agent যোগ করা framework-এর স্বাভাবিক capability হবে।

Testable

প্রতিটি component আলাদাভাবে test করা যাবে।

Unit Test, Integration Test এবং Future End-to-End Test architecture-এর অংশ হিসেবে বিবেচিত হবে।

Production First

Development-এর শুরু থেকেই production-quality architecture অনুসরণ করা হবে।

Temporary solution বা quick hack ব্যবহার করা হবে না যদি তা ভবিষ্যতের architecture নষ্ট করে।

Code readability, maintainability এবং reliability সবসময় অগ্রাধিকার পাবে।

Current Implementation

বর্তমানে IRON BOLT-এর foundation তৈরি হয়েছে।

এ পর্যন্ত মূলত Core Architecture-এর ভিত্তি প্রস্তুত করা হয়েছে, যার মধ্যে রয়েছে—

- Core project structure
- Configuration system

- Provider abstraction
- Provider manager
- Startup validation
- Multiple provider support-এর foundation
- Runtime configuration
- Provider health checking
- Test infrastructure

এগুলো ভবিষ্যতের সকল component-এর ভিত্তি হিসেবে কাজ করবে।

Future Direction

এই foundation-এর উপর ধাপে ধাপে নিচের major systems যুক্ত করা হবে—

- Model Manager
- Tool System
- Memory System
- Agent Engine
- Workflow Engine
- Knowledge Layer
- API Server
- Plugin System
- Monitoring
- Deployment Infrastructure

প্রতিটি system আগের chapter-এর উপর ভিত্তি করে তৈরি হবে, যাতে পুরো architecture একটি cohesive এবং production-ready ecosystem হিসেবে গড়ে ওঠে।

Chapter 2 — Core Architecture

Purpose

Core Architecture হলো IRON BOLT-এর কঙ্কাল (Skeleton)।

যেমন একটি ভবনের foundation ঠিক করে দেয় কোথায় কোন কক্ষ থাকবে, কীভাবে বিদ্যুৎ, পানি ও সিঁড়ি সংযুক্ত থাকবে—তেমনি Core Architecture নির্ধারণ করে দেয় পুরো AI Framework-এর প্রতিটি component কোথায় থাকবে, কী দায়িত্ব পালন করবে এবং কীভাবে অন্য component-এর সাথে যোগাযোগ করবে।

এই architecture-এর মূল উদ্দেশ্য হলো এমন একটি structure তৈরি করা, যা ছোট project থেকে শুরু করে production-scale AI system পর্যন্ত সহজে সম্প্রসারণ (scale) করা যায়।

Architecture Philosophy

IRON BOLT Layered Architecture অনুসরণ করে।

এর অর্থ হলো পুরো system-কে একাধিক স্বাধীন layer-এ ভাগ করা হয়েছে।
প্রতিটি layer-এর একটি নির্দিষ্ট দায়িত্ব রয়েছে এবং কোনো layer অন্য layer-এর দায়িত্ব গ্রহণ করবে না।
Layer Architecture-এ connection হওয়া উচিত interface-এর মাধ্যমে, সরাসরি module dependency দিয়ে নয়।

```
from memory import Memory
from tools import ToolManager
from providers import ProviderManager
```

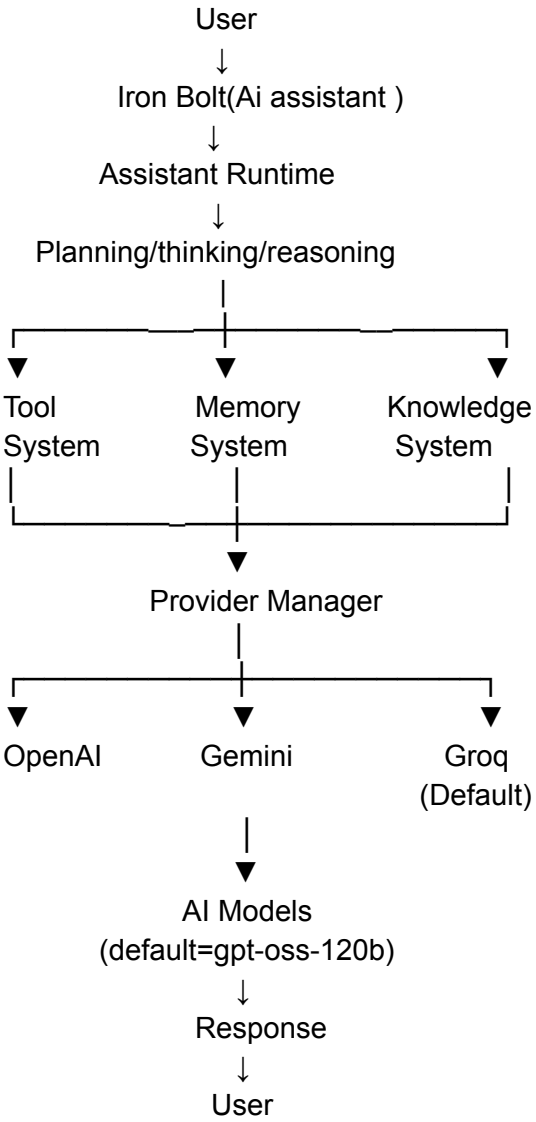
তাই একটা **rule:**

- ✅ core.* → যেকোনো layer import করতে পারবে।
- ⚠️ provider.* → শুধু Assistant (বা যে layer-এর দরকার) ব্যবহার করবে।
- ⚠️ memory.* → প্রয়োজন হলে Assistant ব্যবহার করবে।
- ⚠️ tools.* → প্রয়োজন হলে Assistant ব্যবহার করবে।
- ❌ Memory কখনও Tool import করবে না।
- ❌ Tool কখনও Provider import করবে না।
- ❌ Provider কখনও Memory import করবে না।

এর ফলে—

- Code পরিষ্কার থাকে।
- Debug করা সহজ হয়।
- Testing সহজ হয়।
- নতুন feature যোগ করা সহজ হয়।
- Component পরিবর্তন করলেও পুরো system ভেঙে পড়ে না।

High-Level Architecture



- IRON BOLT — Assistant হিসেবে পুরো conversation পরিচালনা করে।
- Assistant Runtime — Request-এর execution পরিচালনা করে।
- Planner / Reasoning — কী করতে হবে তা নির্ধারণ করে।

- Memory / Knowledge / Tools — Planner-এর প্রয়োজন অনুযায়ী তথ্য, অভিজ্ঞতা বা capability সরবরাহ করে।
- Provider Manager — কোন AI Provider ব্যবহার হবে তা পরিচালনা করে।
- Provider — OpenAI, Gemini, Groq-এর মতো নির্দিষ্ট AI service-এর সাথে যোগাযোগ করে।
- AI Model — চূড়ান্ত reasoning এবং response generationAgent

এই Flow-এর মাধ্যমে User-এর Request ধাপে ধাপে বিভিন্ন layer অতিক্রম করে AI Model পর্যন্ত পৌঁছায় এবং Response একই পথ ধরে ফিরে আসে।

Architectural Layers

IRON BOLT-এর Architecture কয়েকটি প্রধান Layer নিয়ে গঠিত।

Foundation Layer

এটি পুরো Framework-এর ভিত্তি।

এখানে থাকে—

- Configuration System
- Settings
- Validation
- Startup Logic
- Common Utilities
- Base Classes

অন্য সব layer এই foundation-এর উপর নির্ভর করবে।

Provider Layer

এই Layer-এর কাজ হলো বিভিন্ন AI Provider-এর সাথে যোগাযোগ করা।
এটি কখনো Business Logic পরিচালনা করবে না।

এর দায়িত্ব শুধুমাত্র—

- Model Request পাঠানো
- Response গ্রহণ করা
- Authentication পরিচালনা করা
- Provider Health Check করা
- Runtime Model নির্বাচন করা

বর্তমানে এই Layer-এর ভিত্তি তৈরি হয়েছে।
ভবিষ্যতে আরও Provider যুক্ত হবে।

Agent Layer

Agent Layer পুরো Framework-এর "Brain" হিসেবে কাজ করবে।

এখানেই AI সিদ্ধান্ত নেবে—

- Tool লাগবে কি না।
- Memory ব্যবহার করবে কি না।
- কোন Model ব্যবহার করবে।
- পরবর্তী Action কী হবে।

বর্তমানে এটি Future Implementation।

Tool Layer

AI যখন বাইরের পৃথিবীর সাথে কাজ করবে তখন Tool Layer ব্যবহার হবে।
যেমন—

- File পড়া
- Calculator
- Search
- Python Execution
- Database
- Email
- Browser

Agent সরাসরি Tool ব্যবহার করবে না।
সব Tool Tool Manager-এর মাধ্যমে পরিচালিত হবে।

ToolManager-এর কাজ হবে:

- সব Tool register করা।
- Tool খুঁজে বের করা।
- Tool execute করা।
- Tool enable/disable করা।
- Permission check করা।

অর্থাৎ Assistant কখনো TerminalTool() বা PythonTool() সরাসরি ব্যবহার করবে না।

বরং করবে:
tool_manager.execute("terminal", command)

tools/
├── tool_manager.py
├── base_tool.py
├── terminal_tool.py

```
├── python_tool.py
├── file_tool.py
├── calculator_tool.py
└── ...
```

Memory Layer

AI-এর দীর্ঘমেয়াদী ও স্বল্পমেয়াদী তথ্য সংরক্ষণ করবে।

এখানে থাকতে পারে—

- Conversation Memory
- Session Memory
- Long-term Memory
- Vector Memory
- User Preferences
- বর্তমানে Future Scope।

MemoryManager-এর কাজ:

- Memory save করা।
- Memory retrieve করা।
- কোন Memory ব্যবহার হবে তা নির্বাচন করা।
- ভবিষ্যতে একাধিক Memory backend থাকলে সেগুলো manage করা।

```
memory/
├── memory_manager.py
├── base_memory.py
├── short_term.py
├── long_term.py
├── conversation.py
└── ...
```

Knowledge Layer

AI-এর বাহ্যিক জ্ঞান ব্যবস্থাপনা। যেমন—

- Documents
- PDF
- Database
- Vector Store
- Knowledge Base
- Retrieval

Memory এবং Knowledge এক জিনিস নয়।
Memory হলো AI-এর অভিজ্ঞতা।
Knowledge হলো AI-এর তথ্যভান্ডার।

Architecture Rules

IRON BOLT-এর Architecture কিছু কঠোর নিয়ম অনুসরণ করবে।

কোনো component অন্য component-এর Internal Code ব্যবহার করবে না।
সব Communication Public Interface-এর মাধ্যমে হবে।
প্রতিটি Layer শুধুমাত্র তার নিচের Layer-এর Service ব্যবহার করবে।
Circular Dependency সম্পূর্ণভাবে নিষিদ্ধ থাকবে।
Shared Logic কখনো বিভিন্ন জায়গায় কপি করা হবে না; প্রয়োজনে Common Module-এ রাখা হবে।
প্রতিটি Module-এর একটি মাত্র দায়িত্ব (Single Responsibility) থাকবে।

Architecture Rules-এর কাজ হলো শুরুতেই এই ধরনের জিনিস নিষিদ্ধ করা।

যেমন:

- core → সবাই ব্যবহার করতে পারবে।
- Assistant → ToolManager, MemoryManager, ProviderManager ব্যবহার করতে পারবে।
- Tool → Provider import করতে পারবে না।
- Memory → Tool import করতে পারবে না।
- Provider → Memory import করতে পারবে না।
- Layer-গুলোর dependency একমুখী (one-way) হবে।

Current Implementation

বর্তমানে Core Foundation-এর উল্লেখযোগ্য অংশ তৈরি হয়েছে।

এর মধ্যে রয়েছে—

- Project Structure
- Configuration System
- Provider Interface
- Provider Manager
- Runtime Configuration
- Startup Validation
- Provider Health Check
- Test Infrastructure

এই অংশগুলো বর্তমানে কার্যকর এবং ভবিষ্যতের সকল layer-এর ভিত্তি হিসেবে কাজ করবে।

Future Implementation

Core Architecture অপরিবর্তিত থাকবে, তবে এর উপর ধাপে ধাপে নতুন layer যুক্ত হবে।

আগামী development পর্যায়ে যুক্ত হবে—

- Model Manager
- Tool Manager
- Memory Manager
- Agent Engine
- Workflow Engine
- Knowledge System
- Plugin Framework
- Monitoring System
- Deployment Layer

এই নতুন component-গুলো বিদ্যমান architecture-এর নিয়ম মেনেই যুক্ত হবে, যাতে framework-এর modularity, scalability এবং maintainability অক্ষুণ্ণ থাকে।

Chapter 3 — Core Foundation

Purpose

Core Foundation হলো IRON BOLT-এর হৃদয় (Core Runtime)।
যদি পুরো Framework-কে একটি শহরের সাথে তুলনা করা হয়, তাহলে Core Foundation হলো সেই শহরের বিদ্যুৎ, রাস্তা, পানি ও প্রশাসন। সাধারণ ব্যবহারকারী এগুলো সরাসরি দেখে না, কিন্তু এগুলো ছাড়া পুরো শহর অচল।

ঠিক তেমনি, AI Provider, Tool System, Memory, Agent কিংবা API—কোনোটিই সরাসরি একে অপরের উপর নির্ভর করবে না। তারা সবাই Core Foundation-এর উপর নির্ভর করবে।
এই layer-এর প্রধান কাজ হলো Framework-এর জন্য একটি stable, predictable এবং reusable runtime environment তৈরি করা।

Why Core Foundation Exists

যদি প্রতিটি component নিজে নিজে Configuration পড়ে, Provider তৈরি করে, Error Handle করে, Logging করে এবং Validation চালায়, তাহলে একই ধরনের Code বারবার লেখা হবে।

এর ফলে—

- Code duplication বাড়বে।

- Bug বাড়বে।
- Maintenance কঠিন হবে।
- নতুন Developer-এর জন্য Project বোঝা কঠিন হবে।
- ভবিষ্যতে বড় পরিবর্তন করা ঝুঁকিপূর্ণ হবে।

Core Foundation এই সমস্যাগুলো সমাধান করে।

এটি Framework-এর জন্য Common Infrastructure প্রদান করে, যাতে প্রতিটি Component শুধুমাত্র নিজের কাজের উপর মনোযোগ দিতে পারে।

Responsibilities of Core Foundation

Core Foundation-এর কাজ Business Logic পরিচালনা করা নয়।
এর দায়িত্ব শুধুমাত্র Framework-এর মৌলিক অবকাঠামো (Infrastructure) প্রদান করা।

এর মধ্যে রয়েছে—

- Project Configuration পরিচালনা করা।
- Runtime Environment প্রস্তুত করা।
- Provider Lifecycle পরিচালনা করা।
- System Startup Validation করা।
- Common Base Classes প্রদান করা।
- Shared Utilities প্রদান করা।
- Framework Initialization পরিচালনা করা।
- Framework-এর বিভিন্ন Module-কে একসাথে সংযুক্ত করার ভিত্তি তৈরি করা।

Core Directory

ভবিষ্যতে core/ directory হবে Framework-এর সবচেয়ে গুরুত্বপূর্ণ অংশ।
এই directory-তে শুধুমাত্র সেই Component থাকবে, যেগুলো Framework-এর একাধিক Layer ব্যবহার করবে।
উদাহরণস্বরূপ—

```
core/
├── config.py
├── provider_manager.py
├── startup_validation.py
├── exceptions.py
├── logging.py
├── lifecycle.py
├── registry.py
├── constants.py
└── utils.py
```

এই Structure সময়ের সাথে পরিবর্তিত হতে পারে, কিন্তু নীতিটি অপরিবর্তিত থাকবে—Core শুধুমাত্র Shared Infrastructure ধারণ করবে।

Core Components

Configuration System

Configuration System পুরো Framework-এর কেন্দ্রীয় Configuration Manager।

এটি Environment Variables, Configuration Files এবং Runtime Settings পরিচালনা করবে।

Framework-এর কোনো Component নিজে Environment Variable পড়বে না।

সব Configuration শুধুমাত্র Configuration System-এর মাধ্যমে পাওয়া যাবে।

এর ফলে Configuration পরিবর্তন করলে অন্য Component পরিবর্তন করতে হবে না।

বর্তমান অবস্থা**: Implemented**।

Provider Manager

Provider Manager Framework-এর সকল AI Provider-এর কেন্দ্রীয় নিয়ন্ত্রক।

এটি—

- Provider Register করবে।
- Provider Initialize করবে।
- Provider নির্বাচন করবে।
- Provider Health Monitor করবে।
- Provider পরিবর্তন করবে।
- Runtime Provider Switching পরিচালনা করবে।

Agent কিংবা API কখনও সরাসরি OpenAI Provider বা Gemini Provider ব্যবহার করবে না।
সবসময় Provider Manager-এর মাধ্যমে Provider ব্যবহার করা হবে।
বর্তমান অবস্থা: Implemented (Foundation Complete)।

Startup Validation

Framework চালু হওয়ার আগে System প্রস্তুত কিনা তা যাচাই করবে।
যেমন—

- Configuration বৈধ কিনা।
- API Key আছে কিনা।

- Required Dependency আছে কিনা।
- Provider Ready কিনা।
- Required Resource পাওয়া যাচ্ছে কিনা।

যদি Startup Validation ব্যর্থ হয়, তাহলে Framework নিরাপদভাবে Startup বন্ধ করবে এবং পরিষ্কার Error দেখাবে।

বর্তমান অবস্থা**: Implemented**।

Exception System (Pause)

Framework-এর সকল Error-এর জন্য একটি Standard Exception Architecture থাকবে।
যাতে—

- Provider Error
- Configuration Error
- Tool Error
- Memory Error
- Validation Error

সবগুলো একটি Standard Format অনুসরণ করে।

এর ফলে Error Handling পুরো Framework জুড়ে একই রকম থাকবে।

বর্তমান অবস্থা**: Future Implementation**।

Logging System (pause)

Framework-এর প্রতিটি গুরুত্বপূর্ণ Event Logging System-এর মাধ্যমে সংরক্ষিত হবে।
যেমন—

- Startup
- Provider Loading
- Tool Execution
- Memory Access
- API Request
- Warning
- Error

Logging System Debugging এবং Monitoring-এর ভিত্তি হবে।
বর্তমান অবস্থা**: Future Implementation**।

Lifecycle Manager (pause)

Framework-এর বিভিন্ন Component-এর জীবনচক্র (Lifecycle) নিয়ন্ত্রণ করবে।
যেমন—

- Initialize
- Ready
- Running
- Shutdown
- Cleanup

সব Component একই Lifecycle অনুসরণ করবে।
বর্তমান অবস্থা**: Future Implementation**।

Registry System (pause)

Registry Framework-এর বিভিন্ন Component-এর একটি কেন্দ্রীয় তালিকা (Registry) হিসেবে কাজ করবে।
যেমন—

- Provider Registry
- Tool Registry
- Memory Registry
- Agent Registry

নতুন Component যোগ করলে Framework সহজেই সেটি খুঁজে ব্যবহার করতে পারবে।
বর্তমান অবস্থা**: Future Implementation**।

Dependency Rules

Core Foundation-এর জন্য কিছু কঠোর নিয়ম থাকবে।

- Core কখনও Tool Layer-এর উপর নির্ভর করবে না।
- Core কখনও Agent Layer-এর উপর নির্ভর করবে না।
- Core কখনও Memory Layer-এর উপর নির্ভর করবে না।
- Core শুধুমাত্র Python Standard Library এবং অনুমোদিত Shared Library ব্যবহার করবে।
- Framework-এর অন্য সব Layer Core ব্যবহার করতে পারবে, কিন্তু Core তাদের ব্যবহার করবে না।

এতে Dependency Direction সবসময় একমুখী থাকবে এবং Circular Dependency তৈরি হবে না।

Current Implementation

বর্তমানে Core Foundation-এর যেসব অংশ সম্পন্ন হয়েছে—

- Project Structure
- Configuration System
- Provider Manager

- Provider Base Architecture
- Runtime Configuration
- Startup Validation
- Provider Health Check
- Unit Test Foundation

এই অংশগুলো বর্তমানে IRON BOLT-এর কার্যকর ভিত্তি হিসেবে কাজ করছে।

Future Implementation

Core Foundation ভবিষ্যতে আরও সমৃদ্ধ হবে।
ধাপে ধাপে যুক্ত হবে—

- Unified Exception System
- Logging Framework
- Lifecycle Manager
- Component Registry
- Event System
- Dependency Injection Support
- Metrics Collection
- Performance Monitoring Utilities

এই Component-গুলো যুক্ত হলেও Core Foundation-এর মূল নীতি অপরিবর্তিত থাকবে: এটি কখনও **Business Logic** বহন করবে না**;** এটি শুধুমাত্র **Framework Infrastructure** প্রদান করবে।

Chapter 4 — Development Philosophy

Development Philosophy

Why does IRON BOLT follow a Development Philosophy?

Every software project needs rules. Without clear principles, developers often add features randomly, change architecture frequently, or write code that only works for the current situation. As the project grows, these shortcuts create technical debt, make maintenance difficult, and slow down future development.

IRON BOLT is designed to become a long-term AI platform. Because of that, every component must be built with consistency, maintainability, and scalability in mind.
This chapter defines the engineering principles that will guide every future decision in the project.

Core Philosophy

The main philosophy of IRON BOLT is simple:

> Build a small but solid foundation first. Expand only after the foundation becomes stable.

Instead of implementing hundreds of features at the beginning, IRON BOLT focuses on creating reliable building blocks.

Every new system should be built on top of an already tested and stable system.

Build Layer by Layer

IRON BOLT follows a layered architecture.

Each layer has a specific responsibility.

A layer should never try to perform the responsibility of another layer.

For example,

- Provider System talks to AI models.
- Memory System stores and retrieves memories.
- Tool System executes external tools.
- Agent System makes decisions.
- Workflow Engine coordinates long-running workflows.

Each layer depends only on the layers it actually needs.
This separation makes the project easier to understand, debug, and extend.

Single Responsibility Principle

Every module should have one clear responsibility.

A component that tries to solve multiple unrelated problems quickly becomes difficult to maintain.

Examples:

- ProviderManager manages providers.
- ToolManager manages tools.

- MemoryManager manages memory.
- AgentEngine controls agent execution.

None of these components should contain the logic of another component.
Keeping responsibilities separate reduces complexity and improves maintainability.

Modular by Default

Every feature in IRON BOLT should be designed as an independent module whenever possible.

A module should be removable, replaceable, or upgradeable without affecting the rest of the system.

For example,
A new AI provider should be added without modifying the Memory System.

A new Tool should be added without changing the Agent Engine.

This modular approach keeps future development clean and predictable.

Expand Without Breaking Existing Code

New features should extend the project instead of modifying stable code.

Whenever possible:

- Add new modules.
- Add new implementations.
- Avoid rewriting stable components.

This philosophy reduces regression bugs and makes updates safer.

Configuration Over Hardcoding

System behavior should come from configuration instead of hardcoded values.

Examples include:

- API Keys
- Default Provider
- Default Model
- Timeouts
- Retry Limits
- Feature Flags

Changing configuration should not require changing source code.

This makes deployment easier and allows the same codebase to work in different environments.

Start Simple, Scale Later

IRON BOLT intentionally avoids unnecessary complexity during early development.

If a feature is not required today, it should not be implemented today.

The roadmap focuses on:

- Building the minimum stable architecture first.
- Validating the design.
- Expanding only when a real need appears.

This prevents over-engineering.

Future Features Remain Independent

Many advanced systems are planned for future development, such as:

- Memory Ranking
- Multi-Agent Collaboration
- Planning Engine
- Workflow Engine
- Plugin System
- MCP Integration
- Knowledge Graph
- Event System

These systems are intentionally postponed.

The current architecture is designed so they can be added later without major restructuring.

Future growth should happen through extension, not replacement.

Test Before Trust

Every important component should be tested before it becomes part of the stable architecture.

Testing helps ensure that new changes do not break existing functio nality.

The project values:

- Unit Tests
- Integration Tests
- Runtime Validation

A feature is considered complete only after it works reliably under testing.

Documentation First

Documentation is treated as a core part of development, not an afterthought.

Before implementing a major system, its purpose, architecture, responsibilities, and expected behavior should be clearly documented.

Good documentation helps:

- understand the project faster,
- reduce development mistakes,
- maintain architectural consistency,
- make future contributions easier.

For this reason, IRON BOLT's documentation evolves alongside its codebase.

Production-Oriented Mindset

Although IRON BOLT is currently under development, every architectural decision is made with production deployment in mind.

This means the project prioritizes:

- Clean architecture
- Predictable behavior
- Error handling
- Scalability
- Maintainability
- Extensibility

Rather than building temporary solutions, the goal is to create components that can continue to serve the project as it grows.

Current Development Scope

The current development phase focuses only on building the core foundation.

This includes:

- Core infrastructure
- Provider System
- Configuration System
- Basic project architecture

Advanced AI capabilities, autonomous reasoning, workflow automation, plugins, and other high-level systems are intentionally deferred to later phases of the roadmap.

This phased approach keeps development focused and ensures that every layer becomes stable before the next one is introduced.

Philosophy Summary

In short, every development decision in IRON BOLT should follow these principles:

- Build the foundation before adding features.
- One component, one responsibility.
- Keep systems modular and independent.
- Prefer extension over modification.
- Use configuration instead of hardcoding.
- Avoid unnecessary complexity.
- Write documentation before major implementation.
- Test every important component.
- Design for long-term scalability.
- Build today's requirements while preparing for tomorrow's expansion.

These principles will guide every chapter of this roadmap and ensure that IRON BOLT grows into a stable, maintainable, and production-ready AI platform.

Chapter 5: Complete Development Roadmap (Phase by Phase)

উদ্দেশ্য

এই Chapter-এ IRON BOLT কীভাবে বর্তমান অবস্থান থেকে একটি পূর্ণাঙ্গ Production-Ready AI AssistaReasoning (Pause. next vesion)ত হবে, তার সম্পূর্ণ Development Roadmap ব্যাখ্যা করা হয়েছে।

এটি শুধুমাত্র একটি Task List নয়। বরং প্রতিটি Phase-এর লক্ষ্য, কেন সেই Phase প্রয়োজন, কোন Component তৈরি হবে, কোন Component-এর উপর নির্ভর করবে এবং কোন ক্রমে Implementation হবে—সবকিছু এখানে উল্লেখ করা হয়েছে।

এই Roadmap অনুসরণ করলে Project-এর Architecture সবসময় পরিষ্কার থাকবে এবং ভবিষ্যতে নতুন Feature যোগ করলেও Foundation পরিবর্তন করতে হবে না।

Development Strategy

IRON BOLT Bottom-Up Architecture অনুসরণ করে তৈরি হবে।

অর্থাৎ, আগে Foundation তৈরি হবে, তারপর তার উপর আরও উচ্চ স্তরের System তৈরি হবে।

কারণ একটি AI System-এর সবচেয়ে বড় শক্তি নতুন Feature নয়, বরং একটি শক্তিশালী Core Architecture।

এই কারণে প্রতিটি নতুন Layer আগের Layer-এর উপর দাঁড়িয়ে তৈরি হবে।

বর্তমান অবস্থা (Completed)

বর্তমানে Project-এর মূল Foundation তৈরি হয়েছে।

এর মধ্যে রয়েছে—

- Project Structure
- Configuration System
- Provider Interface
- Provider Manager
- Base Provider
- OpenAI Provider
- Groq Provider
- Startup Validation
- Runtime Configuration
- Initial Testing Framework

এই অংশগুলো IRON BOLT-এর ভিত্তি হিসেবে কাজ করছে।

এখন থেকে Project ধাপে ধাপে Intelligent AI Operating System-এ রূপান্তরিত হবে।

Phase 1 — Core Foundation

উদ্দেশ্য

IRON BOLT-এর সমস্ত Component যেন একই নিয়মে কাজ করতে পারে, সেই ভিত্তি তৈরি করা।

এই Phase-এর পরে Project-এর Core Structure স্থায়ী হবে।

এখানে তৈরি হবে

- Core Layer
- Shared Utilities
- Base Interfaces
- Common Types
- Error Handling System
- Logging System
- Event Infrastructure (বর্তমানে প্রয়োজনীয় অংশ)
- Internal Communication Rules

কেন

যদি প্রতিটি Module নিজের মতো করে কাজ করে, তাহলে ভবিষ্যতে Project Maintain করা কঠিন হয়ে যাবে।

Core Layer সব Component-এর জন্য Common Language তৈরি করবে।

এরপর কারা এটি ব্যবহার করবে

- Provider System
- Memory System
- Tool System
- Agent System
- Workflow Engine

Phase 2 — Provider System

উদ্দেশ্য

যে কোনো AI Model-এর সাথে একই Interface ব্যবহার করে যোগাযোগ করা।

এখানে তৈরি হবে

- Provider Manager উন্নত করা
- Provider Registry
- Model Discovery
- Model Capability Information
- Streaming Support
- Retry System
- Health Check
- Failover Logic
- Multi Provider Support

কেন

IRON BOLT যেন একটি নির্দিষ্ট AI কোম্পানির উপর নির্ভরশীল না থাকে।

যেকোনো নতুন Provider ভবিষ্যতে খুব সহজে যোগ করা যাবে।

Dependency

এই Phase সম্পূর্ণভাবে Core Layer-এর উপর নির্ভর করবে।

Phase 3 — Memory System

উদ্দেশ্য

AI যেন শুধু বর্তমান Message নয়, বরং Context মনে রাখতে পারে।

এখানে তৈরি হবে

- Conversation Memory
- Session Memory
- Context Builder
- Memory Storage Interface
- Memory Retrieval
- Memory Compression

- Context Window Management

কেন

LLM নিজে স্থায়ীভাবে কিছু মনে রাখে না।

Memory System AI-কে দীর্ঘ Conversation বুঝতে সাহায্য করবে।

Dependency

Memory System Provider System ব্যবহার করে Context তৈরি করবে।

Phase 4 — Tool System

উদ্দেশ্য

AI যেন শুধু উত্তর না দিয়ে বাস্তব কাজও করতে পারে।

এখানে তৈরি হবে

- Tool Manager
- Base Tool
- Tool Registry
- Tool Executor
- Tool Validation
- Tool Result Processing

পরবর্তীতে বিভিন্ন Tool যুক্ত হবে।

যেমন—

- Python Tool
- Terminal Tool
- File Tool
- Search Tool
- Browser Tool
- API Tool

কেন

Tool ছাড়া AI বাস্তব জগতের কোনো কাজ করতে পারে না।

Tool System IRON BOLT-এর Action Layer হবে।

Dependency

- Core Layer
- Memory System

Phase 5 — Agent System

উদ্দেশ্য

IRON BOLT-কে একটি Autonomous AI Agent-এ রূপান্তর করা।

এখানে তৈরি হবে

- Agent
- Task Execution Loop
- Decision Controller
- Agent State
- Goal Tracking
- Response Generator

কেন

Provider শুধু Model-এর সাথে কথা বলে।

Agent সিদ্ধান্ত নেয় কী করতে হবে।

Dependency

- Provider
- Memory
- Tool

Phase 6 — Workflow Engine (Pause. next vesion)

উদ্দেশ্য

একাধিক Step ধারাবাহিকভাবে Execute করা।

এখানে তৈরি হবে

- Workflow Definition
- Workflow Runner
- Step Executor
- Conditional Execution
- Retry
- Workflow Context

কেন

বাস্তব Task এক ধাপে শেষ হয় না।

Workflow Engine অনেকগুলো Step একসাথে পরিচালনা করবে।

Phase 7 — Planning & Reasoning (Pause. next vesion. Implement with phase 6)

উদ্দেশ্য

বড় সমস্যাকে ছোট ছোট Task-এ ভাগ করা।

এখানে তৈরি হবে

- Planner

- Goal Analyzer
- Task Breakdown
- Dependency Planning
- Reasoning Controller
- Execution Strategy

কেন

AI যেন নিজে পরিকল্পনা করতে পারে।

Phase 8 — Prompt Management (Pause. next vesion)

উদ্দেশ্য

Prompt-কে Structured এবং Reusable করা।

এখানে তৈরি হবে

- Prompt Templates
- Prompt Builder
- System Prompt
- Dynamic Prompt Injection
- Prompt Versioning

কেন

Project বড় হলে Prompt-ও Software Component হয়ে যায়।

Phase 9 — Plugin & MCP System (Pause. next vesion)

উদ্দেশ্য

IRON BOLT-এর Capability বাড়ানো।

এখানে তৈরি হবে

- Plugin Loader
- Plugin Registry
- Plugin Lifecycle
- MCP Client
- MCP Server Communication

কেন

Core Code পরিবর্তন না করেই নতুন Feature যোগ করা যাবে।

Phase 10 — Security & Permission (Pause. next vesion)

উদ্দেশ্য

AI কী করতে পারবে এবং কী পারবে না, তা নিয়ন্ত্রণ করা।

এখানে তৈরি হবে

- Permission System
- Tool Permission
- Provider Permission
- File Permission
- User Confirmation
- Security Policies

কেন

Autonomous AI-এর জন্য নিরাপত্তা অত্যন্ত গুরুত্বপূর্ণ।

Phase 11 — Execution Flow (Pause. next vesion)

উদ্দেশ্য

User Request শুরু থেকে শেষ পর্যন্ত কীভাবে চলবে তা নির্ধারণ করা।

এখানে তৈরি হবে

- Request Pipeline
- Context Pipeline
- Tool Pipeline
- Response Pipeline

এই Phase-এর পরে সম্পূর্ণ Request Flow স্থায়ী হবে।

Phase 12 — Architecture Refinement (Pause. next vesion)

উদ্দেশ্য

সম্পূর্ণ System-কে Production Quality-তে নিয়ে যাওয়া।

এখানে উন্নত করা হবে

- Performance
- Scalability
- Caching
- Monitoring
- Testing
- Optimization
- Documentation

এই Phase-এ নতুন Feature-এর চেয়ে বিদ্যমান Architecture আরও শক্তিশালী করা হবে।

Phase 13 — Future Expansion (Pause. next version)

Production Version সম্পূর্ণ হওয়ার পরে ভবিষ্যতে IRON BOLT-এ আরও উন্নত Capability যোগ করা হবে।

সম্ভাব্য Expansion-এর মধ্যে রয়েছে—

- Multi-Agent Collaboration
- Distributed Execution
- Advanced Planning
- Long-Term Knowledge System
- Self Improvement
- Learning Pipeline
- Cloud Deployment
- Team Collaboration
- Visual Workflow Builder
- Enterprise Features

এই অংশগুলো বর্তমান Development-এর অংশ নয়। এগুলো ভবিষ্যতের Expansion হিসেবে পরিকল্পিত থাকবে।

Implementation Order

IRON BOLT নিম্নোক্ত ক্রমে তৈরি হবে—

1. Core Layer
2. Provider System
3. Memory System
4. Tool System
5. Agent System
6. Workflow Engine
7. Planning & Reasoning
8. Prompt Management
9. Plugin & MCP System
10. Security & Permissions
11. Execution Flow

12. Architecture Refinement
13. Future Expansion

এই ক্রম ইচ্ছাকৃতভাবে নির্ধারণ করা হয়েছে। কারণ প্রতিটি Phase আগের Phase-এর উপর নির্ভরশীল। Foundation সম্পূর্ণ না করে পরবর্তী Layer তৈরি করলে Architecture জটিল ও অস্থিতিশীল হয়ে যেতে পারে।

এই **Chapter-**এর সারসংক্ষেপ

এই Chapter-এ IRON BOLT-এর সম্পূর্ণ Development Journey উপস্থাপন করা হয়েছে। বর্তমান Foundation থেকে শুরু করে একটি পূর্ণাঙ্গ AI Operating System হয়ে ওঠার প্রতিটি ধাপ এখানে সংজ্ঞায়িত করা হয়েছে।

পরবর্তী Chapter থেকে প্রতিটি Phase-এর অভ্যন্তরীণ Architecture বিস্তারিতভাবে ব্যাখ্যা করা হবে। প্রথমে আলোচনা করা হবে **Core Layer**, কারণ Project-এর বাকি সব System এই Foundation-এর উপর নির্মিত হবে।

Chapter 6 — Core Layer

উদ্দেশ্য

Core Layer হলো IRON BOLT-এর Foundation।

পুরো Project-এর প্রতিটি Component এই Layer-এর উপর নির্ভর করবে।

যেমন একটি Operating System-এর Kernel পুরো System-কে নিয়ন্ত্রণ করে, ঠিক তেমনি IRON BOLT-এর Core Layer সমস্ত Module-এর জন্য একটি Common Infrastructure প্রদান করবে।

Provider, Memory, Tool, Agent, Workflow, Planning কিংবা Prompt Management—কোনো Module-ই সরাসরি একে অপরের উপর নির্ভর করবে না। তারা সবাই Core Layer-এর মাধ্যমে নিজেদের কাজ করবে।

এর ফলে System হবে—

- Modular
- Maintainable

- Testable
- Scalable
- Production Ready

Core Layer-এর সবচেয়ে বড় দায়িত্ব হলো পুরো Project-এর Common Logic এক জায়গায় রাখা, যাতে একই Code বারবার লিখতে না হয় এবং ভবিষ্যতে নতুন Feature যোগ করাও সহজ হয়।

কেন **Core Layer** দরকার

যদি Core Layer না থাকে, তাহলে প্রতিটি Module-কে নিজস্ব Configuration, Logging, Error Handling, Validation, Event Processing এবং Communication System তৈরি করতে হবে।

ফলাফল হবে—

- একই Logic বহু জায়গায় লেখা হবে
- Code Maintain করা কঠিন হবে
- Bug বৃদ্ধি পাবে
- নতুন Feature যোগ করা কঠিন হবে
- Module গুলোর মধ্যে Strong Dependency তৈরি হবে

Core Layer এই সমস্যাগুলো সম্পূর্ণভাবে সমাধান করে।

Core Layer-এর দায়িত্ব

Core Layer কোনো AI Feature তৈরি করে না।

এটি AI-কে চালানোর জন্য প্রয়োজনীয় Infrastructure তৈরি করে।

এর প্রধান কাজ হলো—

- Configuration পরিচালনা করা
- Logging System প্রদান করা
- Error Handling করা
- Runtime Context পরিচালনা করা
- Shared Utilities রাখা
- Validation করা
- Communication-এর ভিত্তি তৈরি করা
- Project-এর Common Services প্রদান করা

Core Layer Architecture

Core Layer-এর অধীনে নিম্নলিখিত Module থাকবে।

core/

├── config.py
├── runtime.py
├── exceptions.py
├── logger.py
├── validation.py
├── startup_validation.py
├── constants.py
├── context.py
├── registry.py
├── events.py
├── metrics.py
├── lifecycle.py
└── utils/

বর্তমান Version-এ সব Module Implement করা হবে না।

কিছু Module এখনই প্রয়োজন, আর কিছু ভবিষ্যতের জন্য সংরক্ষিত থাকবে।

বর্তমান **Version-**এ **Implement** হবে

Config System

উদ্দেশ্য

পুরো Project-এর Configuration এক জায়গা থেকে পরিচালনা করা।

যেমন—

- API Key
- Default Provider

- Default Model
- Timeout
- Retry
- Debug Mode
- Environment Variable
- Runtime Settings

Provider System, Tool System, Memory System—সব Module এখান থেকে Configuration গ্রহণ করবে।

বর্তমানে এটি Core Layer-এর সবচেয়ে গুরুত্বপূর্ণ Module।

Startup Validation

Project শুরু হওয়ার আগে System সঠিকভাবে প্রস্তুত কিনা তা পরীক্ষা করবে।

যেমন—

- API Key আছে কি না
- Required Configuration আছে কি না
- Invalid Setting আছে কি না
- Environment প্রস্তুত কি না

কোনো সমস্যা থাকলে শুরুতেই পরিষ্কার Error দেখাবে।

Exception System

System-এর সব Error-এর জন্য একটি Standard Structure প্রদান করবে।

যেমন—

- Provider Error
- Tool Error
- Memory Error
- Validation Error
- Permission Error

এতে Error Handling পুরো Project জুড়ে একই নিয়ম অনুসরণ করবে।

Logging System

System-এর প্রতিটি গুরুত্বপূর্ণ কাজ Record করবে।

যেমন—

- কোন Provider ব্যবহার হয়েছে
- কোন Tool চালানো হয়েছে
- Response আসতে কত সময় লেগেছে
- কোথায় Error হয়েছে

Production Environment-এ Debugging-এর জন্য এটি অত্যন্ত গুরুত্বপূর্ণ।

Validation System

User Input এবং Internal Data সঠিক কিনা তা যাচাই করবে।

যেমন—

- Empty Input
- Invalid Configuration
- Unsupported Provider
- Missing Parameter

এর ফলে System আরও নির্ভরযোগ্য হবে।

Runtime Context

বর্তমান Execution-এর প্রয়োজনীয় তথ্য সংরক্ষণ করবে।

যেমন—

- Active Provider
- Current Request
- Session Information
- Runtime State

ভবিষ্যতে Workflow Engine এবং Agent System এই Context ব্যবহার করবে।

Utility Module

বারবার ব্যবহৃত Common Function এখানে থাকবে।

যেমন—

- File Utility
- JSON Utility
- String Utility
- Time Utility
- Helper Function

এর ফলে একই Code একাধিক জায়গায় লিখতে হবে না।

Future Version-এ Implement হবে

বর্তমান AR1 Version-এ নিচের Module গুলো থাকবে না।

এগুলো ভবিষ্যতের জন্য পরিকল্পিত।

Registry

System-এর বিভিন্ন Component Register করবে।

যেমন—

- Provider Registry
- Tool Registry
- Agent Registry

এর মাধ্যমে নতুন Component Dynamicভাবে যুক্ত করা যাবে।

Event System

System-এর বিভিন্ন Event পরিচালনা করবে।

যেমন—

- Tool Finished
- Memory Updated
- Workflow Started
- Provider Changed

Future Automation-এর ভিত্তি হবে এই Module।

Metrics

System Performance পরিমাপ করবে।

যেমন—

- Response Time
- Token Usage
- Memory Usage
- Success Rate
- Failure Rate

Production Monitoring-এর জন্য এটি গুরুত্বপূর্ণ।

Lifecycle Manager

প্রতিটি Component-এর জীবনচক্র নিয়ন্ত্রণ করবে।

যেমন—

- Initialize
- Start
- Ready
- Running
- Shutdown
- Cleanup

বড় System-এ Resource Management সহজ করার জন্য এটি ব্যবহার করা হবে।

Core Layer কার সাথে **Connected** থাকবে

Core Layer পুরো Project-এর Common Infrastructure হওয়ায় এটি প্রায় সব Layer-এর সাথে সংযুক্ত থাকবে।

Core Layer
 │
 ├── Provider System
 ├── Memory System
 ├── Tool System
 ├── Agent System
 ├── Workflow Engine
 ├── Planning System
 ├── Prompt Management
 ├── Plugin System
 └── Security Layer

এখানে একটি গুরুত্বপূর্ণ বিষয় হলো, অন্য Module-গুলো নিজেদের মধ্যে সরাসরি নির্ভরশীল না হয়ে Core Layer-এর মাধ্যমে Common Service ব্যবহার করবে। এতে System-এর Coupling কমবে এবং Maintainability বৃদ্ধি পাবে।

Implementation Order

Core Layer-এর Development নিম্নলিখিত ক্রমে করা হবে।

Phase 1

- Config System
- Startup Validation
- Exception System

এগুলো ছাড়া অন্য Module তৈরি করা নিরাপদ নয়।

Phase 2

- Logging System
- Validation System
- Runtime Context
- Utility Module

এগুলো পুরো Project-এর দৈনন্দিন Execution পরিচালনা করবে।

Phase 3 (Future)

- Registry
- Event System
- Metrics
- Lifecycle Manager

এই অংশগুলো AR3 / AR4-এর মতো বড় সংস্করণে যোগ করা হবে, যখন System-এ একাধিক Agent, Tool এবং Workflow একসাথে চলবে।

এই **Chapter-**এর সারসংক্ষেপ

Core Layer হলো IRON BOLT-এর ভিত্তি। এটি কোনো AI Feature নয়; বরং পুরো System-এর Common Infrastructure। এর মাধ্যমে Configuration, Logging, Validation, Exception Handling, Runtime Context এবং Shared Utility এক জায়গায় পরিচালিত হবে। ভবিষ্যতের Registry, Event System, Metrics এবং Lifecycle Manager এই ভিত্তির উপর নির্মিত হবে। একটি স্থিতিশীল, সম্প্রসারণযোগ্য এবং Production-Ready AI Platform তৈরির জন্য Core Layer-ই প্রথম এবং সবচেয়ে গুরুত্বপূর্ণ ধাপ।

Chapter 7: Provider System

Provider System

Provider System হলো IRON BOLT-এর এমন একটি Layer যার মাধ্যমে পুরো Framework বিভিন্ন Large Language Model (LLM)-এর সাথে যোগাযোগ করে।

সহজভাবে বললে, IRON BOLT নিজে কোনো AI Model নয়। এটি একটি AI Runtime Framework। তাই বাস্তবে উত্তর তৈরি করার জন্য এটিকে কোনো না কোনো AI Model ব্যবহার করতে হবে।

এই Model হতে পারে—

- OpenAI
- Groq
- Google Gemini
- Anthropic Claude
- Ollama
- Local LLM
- অথবা ভবিষ্যতে যুক্ত হওয়া অন্য যেকোনো Provider।

Provider System এই সমস্ত Model-কে একটি একক (Unified) Interface-এর মাধ্যমে পরিচালনা করে।

এর ফলে Framework-এর বাকি অংশ কখনোই জানতে পারে না কোন কোম্পানির Model ব্যবহার হচ্ছে।

কেন **Provider System** তৈরি করা হয়েছে

যদি প্রতিটি জায়গায় সরাসরি OpenAI বা Groq API ব্যবহার করা হতো, তাহলে ভবিষ্যতে Provider পরিবর্তন করা অত্যন্ত কঠিন হয়ে যেত।

উদাহরণ হিসেবে,

যদি Project-এর শত শত জায়গায় OpenAI API Call লেখা থাকে, পরে Gemini ব্যবহার করতে চাইলে পুরো Project পরিবর্তন করতে হবে।

এটি Production Software-এর জন্য গ্রহণযোগ্য নয়।

এই সমস্যা সমাধানের জন্য Provider Layer তৈরি করা হয়েছে।

এটি Framework এবং AI Model-এর মাঝে একটি Abstraction Layer হিসেবে কাজ করে।

Framework শুধু Provider-এর সাথে কথা বলে।

Provider বাস্তবে কোন AI Service ব্যবহার করবে, সেটি Provider নিজেই সিদ্ধান্ত নেয়।

Provider Layer-এর দায়িত্ব

Provider System-এর প্রধান দায়িত্বগুলো হলো—

- AI Model-এর সাথে Connection তৈরি করা
- API Key ব্যবহার করে Authentication করা
- কোন Model ব্যবহার হবে তা নির্বাচন করা
- Prompt পাঠানো
- Response গ্রহণ করা
- Error Handle করা
- Retry করা (Future)
- Streaming Support দেওয়া (Future)
- Usage Information সংগ্রহ করা (Future)
- Cost Tracking করা (Future)
- Health Check করা
- Available Model List প্রদান করা

বর্তমান Architecture

বর্তমানে Provider Layer-এর Architecture নিম্নরূপ—

providers/

base.py

manager.py

openai_provider.py

groq_provider.py

__init__.py

বর্তমানে Framework প্রধানত Groq Provider ব্যবহার করে।

Default Model হিসেবে ব্যবহৃত হচ্ছে—

gpt-oss-120b

এটি বর্তমানে IRON BOLT-এর Primary Model।

Base Provider

Base Provider হলো পুরো Provider System-এর Foundation।

এটি একটি Abstract Interface।

এর কাজ হচ্ছে—

Framework-এর সব Provider যেন একই নিয়ম অনুসরণ করে।

যে Provider-ই তৈরি করা হোক না কেন—

- OpenAI
- Groq
- Gemini
- Claude
- Ollama

সবাইকে Base Provider-এর Interface অনুসরণ করতে হবে।

এর ফলে Framework-এর অন্য Layer কখনো Provider অনুযায়ী আলাদা Logic লিখতে হয় না।

Provider Manager

Provider Manager হলো পুরো Provider Layer-এর Controller।

Framework কখনো সরাসরি OpenAI Provider বা Groq Provider ব্যবহার করে না।

সবসময় Provider Manager-এর মাধ্যমে Provider Access করা হয়।

Provider Manager-এর কাজ—

- Provider Register করা
- Provider Initialize করা

- Default Provider নির্বাচন করা
- Active Provider Return করা
- Provider Change করা
- Provider Health Check করা
- ভবিষ্যতে Multiple Provider Manage করা

এটি পুরো Provider Layer-এর কেন্দ্রীয় অংশ।

OpenAI Provider

OpenAI Provider শুধুমাত্র OpenAI API-এর সাথে যোগাযোগ করার জন্য দায়ী।

এর দায়িত্ব—

- API Key ব্যবহার করা
- Request পাঠানো
- Response গ্রহণ করা
- OpenAI Error Handle করা
- Model Select করা

Framework-এর অন্য কোনো Layer OpenAI সম্পর্কে কিছু জানে না।

Groq Provider

Groq Provider বর্তমানে Project-এর প্রধান Provider।

এটি Groq API ব্যবহার করে Model Run করে।

বর্তমানে Default Execution এই Provider-এর মাধ্যমে হয়।

Groq Provider-এর মাধ্যমে—

- Prompt পাঠানো হয়
- Response পাওয়া যায়
- Health Check করা যায়
- Model Information পাওয়া যায়

বর্তমান Version-এর প্রধান AI Runtime এই Provider-এর উপর নির্ভরশীল।

Provider Initialization

Framework Start হওয়ার সময় Provider Initialize করা হয়।

এর সময়—

- Configuration Load হয়
- API Key যাচাই করা হয়
- Provider তৈরি হয়
- Provider Ready করা হয়
- Default Provider নির্বাচন করা হয়

Initialization সফল না হলে Framework Start হবে না।

কারণ Provider ছাড়া AI Runtime কাজ করতে পারবে না।

Provider Selection

বর্তমানে একটি Default Provider ব্যবহার করা হয়।

Future Version-এ User অথবা System Runtime অনুযায়ী Provider নির্বাচন করতে পারবে।

যেমন—

- Coding Task → OpenAI
- Fast Response → Groq
- Long Reasoning → Claude
- Offline Mode → Ollama

অর্থাৎ Task অনুযায়ী Provider পরিবর্তন করা সম্ভব হবে।

Error Handling

Provider Layer শুধুমাত্র API Call করে না।

এটি বিভিন্ন ধরনের Error Handle করারও দায়িত্ব পালন করে।

যেমন—

- Invalid API Key
- Timeout
- Network Failure
- Rate Limit
- Unsupported Model
- Authentication Failure

Future Version-এ Retry Strategy এবং Automatic Recovery যুক্ত করা হবে।

বর্তমান Implementation

বর্তমানে Provider Layer-এ নিম্নলিখিত Feature রয়েছে—

- Provider Architecture
- Base Provider Interface
- Provider Manager
- OpenAI Provider
- Groq Provider
- Configuration Integration
- Startup Validation
- Health Check
- Model Initialization
- Default Provider Support

বর্তমান Version-এর জন্য এগুলোই যথেষ্ট।

Future Implementation

বর্তমান Version-এ ইচ্ছাকৃতভাবে কিছু Feature যুক্ত করা হয়নি।

পরবর্তী Version-এ যোগ করা হবে—

- Gemini Provider
- Claude Provider
- Ollama Provider

- Local Model Provider
- Multi Provider Execution
- Automatic Provider Switching
- Provider Priority
- Load Balancing
- Streaming Response
- Response Cache
- Cost Tracking
- Token Usage Tracking
- Retry Engine
- Circuit Breaker
- Provider Benchmark
- Model Capability Detection

এগুলো AR3 এবং পরবর্তী Version-এ ধাপে ধাপে যুক্ত হবে।

Provider System কোন কোন **Layer-**এর সাথে যুক্ত

Provider Layer সরাসরি যুক্ত থাকবে—

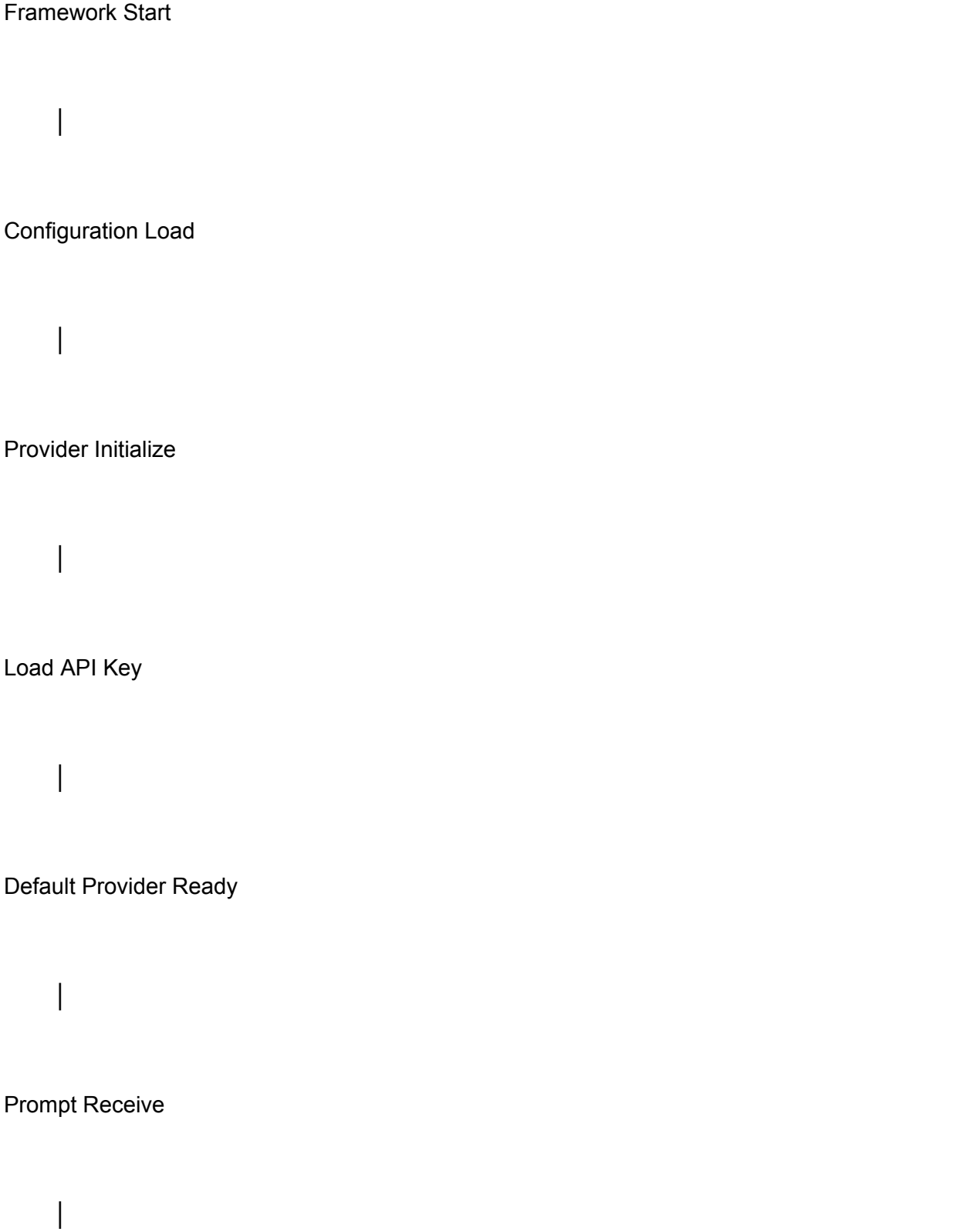
- Core Layer
- Configuration System
- Agent System
- Planning System
- Workflow Engine
- Prompt Management
- Memory System
- Tool System
- Execution Engine

Provider Layer Framework-এর প্রায় প্রতিটি গুরুত্বপূর্ণ Layer-এর সাথে সংযুক্ত।

কারণ AI Model ব্যবহার করার প্রয়োজন প্রায় সব Layer-এরই হয়।

Execution Flow

Provider System সাধারণভাবে নিচের ধাপে কাজ করে—


Framework Start
Configuration Load
Provider Initialize
Load API Key
Default Provider Ready
Prompt Receive

Selected Model Execute

|

Response Receive

|

Return To Framework

এটি বর্তমান Version-এর Provider Execution Flow।

সারসংক্ষেপ

Provider System হলো IRON BOLT-এর AI Communication Layer।

এটি Framework-কে নির্দিষ্ট কোনো AI কোম্পানির উপর নির্ভরশীল হতে দেয় না। বরং একটি Unified Interface-এর মাধ্যমে বিভিন্ন AI Provider পরিচালনা করে।

বর্তমান Version-এ এর মূল লক্ষ্য একটি স্থিতিশীল, নির্ভরযোগ্য এবং সহজে সম্প্রসারণযোগ্য Provider Architecture তৈরি করা। ভবিষ্যতের সংস্করণগুলোতে এই Layer আরও শক্তিশালী হবে, যেখানে একাধিক Provider, Intelligent Routing, Streaming, Cost Optimization এবং Automatic Provider Selection-এর মতো উন্নত সুবিধা যুক্ত হবে।

Chapter 8: Memory System

উদ্দেশ্য

Memory System হলো IRON BOLT-এর এমন একটি অংশ, যা AI-কে প্রতিটি কথোপকথনকে সম্পূর্ণ নতুনভাবে শুরু করতে বাধ্য না করে পূর্বের গুরুত্বপূর্ণ তথ্য মনে রাখতে সাহায্য করবে।

সাধারণ LLM-এর একটি সীমাবদ্ধতা হলো, তারা শুধুমাত্র বর্তমান context window-এর মধ্যে থাকা তথ্য ব্যবহার করতে পারে। Conversation শেষ হয়ে গেলে বা context limit অতিক্রম করলে সেই তথ্য আর ব্যবহার করা যায় না।

IRON BOLT-এর Memory System এই সমস্যার সমাধান করবে।

এর মাধ্যমে AI ব্যবহারকারীর অনুমতি অনুযায়ী গুরুত্বপূর্ণ তথ্য সংরক্ষণ করবে, ভবিষ্যতে প্রয়োজন হলে সেই তথ্য পুনরুদ্ধার করবে এবং আরও ব্যক্তিগত, ধারাবাহিক ও বুদ্ধিমান অভিজ্ঞতা প্রদান করবে।

Memory System কেন প্রয়োজন?

যদি Memory না থাকে, তাহলে প্রতিবার AI-কে একই তথ্য আবার বলতে হবে।

উদাহরণ:

- User-এর নাম
- User-এর পছন্দ
- চলমান Project
- পূর্বের সিদ্ধান্ত
- দীর্ঘমেয়াদী Goal
- Workflow-এর Progress

Memory থাকার ফলে AI প্রতিবার নতুনভাবে শুরু না করে আগের তথ্যের উপর ভিত্তি করে কাজ চালিয়ে যেতে পারবে।

এটি IRON BOLT-কে শুধুমাত্র একটি Chat Assistant নয়, বরং একটি দীর্ঘমেয়াদী Intelligent Assistant-এ পরিণত করবে।

Memory System-এর দায়িত্ব

Memory System-এর মূল কাজগুলো হবে—

- গুরুত্বপূর্ণ তথ্য সংরক্ষণ করা।
- প্রয়োজন হলে তথ্য পুনরুদ্ধার করা।
- অপ্রয়োজনীয় তথ্য বাদ দেওয়া।
- Memory Update করা।
- Duplicate Memory এড়ানো।
- Memory Search করা।
- Memory Delete করা।
- Conversation থেকে গুরুত্বপূর্ণ তথ্য শনাক্ত করা।
- User অনুযায়ী Memory আলাদা রাখা।
- অন্য Component-কে Memory সরবরাহ করা।

Memory System নিজে কোনো সিদ্ধান্ত নেবে না।

এটি শুধুমাত্র তথ্য সংরক্ষণ ও সরবরাহ করবে।

Memory-এর ধরন

IRON BOLT-এ সব তথ্য একইভাবে সংরক্ষণ করা হবে না।

ভিন্ন ধরনের তথ্য ভিন্নভাবে পরিচালনা করা হবে।

১. User Memory

ব্যবহারকারীর দীর্ঘমেয়াদী তথ্য।

যেমন—

- নাম
- ভাষা
- পছন্দ
- অপছন্দ
- শেখার লক্ষ্য
- ব্যক্তিগত Preference

এই তথ্য দীর্ঘ সময় পর্যন্ত সংরক্ষিত থাকতে পারে।

---

২. Conversation Memory

বর্তমান Conversation-এর গুরুত্বপূর্ণ তথ্য।

যেমন—

- বর্তমানে কী নিয়ে আলোচনা হচ্ছে
- Conversation-এর Flow
- সাম্প্রতিক সিদ্ধান্ত
- অস্থায়ী Context

Conversation শেষ হলে এর কিছু অংশ মুছে যেতে পারে, আবার গুরুত্বপূর্ণ অংশ Long-term Memory-তে সংরক্ষিত হতে পারে।

---

৩. Project Memory

কোনো Project সম্পর্কিত তথ্য।

যেমন—

- Project Structure
- Development Status
- Completed Phase
- Pending Task
- Architecture Decision

এটি দীর্ঘমেয়াদী Project Continuity বজায় রাখতে সাহায্য করবে।

---

৪. Task Memory

চলমান কাজ সম্পর্কিত তথ্য।

যেমন—

- বর্তমান Task
- Remaining Step
- Progress
- Execution State

Workflow Engine এবং Agent System এই Memory ব্যবহার করবে।

---

৫. Knowledge Memory (Future)

ভবিষ্যতে IRON BOLT নিজস্ব Knowledge Store তৈরি করতে পারবে।

যেখানে সংরক্ষণ করা যাবে—

- Documentation
- Notes
- Research
- Reference
- User-এর নিজস্ব তথ্য

বর্তমান AR1 Version-এ এটি থাকবে না।

Memory Lifecycle

Memory সবসময় একই অবস্থায় থাকবে না।

একটি Memory সাধারণত নিচের ধাপগুলো অতিক্রম করবে।

Conversation

↓

Information Detection

↓

Importance Evaluation

↓

Memory Creation

↓

Storage

↓

Retrieval

↓

Update

↓

Archive অথবা Delete

Memory Retrieval

শুধু Memory সংরক্ষণ করলেই হবে না।

প্রয়োজনের সময় দ্রুত সঠিক Memory খুঁজে বের করতে হবে।

Memory Retrieval-এর কাজ—

- Keyword Search
- Similar Meaning Search (Future)
- Recent Memory
- Project-based Memory
- User-based Memory

যে Component Memory চাইবে, Memory System সেটি সরবরাহ করবে।

Memory Update

একই তথ্য বারবার সংরক্ষণ করা হবে না।

যদি নতুন তথ্য পুরোনো তথ্যকে পরিবর্তন করে, তাহলে Memory Update হবে।

উদাহরণ—

পুরোনো Memory:

Preferred Language = English

নতুন তথ্য:

Preferred Language = Bangla

তাহলে পুরোনো Memory পরিবর্তন হয়ে নতুন তথ্য সংরক্ষিত হবে।

Memory Decision

এখানে memory decision নিবে। গুরুত্বপূর্ণ conversation সংরক্ষণ করা হবে।

যেমন:
Memory Decision করার সময় LLM-কে একটি আলাদা কাজ দেওয়া হবে। যেমন:

"এই Conversation বিশ্লেষণ করো। যদি এতে এমন কোনো তথ্য থাকে যা ভবিষ্যতের Conversation-এ ব্যবহারকারীর জন্য উপকারী হবে, তাহলে memory_candidate তৈরি করো। না হলে 'no_memory' ফেরত দাও।"

তারপর LLM সিদ্ধান্ত দেবে।

উদাহরণ:
Conversation ১
User: "আমার নাম মাহির।"
LLM বুঝবে এটি দীর্ঘমেয়াদী তথ্য।

```
{
  "save": true,
  "type": "profile",
  "memory": "User's preferred name is Mahir."
}
```

এটি Memory-তে যাবে।

Conversation ২
User: "আজ আমাকে Python শেখাও।"
এটি সাময়িক অনুরোধ।

```
{
  "save": false
}
```

Memory-তে যাবে না।

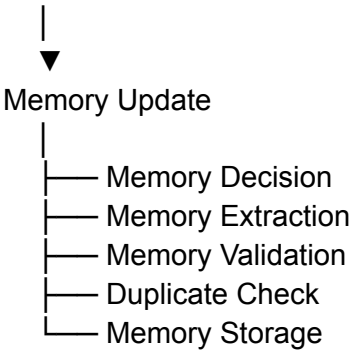
memory decision memory update এর একটা অংশ।
Memory Update

```
│
├── Memory Decision
│      │
│      ├── No → Stop
│      │
│      └── Yes
│
├── Memory Extraction
├── Memory Validation
├── Duplicate Check
├── Memory Storage
└── Index Update
```

- Memory Decision Engine → এই Conversation আদৌ Memory-তে যাবে কি না তা নির্ধারণ করবে।
- Memory Extractor → পুরো Conversation থেকে শুধু দরকারি তথ্য বের করবে।

- Memory Validator → তথ্যটি সঠিক, ডুপ্লিকেট নয় এবং সংরক্ষণের উপযোগী কিনা যাচাই করবে।
- Memory Storage → যাচাইকৃত তথ্য Database/Vector Store-এ সংরক্ষণ করবে।
- Memory Decision → এই Conversation থেকে কোনো Memory তৈরি করা দরকার কি না।
- Memory Extraction → কী তথ্য সংরক্ষণ করা হবে।
- Memory Validation → তথ্যটি বৈধ ও উপযুক্ত কি না।
- Duplicate Check → একই তথ্য আগে থেকেই আছে কি না।
- Memory Storage → তথ্য সংরক্ষণ করা।
- Index Update → ভবিষ্যতে দ্রুত খুঁজে পাওয়ার জন্য Index আপডেটRanking

অর্থাৎ সম্পর্কটি হলো:



Memory Ranking (Future)

সব Memory সমান গুরুত্বপূর্ণ নয়।

ভবিষ্যতে প্রতিটি Memory-এর Importance Score থাকতে পারে।

উদাহরণ—

High Importance

Medium Importance

Low Importance

এতে Retrieval আরও দ্রুত ও নির্ভুল হবে।

বর্তমানে এটি Implement করা হবে না।

Memory Manager

Memory System-এর মূল Controller হবে **Memory Manager**।

এর কাজ—

- Memory Save করা
- Memory Load করা
- Memory Update করা
- Memory Delete করা
- Memory Search করা
- Memory Retrieve করা
- Memory Provider-এর সাথে যোগাযোগ করা
- অন্য Component-কে Memory সরবরাহ করা

Memory Manager পুরো Memory Layer-এর Entry Point হিসেবে কাজ করবে।

Memory Storage

বর্তমান Version-এ Memory খুবই সহজভাবে সংরক্ষণ করা হবে।

পরবর্তীতে বিভিন্ন Storage Backend ব্যবহার করা যাবে।

সম্ভাব্য Storage—

- JSON
- SQLite
- PostgreSQL
- Redis
- Vector Database

Storage পরিবর্তন হলেও Memory Manager-এর Interface একই থাকবে।

ফলে অন্য Component-এ কোনো পরিবর্তন করতে হবে না।

Memory Provider (Future)

ভবিষ্যতে IRON BOLT একাধিক Memory Backend ব্যবহার করতে পারবে।

যেমন—

- Local Memory
- Cloud Memory
- Vector Memory

Memory Provider Layer এই বিভিন্ন Storage Backend-কে একটি একক Interface-এর মাধ্যমে পরিচালনা করবে।

বর্তমান AR1 Version-এ এটি থাকবে না।

Memory Search (Future)

বর্তমানে সাধারণ Search যথেষ্ট হবে।

ভবিষ্যতে উন্নত Search যুক্ত করা যেতে পারে।

যেমন—

- Semantic Search
- Vector Search
- Hybrid Search
- Similar Conversation Search

এগুলো Large Memory Database-এর জন্য উপযোগী হবে।

Memory Security

সব তথ্য সংরক্ষণ করা হবে না।

Memory System Privacy-কেও গুরুত্ব দেবে।

ভবিষ্যতে নিচের বিষয়গুলো যুক্ত করা হবে—

- Permission-based Memory
- User Approval
- Memory Encryption
- Secure Storage
- Delete Permission

বর্তমান AR1 Version-এ সম্পূর্ণ Security Layer থাকবে না।

অন্যান্য **Component-**এর সাথে সংযোগ

Memory System একা কাজ করে না।

এটি পুরো Architecture-এর বিভিন্ন অংশের সঙ্গে যুক্ত থাকবে।

Core Layer

Core Memory Manager শুরু করবে এবং Memory-এর Configuration পরিচালনা করবে।

Provider System

Provider যখন Response তৈরি করবে, তখন প্রয়োজনীয় Context Memory System থেকে সংগ্রহ করতে পারবে।

Agent System

Agent পূর্বের Goal, সিদ্ধান্ত এবং Project Status জানতে Memory ব্যবহার করবে।

Tool System

কোনো Tool যদি পূর্বের তথ্য প্রয়োজন করে, তাহলে Memory System থেকে সেই তথ্য সংগ্রহ করবে।

Workflow Engine

Workflow-এর বর্তমান অবস্থা, Progress এবং Execution State Memory-তে সংরক্ষণ করা হবে।

Prompt Management

Prompt তৈরি করার সময় প্রাসঙ্গিক Memory যুক্ত করা হবে, যাতে LLM আরও সঠিক উত্তর দিতে পারে।

বর্তমান **Implementation (AR1)**

বর্তমান AR1 Version-এ Memory System-এর লক্ষ্য হবে একটি স্থিতিশীল ভিত্তি তৈরি করা।

এই সংস্করণে পরিকল্পিত বৈশিষ্ট্যগুলো হলো—

- Memory Manager
- Basic Memory Interface
- Save Memory
- Load Memory
- Update Memory
- Delete Memory
- Simple Search
- User Memory
- Conversation Memory
- Project Memory

এটি ভবিষ্যতের উন্নত Memory Layer-এর ভিত্তি হিসেবে কাজ করবে।

ভবিষ্যৎ **Implementation**

AR2, AR3 এবং পরবর্তী সংস্করণগুলোতে ধীরে ধীরে আরও উন্নত সুবিধা যোগ করা হবে।

যেমন—

- Semantic Memory
- Vector Database
- Memory Ranking
- Automatic Memory Extraction
- Long-term Knowledge Base
- Memory Provider Layer
- Cloud Synchronization
- Multi-user Memory
- Memory Compression
- Hybrid Retrieval
- AI-based Memory Optimization

এই **Chapter-**এর সারসংক্ষেপ

Memory System হলো IRON BOLT-এর দীর্ঘমেয়াদী স্মৃতিশক্তির ভিত্তি।

এটি AI-কে Conversation, Project এবং User সম্পর্কিত গুরুত্বপূর্ণ তথ্য সংরক্ষণ, হালনাগাদ এবং পুনরুদ্ধার করার ক্ষমতা দেবে। বর্তমান AR1 সংস্করণে একটি সরল কিন্তু নির্ভরযোগ্য Memory Foundation তৈরি করা হবে।

ভবিষ্যতের সংস্করণগুলোতে এর উপর ভিত্তি করে Semantic Search, Vector Memory, Knowledge Base এবং আরও উন্নত Memory Intelligence যুক্ত হবে।

structure

- Iron Bolt
- ├── memory/
- │   ├── __init__.py
- │   ├── manager.py
- │   ├── models.py
- │   ├── storage.py
- │   ├── retrieval.py
- │   ├── update.py
- │   ├── decision.py
- │   ├── config.py
- │   ├── exceptions.py
- │   └── utils.py
- │
- ├── data/
- │   └── memory.json
- │
- ├── providers/
- ├── tools/
- ├── agents/
- └── ...

Chapter 9 — Agent System

উদ্দেশ্য

Agent System হল IRON BOLT-এর সিদ্ধান্ত গ্রহণকারী (Decision Layer)। এটি পুরো Framework-এর "Brain" হিসেবে কাজ করবে।

Provider System শুধুমাত্র AI Model-এর সাথে যোগাযোগ করে, Memory System তথ্য সংরক্ষণ ও পুনরুদ্ধার করে, আর Tool System বাস্তব কাজ সম্পন্ন করে। কিন্তু কোন পরিস্থিতিতে কোন Tool ব্যবহার করতে হবে, কখন Memory থেকে তথ্য নিতে হবে, কোন Provider-এর কাছে Request পাঠাতে হবে এবং শেষ পর্যন্ত ব্যবহারকারীকে কীভাবে উত্তর দিতে হবে—এই সমস্ত সিদ্ধান্ত Agent System গ্রহণ করবে।

সহজভাবে বললে, Agent System হল সেই অংশ, যা IRON BOLT-এর সমস্ত Capability-কে একত্রিত করে একটি বুদ্ধিমান Assistant-এ পরিণত করবে।

কেন **Agent System** প্রয়োজন?

যদি Agent System না থাকে, তাহলে Framework-এর প্রতিটি অংশ আলাদাভাবে কাজ করবে, কিন্তু তাদের মধ্যে সমন্বয় থাকবে না।

উদাহরণ হিসেবে ধরা যাক, একজন ব্যবহারকারী লিখল—

"আমার Project-এর বর্তমান অবস্থা দেখাও এবং প্রয়োজন হলে নতুন Task তৈরি করো।"

এই Request সম্পন্ন করতে IRON BOLT-কে একাধিক কাজ করতে হবে।

প্রথমে Project সম্পর্কিত তথ্য Memory System থেকে সংগ্রহ করতে হবে। এরপর সেই তথ্য বিশ্লেষণ করতে হবে। প্রয়োজন হলে Tool ব্যবহার করে নতুন Task তৈরি করতে হবে। সবশেষে একটি অর্থপূর্ণ উত্তর ব্যবহারকারীর কাছে পাঠাতে হবে।

এই পুরো প্রক্রিয়াটি সমন্বয় করার দায়িত্ব Agent System-এর।

অর্থাৎ Agent System ছাড়া Framework-এর বিভিন্ন Component নিজেদের কাজ করতে পারলেও তারা একসাথে একটি সম্পূর্ণ Intelligent Assistant হিসেবে কাজ করতে পারবে না।

Agent System-এর দায়িত্ব

Agent System-এর মূল দায়িত্বগুলো হবে—

- User-এর Request বিশ্লেষণ করা।
- Request-এর উদ্দেশ্য (Intent) বুঝতে চেষ্টা করা।
- Memory প্রয়োজন হলে Memory System ব্যবহার করা।
- Tool প্রয়োজন হলে Tool Manager-এর মাধ্যমে Tool চালানো।
- Provider Manager-এর মাধ্যমে উপযুক্ত AI Model ব্যবহার করা।
- প্রয়োজন হলে একাধিক ধাপে Task সম্পন্ন করা।
- বিভিন্ন Component-এর ফলাফল একত্রিত করা।

- চূড়ান্ত Response তৈরি করে ব্যবহারকারীর কাছে পাঠানো।

Agent System নিজে কোনো Memory সংরক্ষণ করবে না, কোনো Tool বাস্তবায়ন করবে না এবং কোনো Provider-এর সাথে সরাসরি API Call করবে না। এটি শুধুমাত্র পুরো Framework-এর কাজ সমন্বয় করবে।

Agent System Architecture

ভবিষ্যতে Agent Layer-এর সম্ভাব্য Structure হতে পারে—

agent/

├── agent.py

├── agent_manager.py

├── decision_engine.py

├── response_builder.py

├── state.py

├── task_controller.py

├── context_builder.py

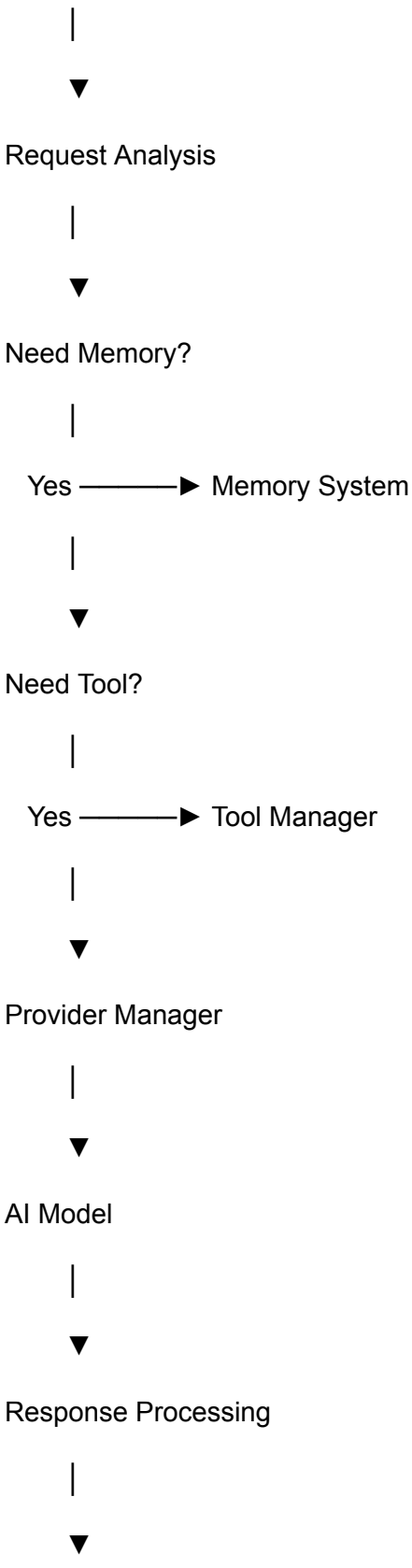
└── interfaces.py

বর্তমান AR1 Version-এ এই সমস্ত Module Implement করা হবে না। তবে Architecture শুরু থেকেই এমনভাবে তৈরি করা হবে, যাতে ভবিষ্যতে সহজেই এগুলো যুক্ত করা যায়।

Agent-এর কাজের ধাপ

একটি Request Agent-এর কাছে এলে সাধারণভাবে নিচের ধাপগুলো অনুসরণ করা হবে।

User Request


Request Analysis
Need Memory?
Yes
Memory System
Need Tool?
Yes
Tool Manager
Provider Manager
AI Model
Response Processing

Final Response

এই Flow-এর মাধ্যমে Agent প্রয়োজন অনুযায়ী বিভিন্ন Component ব্যবহার করে একটি সম্পূর্ণ উত্তর তৈরি করবে।

Agent State

Agent সবসময় একই অবস্থায় কাজ করবে না। একটি Request সম্পন্ন করার সময় Agent বিভিন্ন State অতিক্রম করবে।

উদাহরণস্বরূপ—

- Idle
- Receiving Request
- Thinking
- Retrieving Memory
- Executing Tool
- Waiting for Provider
- Building Response
- Completed
- Failed

বর্তমানে AR1 Version-এ State Management খুবই সাধারণ থাকবে। ভবিষ্যতে এটি আরও উন্নত করা হবে।

Decision Making

Agent System-এর সবচেয়ে গুরুত্বপূর্ণ দায়িত্ব হল সিদ্ধান্ত গ্রহণ।

যেমন—

- Memory ব্যবহার করা প্রয়োজন কি না।
- Tool চালানো প্রয়োজন কি না।
- সরাসরি উত্তর দেওয়া যাবে কি না।
- Provider-এর কাছে নতুন Request পাঠাতে হবে কি না।
- প্রাপ্ত তথ্য যথেষ্ট কি না।
- Task সম্পন্ন হয়েছে কি না।

বর্তমান AR1 Version-এ এই সিদ্ধান্তগুলো তুলনামূলকভাবে সহজ Rule-এর মাধ্যমে নেওয়া হবে।

ভবিষ্যতে Planning & Reasoning Layer যুক্ত হলে Agent আরও জটিল সিদ্ধান্ত নিতে পারবে।

Task Controller

একটি User Request অনেক সময় একাধিক ছোট ধাপে সম্পন্ন করতে হয়।

Task Controller-এর কাজ হবে—

- বড় Task-কে ছোট ধাপে ভাগ করা।
- প্রতিটি ধাপের বর্তমান অবস্থা সংরক্ষণ করা।
- কোন ধাপ সম্পন্ন হয়েছে তা পর্যবেক্ষণ করা।
- ব্যর্থ হলে নিরাপদভাবে Execution বন্ধ করা।

AR1 Version-এ এটি সীমিত থাকবে। Workflow Engine যুক্ত হওয়ার পরে এটি আরও শক্তিশালী হবে।

Response Builder

Memory, Tool এবং Provider থেকে পাওয়া ফলাফল সরাসরি ব্যবহারকারীর কাছে পাঠানো হবে না।

Response Builder-এর কাজ হবে—

- বিভিন্ন উৎস থেকে পাওয়া তথ্য একত্রিত করা।
- প্রয়োজন হলে অপ্রয়োজনীয় তথ্য বাদ দেওয়া।
- উত্তরকে পরিষ্কার ও স্বাভাবিক ভাষায় সাজানো।
- ব্যবহারকারীর জন্য Final Response তৈরি করা।

এর ফলে Framework-এর ভেতরের জটিলতা ব্যবহারকারীর সামনে প্রকাশ পাবে না।

Agent System কার সাথে Connected থাকবে

Agent System হল পুরো Framework-এর কেন্দ্রীয় সমন্বয়কারী Layer।

এটি সরাসরি সংযুক্ত থাকবে—

- Core Layer

- Provider System
- Memory System
- Tool System
- Prompt Management
- Planning & Reasoning
- Workflow Engine
- Execution Flow

তবে একটি গুরুত্বপূর্ণ নিয়ম সবসময় বজায় থাকবে।

Provider, Memory এবং Tool একে অপরকে সরাসরি ব্যবহার করবে না। Agent System-ই তাদের মধ্যে সমন্বয় করবে।

এতে Architecture-এর Coupling কম থাকবে এবং প্রতিটি Layer স্বাধীনভাবে উন্নত করা সম্ভব হবে।

বর্তমান Implementation (AR1)

বর্তমান AR1 Version-এর লক্ষ্য একটি সম্পূর্ণ Autonomous Agent তৈরি করা নয়।

এই সংস্করণে Agent Layer-এর Foundation তৈরি করা হবে।

পরিকল্পিত বৈশিষ্ট্যগুলো হলো—

- Basic Agent Class
- Request Processing
- Provider Integration
- Memory Integration
- Tool Integration
- Simple Decision Flow
- Response Generation
- Basic Agent State

এই Foundation ভবিষ্যতের উন্নত Agent System-এর ভিত্তি হিসেবে কাজ করবে।

ভবিষ্যৎ Implementation

AR2, AR3 এবং পরবর্তী সংস্করণে Agent System আরও উন্নত হবে।

ধীরে ধীরে যুক্ত করা হবে—

- Multi-Agent Architecture
- Goal-based Execution
- Autonomous Task Execution
- Self Monitoring
- Agent Collaboration
- Background Agent
- Task Queue
- Event-driven Agent
- Adaptive Decision Making
- Intelligent Recovery
- Human Approval Workflow

এই Feature-গুলো যুক্ত হওয়ার পর IRON BOLT শুধুমাত্র একটি Chat Assistant থাকবে না; বরং একটি পূর্ণাঙ্গ Autonomous AI Framework-এ পরিণত হবে।

এই **Chapter-**এর সারসংক্ষেপ

Agent System হল IRON BOLT-এর কেন্দ্রীয় সিদ্ধান্ত গ্রহণকারী Layer। এটি Provider, Memory এবং Tool System-এর মধ্যে সমন্বয় করে একটি User Request সম্পূর্ণ করে। বর্তমান AR1 Version-এ একটি সরল কিন্তু নির্ভরযোগ্য Agent Foundation তৈরি করা হবে। ভবিষ্যতের সংস্করণগুলোতে এই Foundation-এর উপর ভিত্তি করে Multi-Agent System, Autonomous Decision Making, Goal Management এবং আরও উন্নত AI Capability যুক্ত হবে।

Chapter 10 — Tool System

উদ্দেশ্য

Tool System হল IRON BOLT-এর **Action Layer**। AI Model শুধুমাত্র চিন্তা করতে এবং উত্তর তৈরি করতে পারে, কিন্তু বাস্তব পৃথিবীতে কোনো কাজ সম্পন্ন করতে পারে না। কোনো ফাইল পড়া, Python Code চালানো, Web Search করা, Database থেকে তথ্য আনা কিংবা কোনো API-এর সাথে যোগাযোগ করা—এসব কাজের জন্য Tool System প্রয়োজন।

অর্থাৎ, Provider System AI-কে চিন্তা করার ক্ষমতা দেয়, Memory System AI-কে তথ্য মনে রাখতে সাহায্য করে, আর Tool System AI-কে বাস্তব কাজ করার ক্ষমতা প্রদান করে।

এই কারণেই Tool System হলো IRON BOLT-এর অন্যতম গুরুত্বপূর্ণ Layer।

কেন **Tool System** প্রয়োজন?

যদি Tool System না থাকে, তাহলে IRON BOLT শুধুমাত্র প্রশ্নের উত্তর দিতে পারবে।

উদাহরণস্বরূপ, একজন ব্যবহারকারী যদি বলে—

"এই CSV File বিশ্লেষণ করো এবং একটি Report তৈরি করো।"

শুধুমাত্র একটি LLM এই কাজ সম্পূর্ণ করতে পারবে না। তাকে প্রথমে File পড়তে হবে, তারপর Data বিশ্লেষণ করতে হবে, প্রয়োজন হলে Python Code চালাতে হবে এবং শেষে ফলাফল উপস্থাপন করতে হবে।

এই সমস্ত বাস্তব কাজ Tool System সম্পন্ন করবে।

অর্থাৎ Tool System AI-এর ক্ষমতাকে শুধুমাত্র কথোপকথনের মধ্যে সীমাবদ্ধ না রেখে বাস্তব জগতের কাজ সম্পন্ন করার সুযোগ করে দেয়।

Tool System-এর দায়িত্ব

Tool System-এর প্রধান কাজগুলো হলো—

- বিভিন্ন Tool Register করা।
- Tool-এর তথ্য সংরক্ষণ করা।
- নিরাপদভাবে Tool Execute করা।
- Tool-এর Input যাচাই করা।
- Tool-এর Output সংগ্রহ করা।
- Execution Status পর্যবেক্ষণ করা।
- Error Handle করা।
- Agent System-কে Result প্রদান করা।

Tool System নিজে কোনো সিদ্ধান্ত নেবে না। কোন Tool ব্যবহার হবে এবং কখন ব্যবহার হবে, সেই সিদ্ধান্ত সবসময় Agent System গ্রহণ করবে।

Tool System Architecture

ভবিষ্যতে Tool Layer-এর সম্ভাব্য Structure হতে পারে—

tools/

├── tool_manager.py

├── base_tool.py

├── registry.py

├── validator.py

├── executor.py

├── result.py

├── permissions.py

├── python_tool.py

├── file_tool.py

├── search_tool.py

├── browser_tool.py

├── calculator_tool.py

├── api_tool.py

└── ...

বর্তমান AR1 Version-এ সব Tool তৈরি করা হবে না। শুরুতে শুধুমাত্র Foundation এবং প্রয়োজনীয় Tool যুক্ত করা হবে।

Base Tool

প্রতিটি Tool একটি সাধারণ Interface অনুসরণ করবে।

এর ফলে Python Tool, File Tool, Search Tool অথবা ভবিষ্যতের যেকোনো Tool একই নিয়মে Framework-এর সাথে যুক্ত হতে পারবে।

Base Tool-এর সাধারণ দায়িত্ব হবে—

- Tool-এর নাম প্রদান করা।
- Tool-এর বর্ণনা প্রদান করা।
- Required Parameter নির্ধারণ করা।
- Execute Method বাস্তবায়ন করা।
- Execution Result Return করা।

এই Unified Interface-এর কারণে Agent System কখনো Tool-এর অভ্যন্তরীণ Implementation সম্পর্কে জানবে না।

Tool Manager

Tool Manager হল পুরো Tool Layer-এর Controller।

Framework কখনো কোনো Tool সরাসরি ব্যবহার করবে না।

সবসময় Tool Manager-এর মাধ্যমে Tool Execute করা হবে।

Tool Manager-এর প্রধান দায়িত্ব—

- Tool Register করা।
- Tool খুঁজে বের করা।
- Tool Execute করা।
- Tool Enable বা Disable করা।
- একই Interface-এর মাধ্যমে সব Tool পরিচালনা করা।
- ভবিষ্যতে Permission যাচাই করা।
- Execution Result Agent System-এ ফেরত পাঠানো।

Tool Manager ছাড়া কোনো Tool Framework-এর অংশ হিসেবে ব্যবহার করা হবে না।

Tool Lifecycle

একটি Tool সাধারণত নিচের ধাপগুলো অনুসরণ করে কাজ করবে—

Agent Request

|

▼

Tool Selection

|

▼

Input Validation

|

▼

Tool Execution

|

▼

Result Processing

|

▼

Return Result

এই Flow নিশ্চিত করবে যে প্রতিটি Tool একই নিয়ম অনুসরণ করে কাজ করছে।

Tool Validation

Tool Execute করার আগে Input যাচাই করা হবে।

যেমন—

- প্রয়োজনীয় Parameter দেওয়া হয়েছে কি না।
- File Path বৈধ কি না।

- Unsupported Argument আছে কি না।
- Input Type সঠিক কি না।

Validation ব্যর্থ হলে Tool Execute করা হবে না এবং একটি পরিষ্কার Error Agent System-এ ফেরত পাঠানো হবে।

Tool Result Processing

প্রতিটি Tool-এর Output একই রকম হবে না।

কোনো Tool Text Return করবে, কোনো Tool File তৈরি করবে, আবার কোনো Tool JSON Data প্রদান করবে।

Tool Result Processing-এর কাজ হবে—

- Output-কে Standard Format-এ রূপান্তর করা।
- Error এবং Success আলাদা করা।
- Metadata সংরক্ষণ করা।
- Agent System-এর জন্য Result প্রস্তুত করা।

এর ফলে Agent System সব Tool-এর Result একইভাবে ব্যবহার করতে পারবে।

Tool-এর ধরন

বর্তমান এবং ভবিষ্যৎ সংস্করণে IRON BOLT-এ বিভিন্ন ধরনের Tool যুক্ত হতে পারে।

সম্ভাব্য Tool-এর মধ্যে রয়েছে—

- Python Tool
- File Tool
- Search Tool
- Browser Tool
- Calculator Tool
- API Tool
- Database Tool
- Terminal Tool
- Document Tool
- Image Tool

প্রতিটি Tool স্বাধীন Module হিসেবে তৈরি হবে এবং Base Tool Interface অনুসরণ করবে।

অন্যান্য **Component-**এর সাথে সংযোগ

Tool System একা কাজ করে না। এটি পুরো Architecture-এর বিভিন্ন Layer-এর সাথে যুক্ত থাকবে।

Core Layer

Configuration, Logging এবং Common Infrastructure প্রদান করবে।

Agent System

কোন Tool ব্যবহার হবে তা সিদ্ধান্ত নেবে এবং Tool Manager-এর মাধ্যমে Execute করবে।

Memory System

Tool Execute করার আগে বা পরে প্রয়োজনীয় তথ্য Memory থেকে নেওয়া বা সংরক্ষণ করা যেতে পারে।

Provider System

Tool-এর ফলাফল AI Model-এ পাঠানো হতে পারে, যাতে Model Final Response তৈরি করতে পারে।

Workflow Engine

একাধিক Tool ধারাবাহিকভাবে Execute করার সময় Workflow Engine Tool System ব্যবহার করবে।

Security & Permission

ভবিষ্যতে কোন Tool কে ব্যবহার করতে পারবে, কোন পরিস্থিতিতে ব্যবহার করা যাবে এবং User-এর অনুমতি লাগবে কি না—এসব নিয়ন্ত্রণ Security Layer করবে।

বর্তমান **Implementation (AR1)**

বর্তমান AR1 Version-এর লক্ষ্য একটি স্থিতিশীল Tool Foundation তৈরি করা।

এই সংস্করণে পরিকল্পিত বৈশিষ্ট্যগুলো হলো—

- Base Tool Interface
- Tool Manager
- Tool Registration
- Tool Discovery
- Tool Execution

- Input Validation
- Standard Result Format
- Basic Error Handling

এই Foundation-এর উপর ভিত্তি করেই ভবিষ্যতের সব Tool যুক্ত হবে।

ভবিষ্যৎ Implementation

AR2, AR3 এবং পরবর্তী সংস্করণে Tool System আরও শক্তিশালী হবে।

ধীরে ধীরে যুক্ত করা হবে—

- Dynamic Tool Loading
- Plugin-based Tool Support
- MCP Tool Integration
- Parallel Tool Execution
- Streaming Tool Result
- Tool Permission System
- Tool Sandbox
- Tool Timeout Management
- Retry Mechanism
- Tool Monitoring
- Tool Usage Analytics
- Distributed Tool Execution

এসব Feature যুক্ত হওয়ার পর IRON BOLT আরও নিরাপদ, দ্রুত এবং সম্প্রসারণযোগ্যভাবে বাস্তব কাজ সম্পন্ন করতে পারবে।

Implementation Order

Tool System নিম্নলিখিত ধাপে উন্নয়ন করা হবে।

Phase 1 (AR1)

- Base Tool
- Tool Manager
- Tool Registration
- Basic Tool Execution

- Result Processing

Phase 2

- Advanced Validation
- Error Recovery
- Tool Metadata
- Multiple Built-in Tools

Phase 3 (Future)

- Plugin Tools
- MCP Integration
- Permission Control
- Parallel Execution
- Distributed Tool Architecture

এই ধাপগুলো অনুসরণ করলে Foundation পরিবর্তন না করেই নতুন Tool যুক্ত করা সম্ভব হবে।

এই **Chapter-**এর সারসংক্ষেপ

Tool System হল IRON BOLT-এর Action Layer, যা AI-কে বাস্তব কাজ সম্পন্ন করার ক্ষমতা প্রদান করে। এটি নিজে কোনো সিদ্ধান্ত নেয় না; Agent System-এর নির্দেশ অনুযায়ী Tool Manager-এর মাধ্যমে বিভিন্ন Tool পরিচালনা করে। বর্তমান AR1 Version-এ একটি নির্ভরযোগ্য Tool Foundation তৈরি করা হবে। ভবিষ্যতের সংস্করণগুলোতে Plugin, MCP, Parallel Execution, Permission System এবং আরও উন্নত Tool Capability যুক্ত হয়ে IRON BOLT-কে একটি পূর্ণাঙ্গ AI Automation Framework-এ রূপান্তর করবে।

Chapter 11 — Workflow Engine

উদ্দেশ্য

Workflow Engine হল IRON BOLT-এর এমন একটি Layer, যা একাধিক ধাপের (Multi-step) কাজকে সঠিক ক্রমে পরিচালনা করবে।

একটি সাধারণ প্রশ্নের উত্তর দেওয়ার জন্য শুধু একটি AI Model-এ Request পাঠানোই যথেষ্ট হতে পারে। কিন্তু বাস্তব জীবনের অনেক কাজ একাধিক ধাপে সম্পন্ন হয়। যেমন—Memory থেকে তথ্য সংগ্রহ করা, Tool ব্যবহার করা, ফলাফল বিশ্লেষণ করা, আবার প্রয়োজনে দ্বিতীয় Tool চালানো এবং সবশেষে ব্যবহারকারীর জন্য একটি চূড়ান্ত উত্তর তৈরি করা।

এই ধরনের ধারাবাহিক কাজগুলোকে নিরাপদ, নিয়ন্ত্রিত এবং পুনঃব্যবহারযোগ্য (Reusable) উপায়ে পরিচালনা করার জন্যই Workflow Engine তৈরি করা হবে।

Workflow Engine কোনো AI Model নয়, কোনো Tool নয় এবং কোনো সিদ্ধান্ত গ্রহণকারী Agent-ও নয়। এটি শুধুমাত্র বিভিন্ন Component-এর কাজকে একটি নির্দিষ্ট ক্রমে পরিচালনা করবে।

কেন **Workflow Engine** প্রয়োজন?

শুধু Agent System থাকলে Agent সিদ্ধান্ত নিতে পারবে, কিন্তু দীর্ঘ বা জটিল কাজের প্রতিটি ধাপ আলাদাভাবে পরিচালনা করা Agent-এর মূল দায়িত্ব নয়।

উদাহরণ হিসেবে ধরা যাক একজন ব্যবহারকারী লিখল—

"আমার Project-এর বর্তমান অবস্থা বিশ্লেষণ করো, প্রয়োজন হলে নতুন Task তৈরি করো, তারপর একটি Summary দাও।"

এই Request সম্পন্ন করতে IRON BOLT-কে একাধিক ধাপ অনুসরণ করতে হবে।

প্রথমে Project Memory থেকে তথ্য সংগ্রহ করতে হবে।

তারপর সেই তথ্য AI Model দিয়ে বিশ্লেষণ করতে হবে।

বিশ্লেষণের ফল অনুযায়ী Tool ব্যবহার করে নতুন Task তৈরি করতে হতে পারে।

এরপর নতুন অবস্থা আবার যাচাই করতে হবে।

সবশেষে ব্যবহারকারীর জন্য একটি পরিষ্কার Summary তৈরি করতে হবে।

এই পুরো প্রক্রিয়া একটি Workflow।

Workflow Engine এই Workflow-এর প্রতিটি ধাপকে নিয়ন্ত্রণ করবে।

Workflow Engine-এর দায়িত্ব

Workflow Engine-এর প্রধান দায়িত্বগুলো হবে—

- একটি Workflow শুরু করা।
- Workflow-এর প্রতিটি Step সঠিক ক্রমে চালানো।
- প্রতিটি Step-এর ফলাফল সংরক্ষণ করা।
- একটি Step ব্যর্থ হলে পরবর্তী করণীয় নির্ধারণ করা।
- প্রয়োজন হলে Retry করা।
- Workflow-এর বর্তমান অবস্থা পর্যবেক্ষণ করা।
- Workflow সম্পন্ন হলে Agent-এর কাছে ফলাফল ফিরিয়ে দেওয়া।
- দীর্ঘ সময় চলা কাজ নিরাপদভাবে পরিচালনা করা।

Workflow Engine নিজে কোনো সিদ্ধান্ত নেবে না।

কোন Workflow চালানো হবে, সেটি Agent System নির্ধারণ করবে।

Workflow Engine Architecture

ভবিষ্যতে Workflow Layer-এর সম্ভাব্য Structure হতে পারে—

workflow/

├── workflow_engine.py

├── workflow_manager.py

├── workflow_definition.py

├── workflow_context.py

├── workflow_state.py

├── step_executor.py

├── step_result.py

├── retry_manager.py

├── condition.py

└── interfaces.py

বর্তমান AR1 Version-এ এই সব Module Implement করা হবে না।

তবে Architecture শুরু থেকেই এমনভাবে ডিজাইন করা হবে, যাতে ভবিষ্যতে নতুন Workflow Feature সহজেই যুক্ত করা যায়।

Workflow-এর কাজের ধাপ

সাধারণভাবে একটি Workflow নিচের ধাপগুলো অনুসরণ করবে—

User Request

|

▼

Agent System

|

▼

Workflow Engine

|

▼

Step 1 Execute

|

▼

Step 2 Execute

|

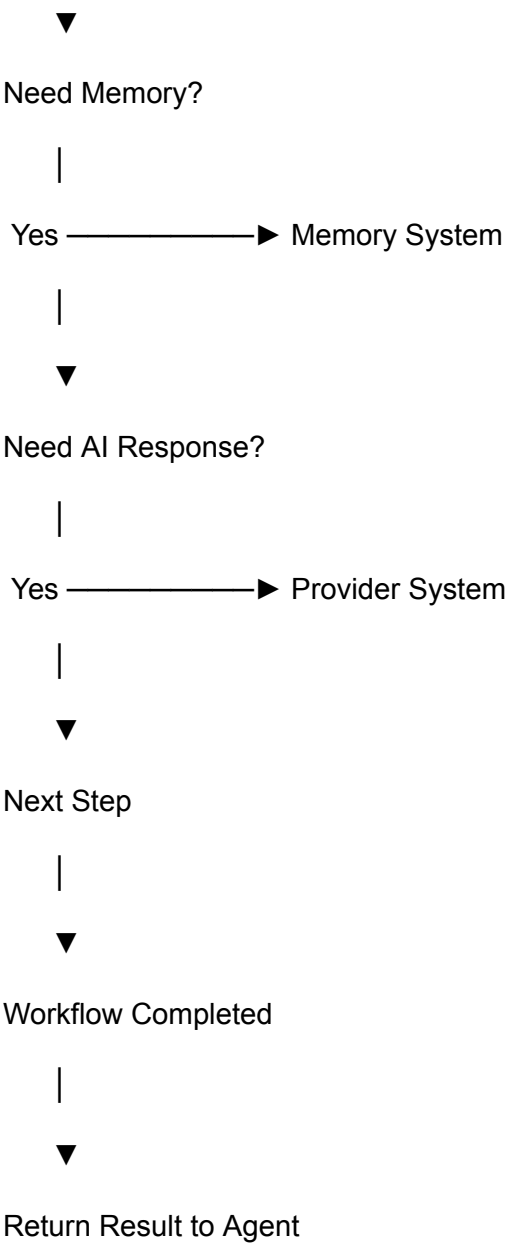
▼

Need Tool?

|

Yes ————————► Tool System

|



এই Flow-এর মাধ্যমে একটি বড় কাজকে ছোট ছোট ধাপে ভাগ করে নিরাপদভাবে সম্পন্ন করা হবে।

Workflow Definition

Workflow Engine সরাসরি কাজ চালাবে না।

প্রথমে একটি Workflow Definition তৈরি হবে।

Workflow Definition-এর মধ্যে থাকবে—

- Workflow-এর নাম।
- কতগুলো Step থাকবে।
- প্রতিটি Step-এর কাজ।
- কোন Step-এর পরে কোন Step চলবে।
- কোন Step ব্যর্থ হলে কী হবে।
- Workflow সফল হওয়ার শর্ত।

এর ফলে একই Workflow ভবিষ্যতে বারবার ব্যবহার করা যাবে।

Step Executor

Workflow-এর প্রতিটি ধাপ বাস্তবায়নের কাজ করবে Step Executor।

এর দায়িত্ব হবে—

- বর্তমান Step চালানো।
- Step-এর Input গ্রহণ করা।
- প্রয়োজন হলে Tool ব্যবহার করা।
- প্রয়োজন হলে Provider ব্যবহার করা।
- Step-এর Output সংগ্রহ করা।
- পরবর্তী Step-এর জন্য Output পাঠানো।

Step Executor কখনো Workflow-এর সামগ্রিক নিয়ন্ত্রণ করবে না। এটি শুধুমাত্র একটি Step সম্পন্ন করবে।

Workflow Context

একটি Workflow চলাকালীন অনেক তথ্য সাময়িকভাবে সংরক্ষণ করতে হয়।

যেমন—

- বর্তমান Step।
- পূর্ববর্তী Step-এর ফলাফল।
- Workflow Variables।
- Temporary Data।
- Execution Metadata।

এই তথ্যগুলো Workflow Context-এ সংরক্ষিত থাকবে।

Workflow শেষ হলে Context প্রয়োজন অনুযায়ী মুছে ফেলা হবে অথবা Memory System-এ সংরক্ষণ করা হবে।

Workflow State

Workflow সবসময় একই অবস্থায় থাকবে না।

একটি Workflow সাধারণত নিচের State-গুলো অতিক্রম করবে—

- Created
- Ready
- Running
- Waiting
- Retrying
- Completed
- Failed
- Cancelled

এই State-এর মাধ্যমে Workflow-এর বর্তমান অবস্থা সহজে পর্যবেক্ষণ করা যাবে।

Retry Mechanism (Future)

বাস্তব পরিবেশে সব Step প্রথমবারেই সফল নাও হতে পারে।

যেমন—

- Network Failure
- Temporary API Error
- Timeout
- External Service Unavailable

Future Version-এ Retry Manager নির্ধারণ করবে—

- কতবার Retry করা হবে।
- কতক্ষণ অপেক্ষা করা হবে।
- Retry ব্যর্থ হলে Workflow বন্ধ হবে নাকি বিকল্প পথ অনুসরণ করবে।

বর্তমান AR1 Version-এ এই সুবিধা থাকবে না।

Conditional Execution (Future)

সব Workflow একই রকম নয়।

অনেক সময় একটি Step-এর ফলাফলের উপর ভিত্তি করে পরবর্তী Step পরিবর্তিত হতে পারে।

উদাহরণ—

যদি File পাওয়া যায় → File Analysis

অন্যথায় → Error Response

এই ধরনের Branching Logic ভবিষ্যতের Workflow Engine-এ যুক্ত করা হবে।

অন্যান্য **Component-**এর সাথে সংযোগ

Workflow Engine একা কাজ করে না।

এটি Architecture-এর বিভিন্ন Layer-এর সাথে সমন্বয় করে কাজ করবে।

Agent System

কোন Workflow চালানো হবে, তা Agent নির্ধারণ করবে।

Tool System

Workflow-এর কোনো Step-এ Tool প্রয়োজন হলে Tool Manager ব্যবহার করা হবে।

Memory System

Workflow চলাকালীন প্রয়োজনীয় Context এবং Progress Memory থেকে নেওয়া বা সেখানে সংরক্ষণ করা হবে।

Provider System

AI Model-এর সাহায্য প্রয়োজন হলে Workflow Provider Manager-এর মাধ্যমে Model ব্যবহার করবে।

Core Layer

Workflow-এর Configuration, Logging, Validation এবং Runtime Context Core Layer পরিচালনা করবে।

Execution Flow

Workflow সম্পন্ন হওয়ার পরে এর ফলাফল Execution Flow-এর মাধ্যমে ব্যবহারকারীর কাছে পৌঁছাবে।

বর্তমান Implementation (AR1)

বর্তমান AR1 Version-এর লক্ষ্য একটি পূর্ণাঙ্গ Workflow Automation Platform তৈরি করা নয়।

এই সংস্করণে শুধুমাত্র ভবিষ্যতের Workflow Layer-এর ভিত্তি প্রস্তুত করা হবে।

পরিকল্পিত বৈশিষ্ট্যগুলো হলো—

- Basic Workflow Engine
- Workflow Interface
- Sequential Step Execution
- Simple Workflow Context
- Basic Workflow State
- Agent Integration
- Provider Integration
- Tool Integration
- Memory Integration

এই Foundation ভবিষ্যতের উন্নত Workflow System-এর ভিত্তি হিসেবে কাজ করবে।

ভবিষ্যৎ Implementation

AR2, AR3 এবং পরবর্তী সংস্করণগুলোতে Workflow Engine আরও শক্তিশালী হবে।

ধীরে ধীরে যুক্ত করা হবে—

- Parallel Workflow Execution
- Conditional Branching
- Retry Manager
- Rollback Mechanism
- Scheduled Workflow
- Event-driven Workflow
- Background Workflow
- Workflow Queue
- Workflow Templates

- Human Approval Step
- Distributed Workflow
- Visual Workflow Builder
- Nested Workflow
- Workflow Monitoring Dashboard

এই Feature-গুলো যুক্ত হওয়ার পরে IRON BOLT শুধু কথোপকথনভিত্তিক Assistant থাকবে না; বরং জটিল ও দীর্ঘমেয়াদী কাজ স্বয়ংক্রিয়ভাবে সম্পন্ন করতে সক্ষম একটি Production-Ready AI Workflow Platform-এ পরিণত হবে।

এই **Chapter-**এর সারসংক্ষেপ

Workflow Engine হল IRON BOLT-এর Process Coordination Layer। এটি Agent System-এর সিদ্ধান্ত অনুযায়ী একাধিক Step সঠিক ক্রমে পরিচালনা করে এবং Provider, Memory ও Tool System-এর মধ্যে সমন্বয় ঘটিয়ে একটি সম্পূর্ণ Workflow সম্পন্ন করে। বর্তমান AR1 Version-এ একটি সরল কিন্তু স্থিতিশীল Workflow Foundation তৈরি করা হবে। ভবিষ্যতের সংস্করণগুলোতে এর উপর ভিত্তি করে Parallel Execution, Conditional Workflow, Retry Mechanism, Background Processing, Human Approval এবং Visual Workflow Builder-এর মতো উন্নত সক্ষমতা যুক্ত হবে, যা IRON BOLT-কে একটি আরও শক্তিশালী ও Production-Ready AI Framework হিসেবে প্রতিষ্ঠিত করবে।

Chapter 12: Planning & Reasoning

Overview

Planning & Reasoning হলো IRON BOLT-এর সেই অংশ, যা Assistant-কে শুধুমাত্র প্রশ্নের উত্তর দেওয়া থেকে বের করে এনে সমস্যা বুঝে চিন্তা করা**,** পরিকল্পনা তৈরি করা এবং সঠিক সিদ্ধান্ত নেওয়ার ক্ষমতা প্রদান করবে।

একটি সাধারণ AI অনেক সময় ব্যবহারকারীর প্রশ্ন পাওয়ার সাথে সাথে সরাসরি উত্তর তৈরি করে। কিন্তু বাস্তব জীবনের অনেক কাজ এত সহজ নয়।

উদাহরণ:

- একটি সম্পূর্ণ Project তৈরি করা
- বড় Codebase Analyze করা
- একাধিক Tool ব্যবহার করে কাজ সম্পন্ন করা

- ধাপে ধাপে কোনো Task শেষ করা
- ভবিষ্যতের কাজের পরিকল্পনা তৈরি করা
- দীর্ঘ Conversation-এর Context বজায় রেখে সিদ্ধান্ত নেওয়া

এ ধরনের কাজের জন্য শুধুমাত্র Language Model যথেষ্ট নয়।

সেই কারণেই IRON BOLT-এর Planning & Reasoning Layer তৈরি করা হবে।

কেন এই **Layer** তৈরি করা হচ্ছে?

Planning & Reasoning Layer-এর প্রধান উদ্দেশ্য হলো—

Assistant যেন কোনো কাজ শুরু করার আগে বুঝতে পারে—

- আসল Problem কী
- কোন তথ্য দরকার
- কোন Tool লাগবে
- কোন Agent কাজ করবে
- Memory লাগবে কি না
- কাজের ধাপগুলো কী হবে
- সম্ভাব্য ভুল কোথায় হতে পারে
- সবচেয়ে কার্যকর সমাধান কোনটি

অর্থাৎ,

Think First → Execute Later

এই নীতিতেই Planning Layer কাজ করবে।

এই **Layer-**এর দায়িত্ব

Planning & Reasoning Layer মূলত নিচের কাজগুলো করবে।

User Intent Analysis

ব্যবহারকারী আসলে কী করতে চায় তা বুঝবে।

অনেক সময় ব্যবহারকারী সরাসরি নিজের উদ্দেশ্য বলে না।

উদাহরণ:

"আমার একটা Python Project বানাতে হবে"

এখানে Assistant বুঝবে—

- Project Type
- Complexity
- Required Files
- Future Steps

Problem Breakdown

বড় সমস্যাকে ছোট ছোট অংশে ভাগ করবে।

উদাহরণ:

একটি Website তৈরি করতে হলে—

- Requirement বোঝা
- Structure তৈরি
- UI Design
- Backend
- Database
- Testing
- Deployment

একসাথে সব না করে ধাপে ধাপে পরিকল্পনা করবে।

Task Planning

কোন কাজ কোন ক্রমে হবে তা নির্ধারণ করবে।

কারণ অনেক কাজের নির্দিষ্ট Dependency থাকে।

উদাহরণ:

Database ছাড়া Backend সম্পূর্ণ করা সম্ভব নয়।

Backend ছাড়া অনেক API পরীক্ষা করা সম্ভব নয়।

Planning Layer এসব Dependency বুঝে কাজ সাজাবে।

Decision Making

একাধিক সম্ভাব্য সমাধান থাকলে কোনটি সবচেয়ে উপযুক্ত তা নির্বাচন করবে।

উদাহরণ:

একটি File পড়ার জন্য—

- Local File
- Memory
- Tool
- Plugin

কোন উৎস ব্যবহার করা হবে তা পরিস্থিতি অনুযায়ী নির্ধারণ করবে।

Multi-Step Thinking

একটি উত্তর তৈরি করার আগে একাধিক ধাপে চিন্তা করবে।

উদাহরণ:

User Prompt

↓

Intent বোঝা

↓

Plan তৈরি

↓

Tool নির্বাচন

↓

Tool Result Analyze

↓

Final Response তৈরি

Error Recovery Planning

কোন ধাপে সমস্যা হলে বিকল্প পরিকল্পনা তৈরি করবে।

যদি কোনো Tool ব্যর্থ হয়—

তাহলে—

- অন্য Tool ব্যবহার করবে
- Provider পরিবর্তন করবে
- পুনরায় চেষ্টা করবে
- ব্যবহারকারীর কাছ থেকে অতিরিক্ত তথ্য চাইবে

Resource Optimization

অপ্রয়োজনীয় কাজ কমিয়ে সবচেয়ে কার্যকর পথ নির্বাচন করবে।

যদি Memory-তে উত্তর আগে থেকেই থাকে,

তাহলে আবার Tool চালানোর প্রয়োজন নাও হতে পারে।

এভাবে সময়, খরচ এবং Token ব্যবহার কমবে।

Planning & Reasoning কোথায় থাকবে?

Architecture অনুযায়ী এই Layer থাকবে—

User

↓

Workflow Engine

↓

Planning & Reasoning

↓

Agent

↓

Tool

↓

Memory

↓

LLM

↓

Response

Planning Layer Workflow Engine-এর কাছ থেকে Task গ্রহণ করবে এবং পুরো কাজের জন্য একটি Execution Plan তৈরি করবে।

কোন কোন **Component-**এর সাথে **Connected** থাকবে?

Planning Layer সরাসরি যুক্ত থাকবে—

Workflow Engine

Workflow থেকে Task গ্রহণ করবে।

Workflow-কে পরবর্তী Step জানাবে।

Agent System

কোন Agent কাজ করবে তা নির্ধারণ করতে সাহায্য করবে।

Tool System

কোন Tool ব্যবহার হবে তা নির্বাচন করবে।

Memory System

পুরনো তথ্য ব্যবহার করা দরকার কি না তা নির্ধারণ করবে।

Provider System

কোন Provider ব্যবহার করা সবচেয়ে উপযুক্ত হবে তা বিবেচনা করতে পারবে।

Prompt Management

LLM-এ পাঠানোর আগে Prompt আরও পরিষ্কার ও কার্যকর করতে সহায়তা করবে।

Core Layer

Configuration, Rules এবং System State ব্যবহার করবে।

Current Implementation (বর্তমান অবস্থা)

বর্তমান AR1 Version-এ Planning & Reasoning Layer আলাদা Component হিসেবে তৈরি করা হয়নি।

বর্তমানে Planning-এর বেশিরভাগ কাজ Language Model নিজেই করে।

Workflow Engine Prompt Provider-এর মাধ্যমে Model-এ Request পাঠায় এবং Model নিজের অভ্যন্তরীণ Reasoning ব্যবহার করে উত্তর তৈরি করে।

অর্থাৎ—

বর্তমান Architecture-এ আলাদা Planner নেই।

Future Implementation

ভবিষ্যতে Planning Layer একটি পূর্ণাঙ্গ স্বাধীন Component হবে।

এটি Workflow থেকে Task নিয়ে Execution Plan তৈরি করবে।

সম্ভাব্য Component—

Planner

Task Analyze করবে।

Goal Manager

চূড়ান্ত লক্ষ্য নির্ধারণ করবে।

Task Decomposer

বড় কাজকে ছোট Step-এ ভাগ করবে।

Dependency Analyzer

কোন Step আগে এবং কোন Step পরে হবে তা নির্ধারণ করবে।

Strategy Selector

একাধিক Strategy-এর মধ্যে সেরা Strategy নির্বাচন করবে।

Decision Engine

Runtime-এ সিদ্ধান্ত নেবে।

Retry Planner

ব্যর্থ হলে নতুন পরিকল্পনা তৈরি করবে।

Reflection Engine

কাজ শেষ হওয়ার পরে ফলাফল মূল্যায়ন করবে এবং প্রয়োজনে পরিকল্পনা সংশোধন করবে।

Implementation Order

Planning & Reasoning Layer ধাপে ধাপে তৈরি হবে।

Stage 1

LLM-এর Native Reasoning ব্যবহার করা হবে।

কোন আলাদা Planner থাকবে না।

Stage 2

Basic Planner তৈরি হবে।

এটি সাধারণ Task Plan তৈরি করতে পারবে।

Stage 3

Task Breakdown যোগ হবে।

বড় কাজকে ছোট ধাপে ভাগ করা শুরু করবে।

Stage 4

Decision Engine যুক্ত হবে।

পরিস্থিতি অনুযায়ী Tool, Agent এবং Strategy নির্বাচন করবে।

Stage 5

Reflection এবং Self-Improvement যুক্ত হবে।

কাজ শেষ হওয়ার পরে নিজের সিদ্ধান্ত মূল্যায়ন করতে পারবে এবং ভবিষ্যতের পরিকল্পনা উন্নত করবে।

কেন **Future-**এ এটি গুরুত্বপূর্ণ হবে?

বর্তমানে ছোট Task-এর জন্য শুধুমাত্র LLM যথেষ্ট।

কিন্তু ভবিষ্যতে IRON BOLT যখন—

- বহু Agent পরিচালনা করবে
- অনেক Tool একসাথে ব্যবহার করবে
- দীর্ঘ Workflow চালাবে
- বড় Project সম্পন্ন করবে
- জটিল Automation পরিচালনা করবে

তখন একটি Dedicated Planning & Reasoning Layer ছাড়া দক্ষভাবে কাজ পরিচালনা করা কঠিন হবে।

এই Layer-ই IRON BOLT-কে সাধারণ Chat Assistant থেকে একটি বাস্তব **Intelligent AI Execution System**-এ রূপান্তর করার অন্যতম প্রধান ভিত্তি হবে।

file format

```
planning/
    ├── __init__.py
    ├── planner.py
    ├── goal_manager.py
    ├── task_decomposer.py
    ├── dependency_analyzer.py
    ├── strategy_selector.py
```

├── decision_engine.py

├── retry_planner.py

├── reflection_engine.py

├── execution_plan.py

├── planning_context.py

├── planning_state.py

├── planning_result.py

├── rules.py

└── exceptions.py

প্রতিটি ফাইলের কাজ সংক্ষেপে এমন হবে:

- planner.py → পুরো Planning System-এর Controller।
- goal_manager.py → User আসলে কী অর্জন করতে চায় তা নির্ধারণ করবে।
- task_decomposer.py → বড় কাজকে ছোট ধাপে ভাগ করবে।
- dependency_analyzer.py → কোন Step আগে ও পরে চলবে তা ঠিক করবে।
- strategy_selector.py → একাধিক সমাধানের মধ্যে সেরা Strategy বেছে নেবে।
- decision_engine.py → Runtime-এ Tool, Agent, Memory বা Provider নির্বাচন করবে।
- retry_planner.py → কোনো Step ব্যর্থ হলে বিকল্প পরিকল্পনা করবে।
- reflection_engine.py → কাজ শেষে নিজের ফলাফল মূল্যায়ন করবে।
- execution_plan.py → পুরো Plan-এর Data Model।
- planning_context.py → Planning চলাকালীন Context সংরক্ষণ করবে।
- planning_state.py → বর্তমান Planning State ট্র্যাক করবে।
- planning_result.py → Planner-এর Final Output বহন করবে।
- rules.py → Planning-এর নিয়ম ও সীমাবদ্ধতা সংরক্ষণ করবে।
- exceptions.py → Planning-সংক্রান্ত Error Handle করবে।

তবে AR1-এর জন্য এগুলোর সবগুলো একসাথে বানানোর প্রয়োজন নেই। AR1-এ LLM-এর native reasoning যথেষ্ট। এই subsystem মূলত AR2 বা পরবর্তী phase-এ ধাপে ধাপে তৈরি করা উচিত, যখন IRON BOLT বহু Agent, Tool এবং জটিল Workflow পরিচালনা করবে।

Tests

tests/

└── planning/

├── test_planner.py

├── test_goal_manager.py

├── test_task_decomposer.py

├── test_strategy_selector.py

├── test_decision_engine.py

├── test_retry_planner.py

└── test_reflection_engine.py

এরপর **AR2-**এ ধীরে ধীরে যোগ করবে**:**

- goal_manager.py
- task_decomposer.py
- dependency_analyzer.py
- strategy_selector.py
- decision_engine.py
- retry_planner.py
- reflection_engine.py

Chapter 13: Prompt Management

Overview

Prompt Management হলো IRON BOLT-এর এমন একটি Layer যার কাজ হলো Large Language Model (LLM)-এর কাছে পাঠানো প্রতিটি Prompt তৈরি, নিয়ন্ত্রণ এবং Standardize করা।

একজন User যখন কোনো Request পাঠায়, তখন সেই Request কখনোই সরাসরি Model-এর কাছে পাঠানো হয় না।

এর আগে IRON BOLT বিভিন্ন Source থেকে Context সংগ্রহ করে, System Rules যুক্ত করে, Memory যোগ করে, Tool-এর তথ্য যুক্ত করে, Agent-এর Instruction যোগ করে এবং শেষে একটি Complete Prompt তৈরি করে।

এই সম্পূর্ণ Process-টিই Prompt Management Layer পরিচালনা করে।

অর্থাৎ—

User যা লিখেছে, Model সেটি সরাসরি দেখে না।

Model দেখে—

- User Request
- System Instruction
- Current Context
- Conversation History
- Memory
- Planning Result
- Tool Information
- Workflow State
- Safety Rules
- Provider Specific Configuration

সবকিছু একত্রে একটি Structured Prompt হিসেবে।

এই Layer-এর প্রধান উদ্দেশ্য হলো—

LLM-কে এমনভাবে নির্দেশনা দেওয়া যাতে প্রতিবার একই ধরনের Quality, Consistency এবং Reliable Output পাওয়া যায়।

কেন **Prompt Management** প্রয়োজন

শুধু User Message Model-এ পাঠালে অনেক সমস্যা দেখা দেয়।

যেমন—

Model পূর্বের Conversation ভুলে যেতে পারে।

Memory ব্যবহার নাও করতে পারে।

ভুল Tool নির্বাচন করতে পারে।

Agent-এর Role Follow নাও করতে পারে।

Planning Result Ignore করতে পারে।

System Rules ভেঙে উত্তর দিতে পারে।

Provider পরিবর্তন হলে Prompt ভেঙে যেতে পারে।

একই প্রশ্নে বিভিন্ন সময় সম্পূর্ণ ভিন্ন উত্তর আসতে পারে।

Prompt Management Layer এই সমস্যাগুলো সমাধান করে।

এটি নিশ্চিত করে যে প্রতিটি Model একই ধরনের Information, একই Structure এবং একই Rule অনুসরণ করে কাজ করবে।

Prompt Management-এর দায়িত্ব

এই Layer-এর প্রধান দায়িত্বগুলো হলো—

User-এর মূল Request গ্রহণ করা।

System Prompt যুক্ত করা।

Conversation History যুক্ত করা।

Memory Context যুক্ত করা (যখন প্রয়োজন হবে)।

Planning Engine-এর Result যুক্ত করা।

Workflow-এর বর্তমান অবস্থা যুক্ত করা।

Tool-এর তথ্য যুক্ত করা।

Agent-এর Role এবং Instruction যুক্ত করা।

Provider অনুযায়ী Prompt Format পরিবর্তন করা।

Final Prompt তৈরি করা।

Provider Layer-এ Prompt পাঠানো।

Prompt তৈরির ধাপ

একটি Prompt একবারে তৈরি হয় না।

এটি ধাপে ধাপে তৈরি হয়।

সাধারণ Flow হবে—

User Request

↓

System Instruction

↓

Current Conversation

↓

Relevant Memory

↓

Planning Result

↓

Workflow Context

↓

Agent Instruction

↓

Tool Information

↓

Safety Rules

↓

Provider Formatting

↓

Final Prompt

↓

LLM

অর্থাৎ Prompt Management শুধুমাত্র একটি Text তৈরি করে না।

এটি বিভিন্ন Layer-এর তথ্য একত্র করে একটি Intelligent Context তৈরি করে।

Prompt-এর প্রধান Component

একটি Production-Level Prompt সাধারণত কয়েকটি অংশ নিয়ে গঠিত হয়।

১. System Prompt

এটি পুরো System-এর Identity নির্ধারণ করে।

এখানে লেখা থাকবে—

IRON BOLT কী,

কীভাবে উত্তর দেবে,

কোন Rule Follow করবে,

কোন কাজ করবে,

কোন কাজ করবে না।

এটি প্রতিটি Conversation-এর ভিত্তি।

---

২. User Prompt

User যা লিখেছে সেটি।

এখানে কোনো পরিবর্তন করা হবে না।

মূল Message যথাসম্ভব সংরক্ষণ করা হবে।

---

৩. Conversation Context

বর্তমান Conversation-এর গুরুত্বপূর্ণ অংশ।

যাতে Model বুঝতে পারে আগের আলোচনা কী ছিল।

---

৪. Memory Context

Memory System থেকে পাওয়া Relevant Information।

যদি Memory প্রয়োজন না হয়,

তাহলে এটি যোগ করা হবে না।

---

৫. Planning Context

Planning & Reasoning Engine যদি কোনো Execution Plan তৈরি করে,

সেটি Prompt-এর অংশ হবে।

যাতে Model পরিকল্পনা অনুযায়ী কাজ করতে পারে।

---

৬. Workflow Context

Workflow বর্তমানে কোন Step-এ আছে,

কোন Task সম্পন্ন হয়েছে,

কোন Step বাকি আছে,

এসব তথ্য এখানে থাকবে।

---

৭. Tool Context

যদি কোনো Tool ব্যবহার করতে হয়,

তাহলে—

Available Tool

Tool Capability

Tool Restriction

Tool Result

ইত্যাদি এখানে যুক্ত হবে।

---

৮. Agent Context

যদি কোনো Agent কাজ করে,

তাহলে—

তার Role

তার Goal

তার Responsibility

তার সীমাবদ্ধতা

Prompt-এ যুক্ত হবে।

---

৯. Safety Instruction

System-এর Security Rule,

Permission Rule,

Policy,

Execution Limitation

এখানে থাকবে।

যাতে Model কোনো Unsafe Action না করে।

---

১০. Provider Formatting

সব Model একই Prompt Format ব্যবহার করে না।

কিছু Provider আলাদা Message Format চায়।

Prompt Management সেই Format অনুযায়ী Prompt পরিবর্তন করবে।

এর ফলে Core System পরিবর্তন না করেও Provider পরিবর্তন করা সম্ভব হবে।

Prompt Template

IRON BOLT-এ Prompt Hardcoded থাকবে না।

বরং Template ব্যবহার করা হবে।

কারণ—

একটি সাধারণ Chat Prompt,

একটি Coding Prompt,

একটি Planning Prompt,

একটি Tool Prompt,

একটি Memory Prompt,

একটি Agent Prompt—

সবগুলোর Structure এক নয়।

Template ব্যবহারের মাধ্যমে প্রতিটি কাজের জন্য আলাদা Prompt তৈরি করা যাবে।

Dynamic Prompt Generation

IRON BOLT Static Prompt ব্যবহার করবে না।

প্রতিটি Request অনুযায়ী Prompt তৈরি হবে।

যেমন—

যদি Memory প্রয়োজন না হয়,

Memory Section বাদ যাবে।

যদি Tool প্রয়োজন না হয়,

Tool Section থাকবে না।

যদি Planning প্রয়োজন না হয়,

Planning অংশ যোগ হবে না।

অর্থাৎ Prompt শুধুমাত্র প্রয়োজনীয় তথ্যই বহন করবে।

এতে Token কম ব্যবহার হবে এবং Response আরও কার্যকর হবে।

Prompt Optimization

Prompt যত বড় হবে,

Model-এর Token Usage তত বাড়বে।

এ কারণে Prompt Management অপ্রয়োজনীয় তথ্য বাদ দেওয়ার চেষ্টা করবে।

যেমন—

পুরনো Conversation সংক্ষিপ্ত করা।

অপ্রাসঙ্গিক Memory বাদ দেওয়া।

Duplicate তথ্য সরিয়ে ফেলা।

অপ্রয়োজনীয় Instruction না পাঠানো।

এর ফলে Performance এবং Cost—দুইই উন্নত হবে।

অন্যান্য **Layer-**এর সাথে সংযোগ

Prompt Management Layer সরাসরি সংযুক্ত থাকবে—

Planning & Reasoning Layer-এর সাথে

Execution Plan গ্রহণ করার জন্য।

Memory System-এর সাথে

Relevant Memory যুক্ত করার জন্য।

Workflow Engine-এর সাথে

বর্তমান Workflow State জানার জন্য।

Agent System-এর সাথে

Agent-এর Role এবং Goal নেওয়ার জন্য।

Tool System-এর সাথে

Tool সম্পর্কিত Context যুক্ত করার জন্য।

Provider System-এর সাথে

Final Prompt পাঠানোর জন্য।

Security Layer-এর সাথে

Safety এবং Permission Rule যুক্ত করার জন্য।

বর্তমান Implementation (AR1)

বর্তমান AR1 Version-এ Prompt Management তুলনামূলকভাবে সহজ থাকবে।

এতে থাকবে—

একটি Basic System Prompt।

User Prompt Handling।

Conversation Context।

Provider-Compatible Prompt Format।

Future Expansion-এর জন্য Modular Structure।

এই Version-এর লক্ষ্য হলো Stable Prompt Pipeline তৈরি করা।

Future Implementation

পরবর্তী Version-গুলোতে Prompt Management আরও শক্তিশালী হবে।

যেমন—

Prompt Template Library।

Dynamic Prompt Composer।

Automatic Context Selection।

Memory Compression।

Conversation Summarization।

Agent-Specific Prompt।

Tool-Specific Prompt।

Workflow-Specific Prompt।

Provider-Specific Optimization।

Automatic Prompt Evaluation।

Prompt Version Control।

Prompt Caching।

Prompt Analytics।

Prompt Quality Scoring।

Self-Improving Prompt Pipeline।

এই Features যুক্ত হলে IRON BOLT বিভিন্ন ধরনের কাজের জন্য নিজেই সবচেয়ে উপযুক্ত Prompt তৈরি করতে পারবে।

Implementation Order

Prompt Management Layer নিম্নলিখিত ধাপে উন্নয়ন করা হবে—

প্রথমে Basic Prompt Pipeline তৈরি হবে।

এরপর System Prompt আলাদা করা হবে।

তারপর Template Structure যোগ হবে।

এরপর Conversation Context যুক্ত হবে।

তারপর Memory Integration হবে।

এরপর Planning Integration হবে।

তারপর Workflow Integration হবে।

এরপর Tool এবং Agent Context যুক্ত হবে।

এরপর Provider Specific Formatting যুক্ত হবে।

সবশেষে Prompt Optimization এবং Dynamic Prompt Generation যুক্ত হবে।

এই **Chapter-**এর ভূমিকা

Planning & Reasoning Layer সিদ্ধান্ত নেয় কী করতে হবে।

Prompt Management Layer সেই সিদ্ধান্ত, Context এবং System Rule একত্র করে Model-এর জন্য সর্বোত্তম Prompt তৈরি করে।

এরপর সেই Prompt Provider System-এর মাধ্যমে LLM-এ পাঠানো হয়।

পরবর্তী Chapter-এ আমরা দেখব কীভাবে বাইরের Plugin, External Service এবং MCP Server-এর সাথে IRON BOLT নিরাপদ ও Modular উপায়ে সংযুক্ত হবে।

file format:

```
prompt/
    ├── prompt_manager.py
    ├── prompt_builder.py
    ├── prompt_templates.py
    ├── context_collector.py
    ├── formatter.py
    ├── optimizer.py
    ├── template_registry.py
    ├── prompt_cache.py
```

```
    ├── prompt_validator.py
    ├── prompt_types.py
    └── versioning.py
```

AR1 এ যা যুকত হবে

```
prompt/
    ├── __init__.py
    ├── prompt_manager.py
    ├── prompt_builder.py
    ├── prompt_templates.py
    ├── context_collector.py
    ├── formatter.py
    └── prompt_types.py
```

Chapter 14: Plugin & MCP System

পরিচিতি

IRON BOLT-এর শুরুতে শুধুমাত্র Built-in Tool ব্যবহার করলেই অনেক কাজ করা সম্ভব। কিন্তু বাস্তব জীবনে একজন AI Assistant-কে অসংখ্য বাহ্যিক Service-এর সাথে কাজ করতে হয়। যেমন—

- Google Drive
- GitHub
- Gmail
- Slack
- Notion
- Calendar
- Database
- Cloud Storage
- CRM
- Internal Company API
- Custom Business Software

এই প্রতিটি **Service-**এর জন্য যদি আলাদা করে **Core Code** লিখতে হয়**,** তাহলে পুরো **Project** ধীরে ধীরে বিশাল**,** জটিল এবং **Maintain** করা কঠিন হয়ে যাবে।

এই সমস্যার সমাধান করার জন্য IRON BOLT-এ ভবিষ্যতে **Plugin & MCP System** যুক্ত করা হবে।

এটি এমন একটি **Extension Layer** হবে, যেখানে নতুন **Capability** যোগ করতে **Core Architecture** পরিবর্তন করতে হবে না। শুধু নতুন **Plugin** যুক্ত করলেই **IRON BOLT** নতুন **Service** ব্যবহার করতে পারবে।

Plugin System কী?

Plugin হলো এমন একটি Module, যা IRON BOLT-এর মূল Source Code পরিবর্তন না করেই নতুন Feature যোগ করতে দেয়।

অর্থাৎ—

Core System অপরিবর্তিত থাকবে।

Plugin আলাদা থাকবে।

যখন প্রয়োজন হবে তখন Plugin Load হবে।

প্রয়োজন না হলে Plugin Load-ই হবে না।

ফলে System ছোট, পরিষ্কার এবং Modular থাকবে।

MCP কী?

MCP-এর পূর্ণরূপ হলো **Model Context Protocol**।

এটি একটি Standard Communication Protocol।

এর কাজ হলো—

LLM যেন বিভিন্ন External Service-এর সাথে একই ধরনের নিয়মে যোগাযোগ করতে পারে।

যদি প্রতিটি Service-এর API আলাদা হয়, তাহলে প্রতিটির জন্য আলাদা Logic লিখতে হয়।

কিন্তু MCP থাকলে—

সব Service একটি Standard Interface অনুসরণ করবে।

ফলে LLM জানবে—

- কোন Tool আছে
- Tool কী করতে পারে

- কোন Parameter লাগবে
- কীভাবে Result ফিরবে

এটি Plugin System-কে আরও Standard এবং Scalable করে।

কেন **Plugin System** দরকার?

IRON BOLT-এর মূল লক্ষ্য হলো একটি দীর্ঘমেয়াদি AI Platform তৈরি করা।

আজ GitHub লাগতে পারে।

আগামীকাল Google Drive লাগতে পারে।

পরের সপ্তাহে Notion লাগতে পারে।

তারপর Company-এর নিজস্ব API লাগতে পারে।

যদি প্রতিবার Core পরিবর্তন করতে হয় তাহলে Development ধীর হয়ে যাবে।

Plugin System এই সমস্যা দূর করে।

কেন **MCP** দরকার?

Plugin থাকলেও যদি প্রতিটি Plugin ভিন্নভাবে কাজ করে তাহলে LLM বিভ্রান্ত হবে।

MCP নিশ্চিত করবে—

সব Plugin একই নিয়ম অনুসরণ করবে।

এর ফলে—

- Integration সহজ হবে
- Maintenance সহজ হবে
- নতুন Plugin দ্রুত যোগ করা যাবে
- বিভিন্ন Provider একই Plugin ব্যবহার করতে পারবে

Plugin System-এর প্রধান দায়িত্ব

Plugin Layer-এর কাজ হবে—

- Plugin Discover করা
- Plugin Load করা
- Plugin Register করা
- Plugin Enable/Disable করা
- Plugin Version Manage করা
- Plugin-এর Permission যাচাই করা
- Plugin-এর Tool গুলো IRON BOLT-এ Register করা
- Plugin Remove করা
- Plugin Update করা

MCP Layer-এর প্রধান দায়িত্ব

MCP Layer-এর কাজ হবে—

- MCP Server-এর সাথে সংযোগ করা
- Available Tool List নেওয়া
- Tool Schema পড়া
- Tool Call করা
- Result গ্রহণ করা
- Error Handle করা
- Standard Format বজায় রাখা

IRON BOLT-এর সাথে Plugin-এর সম্পর্ক

Plugin সরাসরি LLM-এর সাথে যুক্ত হবে না।

এটি একটি Controlled Layer-এর মাধ্যমে কাজ করবে।

সংযোগের ধাপ হবে—

User

↓

Planning & Reasoning

↓

Workflow Engine

↓

Permission System

↓

Plugin Manager

↓

Plugin / MCP Server

↓

External Service

↓

Plugin Manager

↓

Workflow Engine

↓

LLM

↓

User Response

এভাবে Core সবসময় Plugin-এর উপর নিয়ন্ত্রণ রাখবে।

কোন কোন **Component-**এর সাথে **Plugin** যুক্ত থাকবে?

Plugin System ভবিষ্যতে নিচের Component-গুলোর সাথে কাজ করবে—

Workflow Engine

কোন Plugin কখন ব্যবহার হবে তা Workflow Engine নির্ধারণ করবে।

Planning & Reasoning

LLM সিদ্ধান্ত নেবে কোন Plugin দরকার।

Prompt Management

Plugin-এর Description এবং Capability Prompt-এর মধ্যে যোগ করা হবে যাতে LLM জানে কোন Plugin কী করতে পারে।

Security & Permission

সব Plugin ব্যবহারের আগে Permission যাচাই হবে।

কোন Plugin File Access করতে পারবে?

কোন Plugin Internet ব্যবহার করতে পারবে?

কোন Plugin Database ব্যবহার করতে পারবে?

এসব Security Layer নিয়ন্ত্রণ করবে।

Tool System

Plugin-এর প্রতিটি Function শেষ পর্যন্ত একটি Tool হিসেবেই Register হবে।

তাই Plugin System এবং Tool System ঘনিষ্ঠভাবে সংযুক্ত থাকবে।

Memory System

Plugin ব্যবহারের ইতিহাস ভবিষ্যতে Memory-তে সংরক্ষণ করা যেতে পারে।

যেমন—

User প্রায়ই GitHub Plugin ব্যবহার করে।

তাহলে ভবিষ্যতে Planning আরও দ্রুত সিদ্ধান্ত নিতে পারবে।

Provider System

Provider যেই হোক—

OpenAI

Gemini

Groq

বা ভবিষ্যতের অন্য Model—

সবাই একই Plugin Interface ব্যবহার করবে।

বর্তমান Implementation

বর্তমান AR1 Version-এ Plugin System নেই।

এখন IRON BOLT শুধুমাত্র Internal Tool ব্যবহার করে।

বর্তমান Architecture ইচ্ছাকৃতভাবে ছোট রাখা হয়েছে যাতে Foundation শক্তিশালী হয়।

Future Implementation

ভবিষ্যতে নিচের Feature যোগ হবে—

- Plugin Manager
- Plugin Registry
- Plugin Loader
- Plugin Lifecycle Management
- Plugin Configuration

- MCP Client
- MCP Server Support
- Remote Plugin Support
- Local Plugin Support
- Plugin Sandbox
- Plugin Update System
- Plugin Marketplace
- Third-party Plugin Development SDK

এগুলো ধাপে ধাপে তৈরি হবে।

সম্ভাব্য **Folder Structure**

```
plugins/
│
├── manager.py
├── registry.py
├── loader.py
├── lifecycle.py
├── permissions.py
├── config.py
│
├── github/
├── gmail/
├── drive/
├── notion/
└── calendar/

mcp/
│
├── client.py
├── protocol.py
├── transport.py
├── registry.py
└── server_connection.py
```

এটি ভবিষ্যতের একটি সম্ভাব্য Structure। বাস্তব Implementation-এর সময় প্রয়োজন অনুযায়ী পরিবর্তন হতে পারে।

Implementation Order

Plugin & MCP System একবারে তৈরি করা হবে না।

ধাপে ধাপে উন্নয়ন হবে—

Phase 1

- Plugin Interface
- Plugin Manager
- Plugin Registry

↓

Phase 2

- Local Plugin Support
- Plugin Loading
- Plugin Configuration

↓

Phase 3

- MCP Client
- Standard Protocol
- External Tool Communication

↓

Phase 4

- Remote Plugin
- Version Management
- Security Validation

↓

Phase 5

- Plugin Marketplace
- Third-party Plugin SDK
- Enterprise Plugin Support

এই ধাপগুলো অনুসরণ করলে প্রতিটি স্তর স্থিতিশীল থাকবে এবং পরবর্তী স্তর সহজে তৈরি করা যাবে।

এই **Chapter-**এর সারসংক্ষেপ

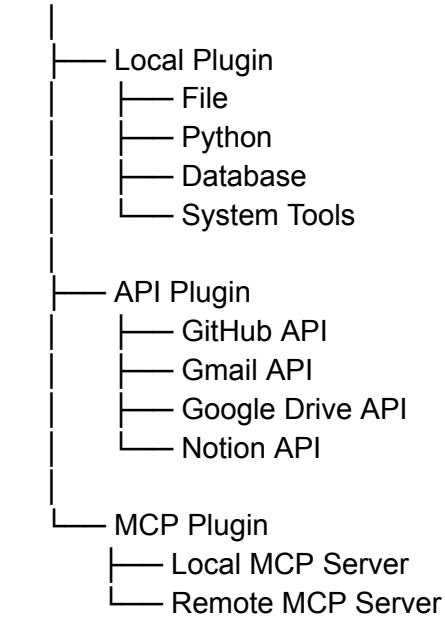
Plugin & MCP System হলো IRON BOLT-এর ভবিষ্যৎ সম্প্রসারণের ভিত্তি। এটি Core System-কে পরিবর্তন না করে নতুন Capability যোগ করার সুযোগ দেবে। Plugin System নতুন Feature সংযুক্ত করবে, আর MCP বিভিন্ন বাহ্যিক Service-এর সাথে একটি Standard উপায়ে যোগাযোগ নিশ্চিত করবে। Workflow Engine, Planning & Reasoning, Tool System, Security, Memory এবং Provider System—সবগুলোর সাথে সমন্বয় করে এই Layer কাজ করবে। যদিও AR1-এ এটি অন্তর্ভুক্ত নয়, ভবিষ্যতের উন্নয়নে এটি IRON BOLT-কে একটি সত্যিকারের Extensible, Modular এবং Production-grade AI Platform-এ পরিণত করার অন্যতম প্রধান ভিত্তি হবে।

IRON BOLT-এর ক্ষেত্রে

তোমার Roadmap অনুযায়ী ভবিষ্যতে সম্ভবত তিনটিই থাকবে—

- Built-in Local Plugin (API লাগবে না)
- API-based Plugin (GitHub, Gmail, Google Drive ইত্যাদি)
- MCP Server Integration (Local ও Remote—দুই ধরনের)

সেরা ডিজাইন হবে **Hybrid ArchiteServer**

IRON BOLT


Chapter 15 — Security & Permissions

পরিচিতি

যেকোনো শক্তিশালী AI Assistant-এর সবচেয়ে গুরুত্বপূর্ণ অংশগুলোর একটি হলো **Security & Permissions System**।

একটি AI যদি শুধুমাত্র প্রশ্নের উত্তর দেয়, তাহলে নিরাপত্তার প্রয়োজন তুলনামূলক কম। কিন্তু IRON BOLT-এর লক্ষ্য শুধুমাত্র Chat করা নয়।

IRON BOLT ভবিষ্যতে—

- Computer-এর File Access করবে।
- GitHub Repository পরিবর্তন করবে।
- API Call করবে।
- Database Access করবে।
- Email পাঠাবে।
- Workflow Execute করবে।
- বিভিন্ন Plugin ব্যবহার করবে।
- MCP Server-এর মাধ্যমে External Service-এর সাথে যোগাযোগ করবে।

অর্থাৎ IRON BOLT বাস্তব পৃথিবীতে Action নিতে পারবে।

এই কারণে Security Layer ছাড়া একটি AI Assistant কখনোই Production Ready হতে পারে না।

Security & Permissions System-এর কাজ হলো—

"IRON BOLT কী করতে পারবে**,** কী করতে পারবে না**,** কখন পারবে**,** কার অনুমতিতে পারবে এবং কোন **Action** কতটা নিরাপদ**—**সবকিছু নিয়ন্ত্রণ করা।**"**

এই Layer পুরো System-এর নিরাপত্তার মূল ভিত্তি।

কেন **Security System** প্রয়োজন

ধরো User লিখল—

"আমার Documents Folder Delete করে দাও।"

অথবা—

"GitHub Repository Force Push করে দাও।"

অথবা—

"আমার Gmail-এর সব Email Delete করো।"

যদি AI কোনো Permission ছাড়াই এগুলো Execute করে ফেলে, তাহলে একটি ছোট Prompt-এর কারণে বিশাল ক্ষতি হতে পারে।

তাই IRON BOLT কখনো Blind Execution করবে না।

প্রতিটি গুরুত্বপূর্ণ Action প্রথমে Security Layer-এর মধ্য দিয়ে যাবে।

Security Layer যাচাই করবে—

- User-এর অনুমতি আছে কি না।
- Action নিরাপদ কি না।
- Resource Access করা যাবে কি না।
- অতিরিক্ত Confirmation লাগবে কি না।
- কোনো Security Rule ভাঙা হচ্ছে কি না।

সবকিছু ঠিক থাকলে তবেই Execution শুরু হবে।

Security Layer কোথায় কাজ করবে

Security Layer কোনো একটি Component নয়।

এটি পুরো Architecture জুড়ে কাজ করবে।

প্রধানত এটি সংযুক্ত থাকবে—

- Provider System
- Memory System
- Agent System
- Tool System
- Workflow Engine
- Plugin System
- MCP System
- Future API Layer

অর্থাৎ যখনই কোনো Component বাস্তব Action নিতে চাইবে, Security Layer তার আগে সিদ্ধান্ত নেবে Action অনুমোদিত হবে কি না।

Permission কী

Permission হলো কোনো নির্দিষ্ট কাজ করার অনুমতি।

উদাহরণ—

User যদি শুধুমাত্র Chat করতে চায়,

তাহলে কোনো File Access Permission-এর প্রয়োজন নেই।

আবার যদি User বলে—

"Downloads Folder-এর PDF পড়ো"

তখন File Read Permission প্রয়োজন হবে।

আবার যদি User বলে—

"Downloads Folder Delete করে দাও"

তখন Delete Permission প্রয়োজন হবে।

অর্থাৎ প্রতিটি Action-এর জন্য আলাদা Permission থাকবে।

Permission-এর ধরন

IRON BOLT ভবিষ্যতে বিভিন্ন ধরনের Permission ব্যবহার করবে।

যেমন—

Read Permission

শুধুমাত্র তথ্য পড়তে পারবে।

উদাহরণ—

- File পড়া

- Database Query
- Email পড়া
- GitHub Repository Clone করা

কোনো পরিবর্তন করা যাবে না।

Write Permission

নতুন Data তৈরি বা পরিবর্তন করার অনুমতি।

উদাহরণ—

- File Save করা
- File Edit করা
- Note তৈরি করা
- GitHub Commit তৈরি করা

Delete Permission

সবচেয়ে সংবেদনশীল Permission।

এটি ছাড়া কোনো Resource Delete করা যাবে না।

উদাহরণ—

- File Delete
- Folder Delete
- Database Record Delete
- Email Delete

Execute Permission

কোনো Program বা Command চালানোর অনুমতি।

উদাহরণ—

- Python Script Run
- Shell Command
- Workflow Execute
- External Tool চালানো

Network Permission

Internet-এর সাথে যোগাযোগের অনুমতি।

যেমন—

- API Call
- Web Search
- Cloud Database
- Remote Server

Plugin Permission

External Plugin ব্যবহার করার অনুমতি।

যেমন—

- GitHub
- Google Drive
- Notion
- Dropbox
- Gmail

Memory Permission

Memory Access-এর Permission।

যেমন—

- Memory পড়া
- Memory Update
- Memory Delete

সব Memory সবসময় Access করা যাবে না।

Permission Level

সব Permission সমান গুরুত্বপূর্ণ নয়।

তাই IRON BOLT বিভিন্ন Level ব্যবহার করবে।

উদাহরণ—

Low Risk

যেসব Action সাধারণত নিরাপদ।

যেমন—

- Calculator
- Local Text Processing
- সাধারণ Chat

Medium Risk

যেখানে Data পরিবর্তন হতে পারে।

যেমন—

- File Edit
- Note Save
- Git Commit

High Risk

যেখানে বড় ক্ষতি হতে পারে।

যেমন—

- File Delete
- Force Push
- Database Delete
- System Command

High Risk Action-এর জন্য অতিরিক্ত Confirmation প্রয়োজন হবে।

Confirmation System

সব Action একইভাবে Execute হবে না।

কিছু Action User-এর Confirmation ছাড়া কখনো Execute করা হবে না।

উদাহরণ—

User বলল—

"Downloads Folder Delete করে দাও।"

IRON BOLT সরাসরি Delete করবে না।

প্রথমে জিজ্ঞাসা করবে—

"আপনি কি নিশ্চিত? এই Action Undo করা যাবে না।"

User নিশ্চিত করার পরই Action Execute হবে।

এতে দুর্ঘটনাজনিত ভুল কমে যাবে।

Tool Security

Tool System Security Layer-এর সাথে সরাসরি সংযুক্ত থাকবে।

Tool কোনো Action শুরু করার আগে Security Layer যাচাই করবে—

- Tool ব্যবহার করা যাবে কি না।
- Permission আছে কি না।
- Resource নিরাপদ কি না।
- অতিরিক্ত Confirmation দরকার কি না।

এরপর Tool Execute হবে।

Workflow Security

Workflow Engine একসাথে অনেকগুলো Step Execute করতে পারে।

যেমন—

- File Read
- API Call
- Database Update
- Email Send

Workflow-এর প্রতিটি Step আলাদাভাবে Security Check-এর মধ্য দিয়ে যাবে।

একটি Step অনুমোদিত হলেও পরবর্তী Step আবার যাচাই করা হবে।

ফলে পুরো Workflow নিরাপদ থাকবে।

Plugin Security

Plugin হলো বাইরের System-এর সাথে সংযোগ।

যেমন—

- GitHub
- Gmail
- Google Drive
- Notion

এই ধরনের Service-এ Access করার আগে—

- Authentication
- Permission Validation
- Scope Verification

সম্পন্ন হবে।

Plugin কখনোই নিজের ইচ্ছামতো User-এর Data Access করতে পারবে না।

MCP Security

ভবিষ্যতে MCP Server ব্যবহার করলে বিভিন্ন External Server-এর সাথে যোগাযোগ হবে।

Security Layer নিশ্চিত করবে—

- Trusted Server কিনা।
- কোন Capability ব্যবহার করা যাবে।
- কোন Resource Access করা যাবে।
- কোন Permission অনুমোদিত।

এর ফলে একটি অনিরাপদ MCP Server পুরো System-এর ক্ষতি করতে পারবে না।

Memory Security

Memory System-ও Security-এর অধীনে থাকবে।

সব Agent সব Memory Access করতে পারবে না।

প্রয়োজনে Memory Read, Memory Update এবং Memory Delete—প্রতিটি Action-এর জন্য আলাদা Permission প্রয়োগ করা হবে।

এর ফলে ব্যক্তিগত তথ্য আরও নিরাপদ থাকবে।

Audit Log

Production System-এ প্রতিটি গুরুত্বপূর্ণ Action-এর একটি Record রাখা হবে।

যেমন—

- কখন Action হয়েছে।
- কোন User করেছে।
- কোন Tool ব্যবহার হয়েছে।
- কোন Permission ব্যবহৃত হয়েছে।
- Action সফল হয়েছে নাকি ব্যর্থ হয়েছে।

এই Record ভবিষ্যতে Debugging, Monitoring এবং Security Investigation-এ গুরুত্বপূর্ণ ভূমিকা রাখবে।

Current Implementation

বর্তমান AR1 Version-এ Security System সম্পূর্ণ বাস্তবায়িত নয়।

বর্তমানে মূল লক্ষ্য হলো Foundation Architecture তৈরি করা।

বর্তমান অবস্থায়—

- Basic Structure তৈরি হবে।
- Permission Framework-এর ভিত্তি থাকবে।
- ভবিষ্যতের Integration Point নির্ধারণ করা হবে।

Advanced Security এখনো Implement করা হবে না।

Future Implementation

ভবিষ্যতের Version-গুলোতে ধাপে ধাপে যুক্ত হবে—

- Complete Permission Manager
- Role-Based Access Control (RBAC)
- Fine-Grained Permission System
- Multi-Level Confirmation
- Secure Credential Management
- Plugin Sandbox
- MCP Permission Control
- Workflow Policy Engine
- Tool Isolation
- Audit Logging System
- Security Rules Engine
- Encryption Support
- Secret Management
- API Authentication
- Session Security
- Advanced Access Policy
- Enterprise Security Features

এর মাধ্যমে IRON BOLT শুধুমাত্র একটি বুদ্ধিমান AI Assistant নয়, বরং বাস্তব কাজ করার জন্য নিরাপদ, নির্ভরযোগ্য এবং Production-Grade AI Platform-এ পরিণত হবে।

পরবর্তী **Chapter**

পরবর্তী Chapter-এ আলোচনা করা হবে—

Chapter 16 — Execution Flow

সেখানে দেখানো হবে User একটি Prompt দেওয়ার পর সেটি কীভাবে ধাপে ধাপে Provider, Planning, Tool, Memory, Workflow এবং Response Pipeline অতিক্রম করে চূড়ান্ত উত্তর বা Action-এ পৌঁছায়।

প্রতিটি **Action-**এর জন্য প্রয়োজনীয় **Permission** নির্ধারিত থাকবে। একই ধরনের বহু **Action** একটি **Permission** বা **Permission Group-**এর অধীনে থাকতে পারে।

AR1-এর জন্য **Structure**টা এভাবে রাখতাম**:**

- security/
- ├── permission_manager.py
- ├── permission_registry.py
- ├── action_mapping.py
- ├── confirmation.py
- ├── models.py
- └── exceptions.py

আর **AR2** বা পরবর্তী ভার্সনে প্রয়োজন হলে যোগ হবে**:**

- risk_policy.py
- policy_engine.py
- rbac.py
- audit_logger.py

Chapter 16: Execution Flow

Overview

Execution Flow হলো IRON BOLT-এর সম্পূর্ণ কাজ করার ধাপ।

এটি নির্ধারণ করে একজন ব্যবহারকারী কোনো প্রশ্ন বা নির্দেশনা দেওয়ার পর সেই অনুরোধ কীভাবে পুরো সিস্টেমের ভিতর দিয়ে প্রবাহিত হবে, কোন Component কখন কাজ করবে, কোন তথ্য কোথায় যাবে, কখন Tool ব্যবহার হবে,

কখন Memory ব্যবহার হবে, কখন Agent সক্রিয় হবে এবং শেষ পর্যন্ত কীভাবে একটি নিরাপদ, নির্ভুল ও বুদ্ধিমান উত্তর তৈরি হবে।

সহজভাবে বলতে গেলে,

Execution Flow হলো **IRON BOLT-**এর সম্পূর্ণ কাজ করার রোডম্যাপ।

এটি শুধুমাত্র একটি Function Call-এর তালিকা নয়।

বরং এটি পুরো System-এর Operational Blueprint।

কেন **Execution Flow** প্রয়োজন

IRON BOLT-এর ভিতরে অনেকগুলো বড় Component রয়েছে।

যেমন—

- Core Layer
- Provider System
- Prompt Management
- Planning & Reasoning
- Workflow Engine
- Tool System
- Memory System
- Agent System
- Security System

যদি এদের কাজ করার নির্দিষ্ট ক্রম না থাকে, তাহলে—

- Component একে অপরের সাথে সংঘর্ষ করতে পারে।
- একই কাজ একাধিকবার হতে পারে।
- Tool ভুল সময়ে চালু হতে পারে।
- Memory অপ্রয়োজনীয়ভাবে ব্যবহার হতে পারে।
- Response তৈরির সময় অপ্রয়োজনীয় Delay হতে পারে।
- ভবিষ্যতে নতুন Feature যোগ করা কঠিন হয়ে যাবে।

Execution Flow এই সমস্যাগুলো দূর করে।

Execution Flow-এর মূল লক্ষ্য

Execution Flow-এর প্রধান উদ্দেশ্য হলো—

- প্রতিটি Request-এর জন্য একটি নির্দিষ্ট Processing Path তৈরি করা।
- Component-গুলোর দায়িত্ব পরিষ্কারভাবে নির্ধারণ করা।
- System-কে Predictable করা।
- ভবিষ্যতে Feature যোগ করা সহজ করা।
- Performance উন্নত রাখা।
- Reliability বৃদ্ধি করা।
- Debugging সহজ করা।

বর্তমান (Current) Execution Flow

বর্তমান Architecture অনুযায়ী IRON BOLT একটি তুলনামূলক সরল Flow অনুসরণ করে।

User
  |
  ▼
Core
  |
  ▼
Prompt Manager
  |
  ▼
Provider Manager
  |
  ▼
Selected LLM
  |
  ▼
Response

বর্তমান সংস্করণে—

- Request Receive হয়।
- Prompt প্রস্তুত হয়।
- Provider নির্বাচন হয়।
- Model Response তৈরি করে।
- Response ব্যবহারকারীর কাছে ফিরে যায়।

এই Flow মূল Foundation তৈরি করে।

ভবিষ্যতের **Production Execution Flow**

IRON BOLT-এর পূর্ণাঙ্গ সংস্করণে Execution Flow অনেক বেশি শক্তিশালী হবে।

উচ্চ পর্যায়ের Flow হবে—

User
  |
  ▼
Core Layer
  |
  ▼
Security & Permission
  |
  ▼
Prompt Management
  |
  ▼
Planning & Reasoning
  |
  ▼
Workflow Engine
  |
  ▼
Agent Decision
  |
  ▼
Tool / Memory / Plugin / MCP
  |
  ▼
Provider System
  |
  ▼
LLM
  |
  ▼
Response Validation
  |
  ▼
Memory Update
  |

▼
Final Response

এখানে প্রতিটি Component নির্দিষ্ট দায়িত্ব পালন করবে।

ধাপ ১ — User Request গ্রহণ

সবকিছু শুরু হবে User Request থেকে।

উদাহরণ—

"আজকের Weather বলো"

অথবা

"আমার GitHub Issue ঠিক করো"

অথবা

"আগের Conversation মনে আছে?"

এই Request প্রথমে Core Layer-এ পৌঁছাবে।

ধাপ ২ — Core Layer

Core Layer পুরো System-এর Entry Point।

এখানে—

- Request গ্রহণ করা হবে।
- Basic Validation হবে।
- Request ID তৈরি হতে পারে।
- Runtime Context তৈরি হবে।
- System State প্রস্তুত হবে।

এরপর Request পরবর্তী Component-এ পাঠানো হবে।

ধাপ ৩ — Security & Permission

এখানে Request নিরাপদ কিনা তা পরীক্ষা করা হবে।

যেমন—

- User কি Tool ব্যবহার করতে পারবে?
- কোনো Restricted Action আছে কি?
- কোনো Dangerous Command আছে কি?
- কোনো Permission দরকার কি?

যদি Permission Fail করে—

Flow এখানেই থেমে যাবে।

ধাপ ৪ — Prompt Management

Prompt Manager পুরো Request-কে LLM-এর জন্য প্রস্তুত করবে।

এখানে—

- System Prompt যুক্ত হবে।
- User Prompt যুক্ত হবে।
- Runtime Context যুক্ত হবে।
- Conversation History যুক্ত হবে।
- প্রয়োজন হলে Memory Context যুক্ত হবে।
- Tool Information যুক্ত হবে।

এরপর একটি Final Prompt তৈরি হবে।

ধাপ ৫ — Planning & Reasoning

এটি সিদ্ধান্ত নেবে—

এই Request সমাধানের জন্য কী কী ধাপ প্রয়োজন।

যেমন—

"GitHub Issue Fix করো"

তখন Planning করতে পারে—

- Repository খুঁজে বের করো।
- Issue পড়ো।
- Code বিশ্লেষণ করো।
- Fix তৈরি করো।
- Test চালাও।
- Commit প্রস্তুত করো।

এটি সরাসরি কাজ করে না।

এটি পরিকল্পনা তৈরি করে।

ধাপ ৬ — Workflow Engine

Planning শেষ হলে Workflow Engine সেই পরিকল্পনাকে Executable Workflow-এ রূপান্তর করবে।

যদি একাধিক ধাপ থাকে—

Workflow Engine ঠিক করবে—

- কোন Step আগে চলবে।
- কোন Step পরে চলবে।
- কোন Step Fail করলে কী হবে।
- Retry হবে কিনা।
- কোন Output কোথায় যাবে।

ধাপ ৭ — Agent Decision

Agent সিদ্ধান্ত নেবে—

নিজে উত্তর দেওয়া যথেষ্ট নাকি অতিরিক্ত Resource দরকার।

সম্ভাব্য সিদ্ধান্ত—

- শুধুমাত্র LLM
- Tool দরকার

- Memory দরকার
- Plugin দরকার
- MCP দরকার
- একাধিক Agent দরকার

Agent হচ্ছে Decision Maker।

ধাপ ৮ — Tool / Memory / Plugin / MCP

যদি অতিরিক্ত তথ্যের প্রয়োজন হয়—

তখন Agent সংশ্লিষ্ট Component ব্যবহার করবে।

উদাহরণ—

Weather জানতে—

→ Weather Tool

GitHub Fix করতে—

→ GitHub Plugin

আগের Conversation জানতে—

→ Memory

Local File পড়তে—

→ MCP

Database Query করতে—

→ Connector

সব Result আবার Workflow-এ ফিরে আসবে।

ধাপ ৯ — Provider System

সব তথ্য প্রস্তুত হলে Provider Manager ঠিক করবে—

কোন Model ব্যবহার হবে।

উদাহরণ—

- GPT
- Gemini
- Groq
- ভবিষ্যতের অন্য Provider

Provider Request-কে নির্বাচিত Model-এর Format-এ রূপান্তর করবে।

ধাপ ১০ — LLM Execution

এখানে Model আসল Response তৈরি করবে।

Model ব্যবহার করবে—

- User Prompt
- Planning Result
- Tool Result
- Memory
- Workflow Context
- System Instruction

তারপর Response তৈরি করবে।

ধাপ ১১ — Response Validation

Response সরাসরি User-এর কাছে যাবে না।

প্রথমে Validation হবে।

যেমন—

- Response Complete কি?
- Format ঠিক আছে?
- কোনো Error আছে?
- কোনো Sensitive তথ্য আছে?
- কোনো Tool Output অসম্পূর্ণ কি?

প্রয়োজন হলে Response সংশোধন করা হবে।

ধাপ ১২ — Memory Update

যদি Conversation ভবিষ্যতের জন্য গুরুত্বপূর্ণ হয়—

তাহলে Memory System নতুন তথ্য সংরক্ষণ করতে পারে।

সব Conversation Memory-তে যাবে না।

শুধুমাত্র প্রয়োজনীয় তথ্য সংরক্ষণ করা হবে।

ধাপ ১৩ — Final Response

সবশেষে Final Response ব্যবহারকারীর কাছে পাঠানো হবে।

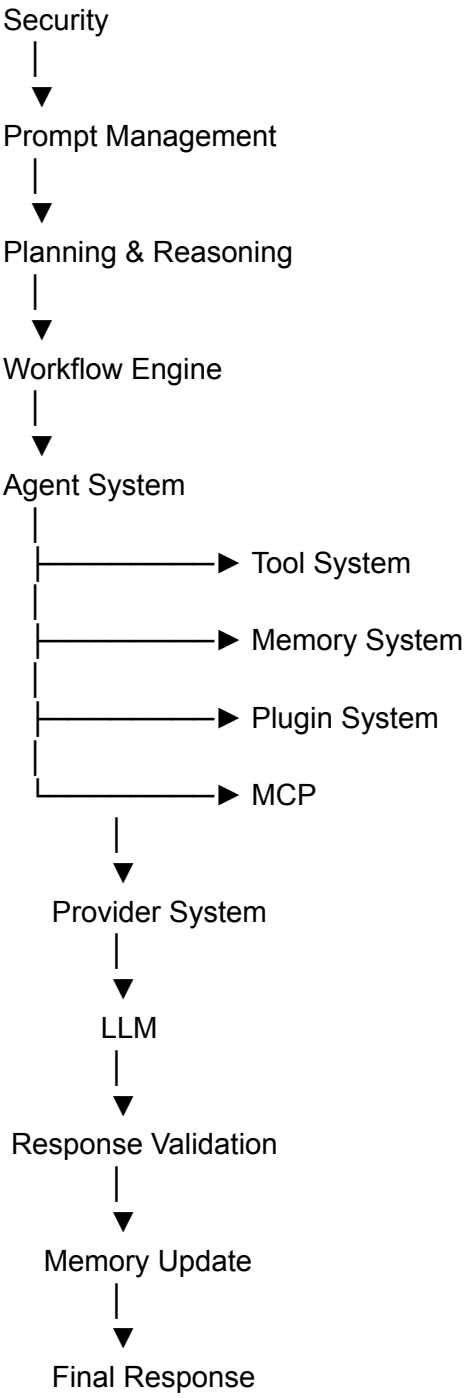
এই Response হবে—

- নিরাপদ
- যাচাইকৃত
- প্রয়োজনীয় Context-সহ
- Tool Result-সহ
- Memory-aware
- Production Ready

Component Connection

Execution Flow-এর সময় Component-গুলোর সম্পর্ক হবে—

User
  |
  ▼
Core Layer
  |
  ▼



বর্তমান Implementation

বর্তমানে IRON BOLT-এ রয়েছে—

- Core Foundation
- Configuration System
- Provider Manager
- Provider Abstraction
- Runtime Validation
- Prompt Processing-এর প্রাথমিক ভিত্তি
- LLM Communication

Execution Flow এখনো সরল এবং Foundation-কেন্দ্রিক।

ভবিষ্যতের **Implementation**

পরবর্তী সংস্করণগুলোতে যুক্ত হবে—

- সম্পূর্ণ Planning Pipeline
- Intelligent Workflow Execution
- Multi-Agent Coordination
- Dynamic Tool Routing
- Advanced Memory Integration
- Plugin Execution Pipeline
- MCP Communication Layer
- Response Validation Engine
- Streaming Response Pipeline
- Background Task Execution
- Parallel Workflow Support
- Failure Recovery System
- Retry Strategy
- Execution Monitoring
- Telemetry ও Performance Analytics

এগুলো যুক্ত হওয়ার মাধ্যমে Execution Flow একটি Production-grade Intelligent Execution Pipeline-এ পরিণত হবে, যা জটিল অনুরোধও ধাপে ধাপে পরিকল্পনা, সিদ্ধান্ত, কার্যসম্পাদন এবং যাচাইয়ের মাধ্যমে নির্ভরযোগ্যভাবে সম্পন্ন করতে সক্ষম হবে।

Chapter 16: Execution Flow

Overview

Execution Flow হলো IRON BOLT-এর সম্পূর্ণ কাজ করার ধাপ।

এটি নির্ধারণ করে একজন ব্যবহারকারী কোনো প্রশ্ন বা নির্দেশনা দেওয়ার পর সেই অনুরোধ কীভাবে পুরো সিস্টেমের ভিতর দিয়ে প্রবাহিত হবে, কোন Component কখন কাজ করবে, কোন তথ্য কোথায় যাবে, কখন Tool ব্যবহার হবে, কখন Memory ব্যবহার হবে, কখন Agent সক্রিয় হবে এবং শেষ পর্যন্ত কীভাবে একটি নিরাপদ, নির্ভুল ও বুদ্ধিমান উত্তর তৈরি হবে।

সহজভাবে বলতে গেলে,

Execution Flow হলো **IRON BOLT-**এর সম্পূর্ণ কাজ করার রোডম্যাপ।

এটি শুধুমাত্র একটি Function Call-এর তালিকা নয়।

বরং এটি পুরো System-এর Operational Blueprint।

কেন **Execution Flow** প্রয়োজন

IRON BOLT-এর ভিতরে অনেকগুলো বড় Component রয়েছে।

যেমন—

- Core Layer
- Provider System
- Prompt Management
- Planning & Reasoning
- Workflow Engine
- Tool System
- Memory System
- Agent System
- Security System

যদি এদের কাজ করার নির্দিষ্ট ক্রম না থাকে, তাহলে—

- Component একে অপরের সাথে সংঘর্ষ করতে পারে।
- একই কাজ একাধিকবার হতে পারে।
- Tool ভুল সময়ে চালু হতে পারে।
- Memory অপ্রয়োজনীয়ভাবে ব্যবহার হতে পারে।

- Response তৈরির সময় অপ্রয়োজনীয় Delay হতে পারে।
- ভবিষ্যতে নতুন Feature যোগ করা কঠিন হয়ে যাবে।

Execution Flow এই সমস্যাগুলো দূর করে।

Execution Flow-এর মূল লক্ষ্য

Execution Flow-এর প্রধান উদ্দেশ্য হলো—

- প্রতিটি Request-এর জন্য একটি নির্দিষ্ট Processing Path তৈরি করা।
- Component-গুলোর দায়িত্ব পরিষ্কারভাবে নির্ধারণ করা।
- System-কে Predictable করা।
- ভবিষ্যতে Feature যোগ করা সহজ করা।
- Performance উন্নত রাখা।
- Reliability বৃদ্ধি করা।
- Debugging সহজ করা।

বর্তমান (Current) Execution Flow (AR0)

বর্তমান Architecture অনুযায়ী IRON BOLT একটি তুলনামূলক সরল Flow অনুসরণ করে।

User

|

▼

main.py

|

▼

Orchestrator

|

▼

Router

|

▼

Provider Manager

|

▼

Groq Provider (Default)

|

▼

GPT-OSS-120B

|

▼

Response

বর্তমান সংস্করণে—

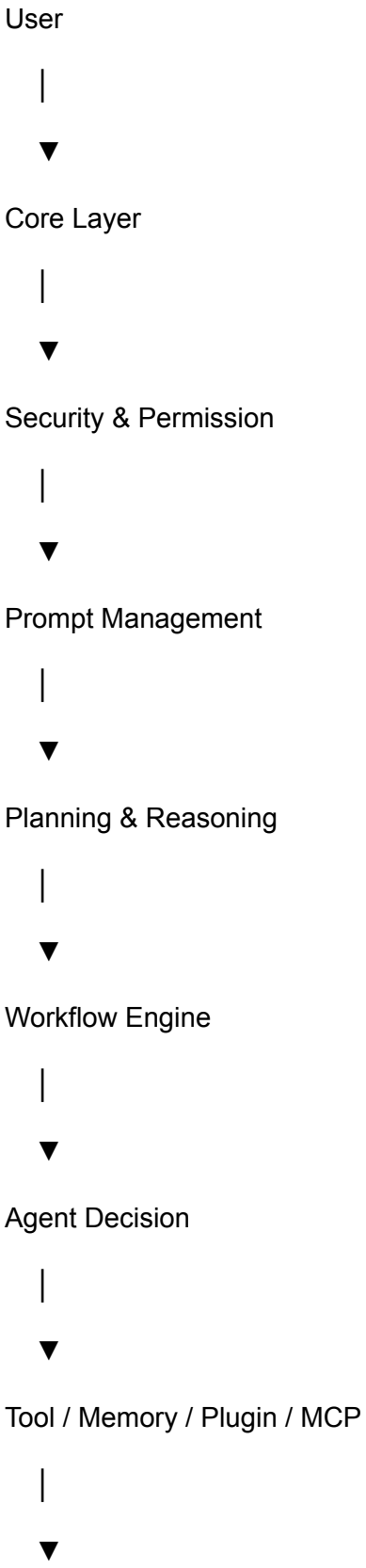
- Request Receive হয়।
- Prompt প্রস্তুত হয়।
- Provider নির্বাচন হয়।
- Model Response তৈরি করে।
- Response ব্যবহারকারীর কাছে ফিরে যায়।

এই Flow মূল Foundation তৈরি করে।

ভবিষ্যতের **Production Execution Flow**

IRON BOLT-এর পূর্ণাঙ্গ সংস্করণে Execution Flow অনেক বেশি শক্তিশালী হবে।

উচ্চ পর্যায়ের Flow হবে—


User
Core Layer
Security & Permission
Prompt Management
Planning & Reasoning
Workflow Engine
Agent Decision
Tool / Memory / Plugin / MCP

Provider System

|

▼

LLM

|

▼

Response Validation

|

▼

Memory Update

|

▼

Final Response

এখানে প্রতিটি Component নির্দিষ্ট দায়িত্ব পালন করবে।

ধাপ ১ — **User Request** গ্রহণ

সবকিছু শুরু হবে User Request থেকে।

উদাহরণ—

"আজকের Weather বলো"

অথবা

"আমার GitHub Issue ঠিক করো"

অথবা

"আগের Conversation মনে আছে?"

এই Request প্রথমে Core Layer-এ পৌঁছাবে।

ধাপ ২ — Core Layer

Core Layer পুরো System-এর Entry Point।

এখানে—

- Request গ্রহণ করা হবে।
- Basic Validation হবে।
- Runtime Context তৈরি হবে।
- System State প্রস্তুত হবে।

এরপর Request পরবর্তী Component-এ পাঠানো হবে।

ধাপ ৩ — Security & Permission

এখানে Request নিরাপদ কিনা তা পরীক্ষা করা হবে।

যেমন—

- User কি Tool ব্যবহার করতে পারবে?
- কোনো Restricted Action আছে কি?
- কোনো Dangerous Command আছে কি?
- কোনো Permission দরকার কি?

যদি Permission Fail করে—

Flow এখানেই থেমে যাবে।

ধাপ ৪ — Prompt Management

Prompt Manager পুরো Request-কে LLM-এর জন্য প্রস্তুত করবে।

এখানে—

- System Prompt যুক্ত হবে।
- User Prompt যুক্ত হবে।
- Runtime Context যুক্ত হবে।
- Conversation History যুক্ত হবে।
- প্রয়োজন হলে Memory Context যুক্ত হবে।
- Tool Information যুক্ত হবে।

এরপর একটি Final Prompt তৈরি হবে।

ধাপ ৫ — Planning & Reasoning

এটি সিদ্ধান্ত নেবে—

এই Request সমাধানের জন্য কী কী ধাপ প্রয়োজন।

যেমন—

"GitHub Issue Fix করো"

তখন Planning করতে পারে—

- Repository খুঁজে বের করো।
- Issue পড়ো।
- Code বিশ্লেষণ করো।
- Fix তৈরি করো।
- Test চালাও।
- Commit প্রস্তুত করো।

এটি সরাসরি কাজ করে না।

এটি পরিকল্পনা তৈরি করে।

ধাপ ৬ — Workflow Engine

Planning শেষ হলে Workflow Engine সেই পরিকল্পনাকে Executable Workflow-এ রূপান্তর করবে।

যদি একাধিক ধাপ থাকে—

Workflow Engine ঠিক করবে—

- কোন Step আগে চলবে।
- কোন Step পরে চলবে।
- কোন Step Fail করলে কী হবে।
- Retry হবে কিনা।
- কোন Output কোথায় যাবে।

ধাপ ৭ — Agent Decision

Agent সিদ্ধান্ত নেবে—

নিজে উত্তর দেওয়া যথেষ্ট নাকি অতিরিক্ত Resource দরকার।

সম্ভাব্য সিদ্ধান্ত—

- শুধুমাত্র LLM
- Tool দরকার
- Memory দরকার
- Plugin দরকার
- MCP দরকার
- একাধিক Agent দরকার

Agent হচ্ছে Decision Maker।

ধাপ ৮ — Tool / Memory / Plugin / MCP

যদি অতিরিক্ত তথ্যের প্রয়োজন হয়—

তখন Agent সংশ্লিষ্ট Component ব্যবহার করবে।

উদাহরণ—

Weather জানতে—

→ Weather Tool

GitHub Fix করতে—

→ GitHub Plugin

আগের Conversation জানতে—

→ Memory

Local File পড়তে—

→ MCP

Database Query করতে—

→ Connector

সব Result আবার Workflow-এ ফিরে আসবে।

ধাপ ৯ — Provider System

সব তথ্য প্রস্তুত হলে Provider Manager ঠিক করবে—

কোন Model ব্যবহার হবে।

উদাহরণ—

- GPT
- Gemini
- Groq
- default এ যে llm selsct করা আছে সেটা ব্যবহার করবে
- ভবিষ্যতের অন্য Provider

Provider Request-কে নির্বাচিত Model-এর Format-এ রূপান্তর করবে।

ধাপ ১০ — LLM Execution

এখানে Model আসল Response তৈরি করবে।

Model ব্যবহার করবে—

- User Prompt
- Planning Result
- Tool Result
- Memory
- Workflow Context
- System Instruction

তারপর Response তৈরি করবে।

ধাপ ১১ — Response Validation

Response সরাসরি User-এর কাছে যাবে না।

প্রথমে Validation হবে।

যেমন—

- Response Complete কি?
- Format ঠিক আছে?
- কোনো Error আছে?
- কোনো Sensitive তথ্য আছে?
- কোনো Tool Output অসম্পূর্ণ কি?

প্রয়োজন হলে Response সংশোধন করা হবে।

ধাপ ১২ — Memory Update

যদি Conversation ভবিষ্যতের জন্য গুরুত্বপূর্ণ হয়—

তাহলে Memory System নতুন তথ্য সংরক্ষণ করতে পারে।

সব Conversation Memory-তে যাবে না।

শুধুমাত্র প্রয়োজনীয় তথ্য সংরক্ষণ করা হবে।

ধাপ ১৩ — Final Response

সবশেষে Final Response ব্যবহারকারীর কাছে পাঠানো হবে।

এই Response হবে—

- নিরাপদ
- যাচাইকৃত
- প্রয়োজনীয় Context-সহ
- Tool Result-সহ
- Memory-aware
- Production Ready

Component Connection

Execution Flow-এর সময় Component-গুলোর সম্পর্ক হবে—

User

|

▼

Core Layer

|

▼

Security

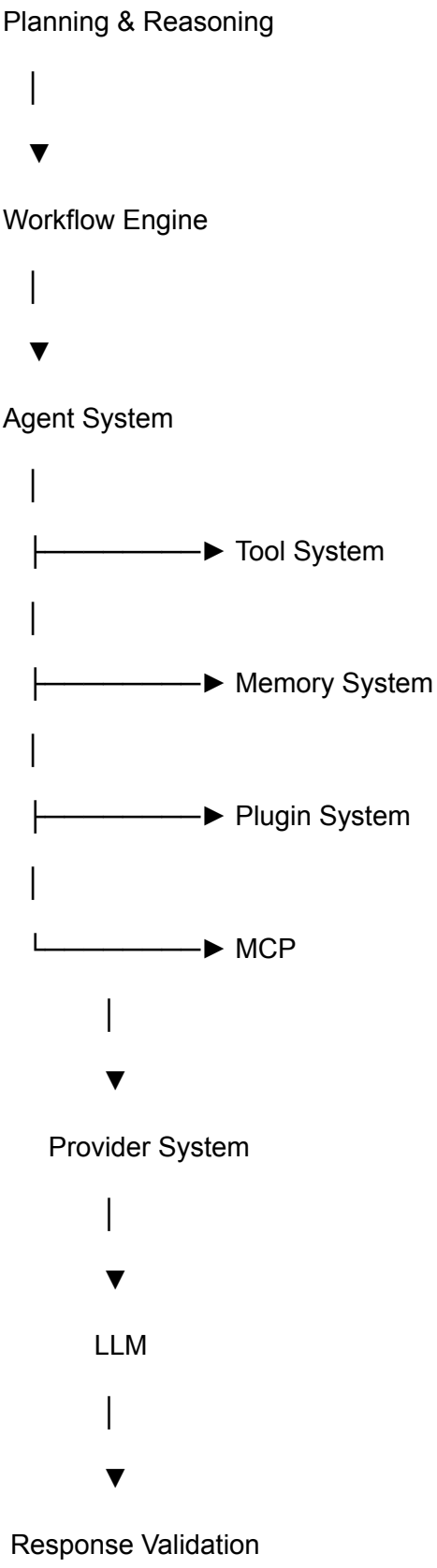
|

▼

Prompt Management

|

▼


Planning & Reasoning
Workflow Engine
Agent System
Tool System
Memory System
Plugin System
MCP
Provider System
LLM
Response Validation

|

▼

Memory Update

|

▼

Final Response

বর্তমান Implementation

বর্তমানে IRON BOLT-এ রয়েছে—

- Core Foundation
- Configuration System
- Provider Manager
- Provider Abstraction
- Runtime Validation
- Prompt Processing-এর প্রাথমিক ভিত্তি
- LLM Communication

Execution Flow এখনো সরল এবং Foundation-কেন্দ্রিক।

ভবিষ্যতের Implementation

পরবর্তী সংস্করণগুলোতে যুক্ত হবে—

- সম্পূর্ণ Planning Pipeline
- Intelligent Workflow Execution
- Multi-Agent Coordination
- Dynamic Tool Routing
- Advanced Memory Integration
- Plugin Execution Pipeline
- MCP Communication Layer

- Response Validation Engine
- Streaming Response Pipeline
- Background Task Execution
- Parallel Workflow Support
- Failure Recovery System
- Retry Strategy
- Execution Monitoring
- Telemetry ও Performance Analytics

এগুলো যুক্ত হওয়ার মাধ্যমে Execution Flow একটি Production-grade Intelligent Execution Pipeline-এ পরিণত হবে, যা জটিল অনুরোধও ধাপে ধাপে পরিকল্পনা, সিদ্ধান্ত, কার্যসম্পাদন এবং যাচাইয়ের মাধ্যমে নির্ভরযোগ্যভাবে সম্পন্ন করতে সক্ষম হবে।

বর্তমান (Current) Execution Flow Explain (AR0)

IRON BOLT Current Project Report (Sprint 2)

Overall Status

Project State: 🟡 Core Foundation Complete

বর্তমানে IRON BOLT একটি **AI Assistant Framework** হিসেবে দাঁড়িয়ে গেছে। অর্থাৎ system-এর backbone তৈরি হয়েছে, কিন্তু AI execution layer এখনো সম্পূর্ণ হয়নি।

এখনকার architecture এমন:

User
  |
  ▼
Router
  |
  ▼
Orchestrator
  |
  ▼
Placeholder

এখানে LLM, Agent, Memory, Tool Execution এখনো production stage-এ নেই।

ইতোমধ্যে যেগুলো তৈরি হয়েছে

✅ Core Infrastructure

- Config System
- Logger
- Router
- Orchestrator
- State Management
- Startup Validation
- Interfaces

এগুলো project-এর foundation।

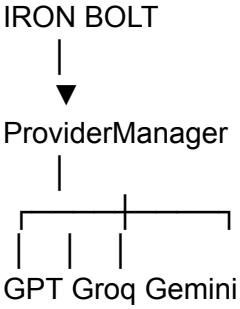
✅ Provider Layer

তুমি ইতোমধ্যে Provider abstraction তৈরি করেছ।

যেমন

- BaseProvider
- ProviderManager
- OpenAI Provider
- Groq Provider
- Gemini Provider (basic)

অর্থাৎ



এই design ভবিষ্যতের জন্য খুব ভালো।

✅ Runtime Configuration

Project এখন provider-independent।

মানে

আজ

GPT

আগামীকাল

Groq

পরশু

Gemini

IRON BOLT-এর code পরিবর্তন না করেও ব্যবহার করা যাবে।

✅ Startup Validation

Project শুরু হওয়ার আগে

- config
- environment
- provider

validate করা যায়।

এটা production-এর জন্য গুরুত্বপূর্ণ।

✅ Logging System

আজকের log-ই তার প্রমাণ।

Router

↓

Orchestrator

↓

Status

↓

Response

সব trace হচ্ছে।

Debugging অনেক সহজ হবে।

✅ Testing

52/52 test pass করেছে।

মানে foundation-এর reliability ভালো।

বর্তমানে যেগুলো নেই

❌ Agent Layer

এটাই সবচেয়ে বড় missing part।

বর্তমানে

Router

↓

general_agent

↓

Placeholder

হচ্ছে।

কিন্তু হওয়া উচিত

general_agent.execute()

↓

LLM

↓

Tool

↓

Memory

↓

Response

❌ LLM Execution

Provider আছে।

কিন্তু

Provider ব্যবহার করে

generate()

call করার execution path এখনো connect হয়নি।

এজন্য

আজকের output এসেছে

Would have routed...

Placeholder...

❌ Prompt Management

এখনো

System Prompt

Developer Prompt

Memory Prompt

User Prompt

combine হচ্ছে না।

❌ Memory

Long-term Memory

Short-term Memory

Conversation Memory

এখনো implement হয়নি।

❌ Planning

Task Breakdown

Reasoning

Planning Engine

নেই।

❌ Tool Execution

Future

GitHub

Browser

Filesystem

Terminal

API

Database

কোনোটাই এখনো integrated না।

❌ Permission System

Risk Analysis

Permission

Confirmation

নেই।

বর্তমান Execution Flow

আজ তুমি যেটা দেখেছ

User

↓

Router

↓

general_agent selected

↓

Orchestrator

↓

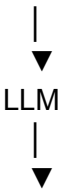
Placeholder Response

এটাই বর্তমান execution।

Sprint 3-এর Main Goal

Sprint 3-এর পর architecture হওয়া উচিত

User
  |
  ▼
Router
  |
  ▼
Agent
  |
  ▼
Planning
  |
  ▼
Memory
  |
  ▼
ProviderManager
  |
  ▼
Selected LLM
  |
  ▼
Tool (if needed)



Project Maturity

Component	Status
Core Architecture	✅ Complete
Provider System	✅ Complete
Config	✅ Complete
Logger	✅ Complete
Router	✅ Complete
Startup Validation	✅ Complete
Tests	✅ Stable
Agent Layer	⏳ Pending
Prompt Manager	⏳ Pending
Memory	⏳ Pending
Planning	⏳ Pending
Tool Execution	⏳ Pending
Permission System	⏳ Pending
Production AI Workflow	⏳ Pending

সারসংক্ষেপ

বর্তমান IRON BOLT-কে আমি প্রায় ৩০–৪০% সম্পূর্ণ বলব, যদি লক্ষ্য হয় একটি পূর্ণাঙ্গ AI assistant (Jarvis/Copilot-ধরনের সিস্টেম) তৈরি করা। তবে এই শতাংশ বিভ্রান্তিকরও হতে পারে, কারণ তুমি যে অংশটি আগে

তৈরি করেছ—Core Architecture, Provider Layer, Logging, Configuration—সেগুলোই সবচেয়ে কঠিন ভিত্তির কাজ। এখন থেকে বাকি উন্নয়নের বেশিরভাগই হবে এই foundation-এর ওপর Agent, Planning, Memory, Tool এবং Workflow যোগ করা। তাই সামনে অনেক feature বাকি থাকলেও, project-এর ভিত্তি ইতোমধ্যে সঠিক দিকেই তৈরি হয়েছে।

Chapter 17 — Future Vision & Expansion

পরিচিতি

IRON BOLT শুধুমাত্র একটি AI Assistant তৈরির প্রকল্প নয়।

এটি এমন একটি Intelligent AI Platform তৈরির দীর্ঘমেয়াদী পরিকল্পনা, যা ভবিষ্যতে একজন মানুষের মতো বিভিন্ন কাজ বুঝতে, পরিকল্পনা করতে, সিদ্ধান্ত নিতে, বাস্তব কাজ সম্পন্ন করতে এবং নতুন দক্ষতা অর্জন করতে পারবে।

বর্তমান Architecture শুধুমাত্র প্রথম Foundation।

এই Foundation-এর উপর ধাপে ধাপে নতুন Capability যুক্ত হবে।

এজন্য পুরো Project শুরু থেকেই Modular Architecture অনুসরণ করে তৈরি করা হয়েছে, যাতে ভবিষ্যতে বড় পরিবর্তন করলেও বিদ্যমান System ভেঙে না যায়।

Long-Term Vision

দীর্ঘমেয়াদে IRON BOLT এমন একটি Unified AI Platform হবে, যেখানে একজন ব্যবহারকারী এক জায়গা থেকেই বিভিন্ন ধরনের কাজ করতে পারবেন।

যেমন—

- সাধারণ প্রশ্নের উত্তর
- Coding Assistance
- Software Development
- Research
- Planning

- Document Analysis
- Automation
- Personal Productivity
- Business Workflow
- Data Analysis
- AI Agent Collaboration
- External Tool Control
- Multi-step Task Execution

অর্থাৎ একজন ব্যবহারকারীকে বিভিন্ন AI Service-এর মধ্যে ঘুরে বেড়াতে হবে না।

IRON BOLT নিজেই বিভিন্ন System সমন্বয় করে কাজ সম্পন্ন করবে।

Continuous Intelligence

বর্তমান Version মূলত Request → Response ভিত্তিক।

ভবিষ্যতে এটি Continuous Intelligent System-এ পরিণত হবে।

অর্থাৎ—

একটি কাজ শেষ হওয়ার পরেও System প্রয়োজন হলে পরবর্তী কাজ পরিকল্পনা করতে পারবে।

নিজের State পর্যবেক্ষণ করতে পারবে।

অসম্পূর্ণ কাজ Resume করতে পারবে।

নতুন তথ্য অনুযায়ী পরিকল্পনা পরিবর্তন করতে পারবে।

এটি ধীরে ধীরে Reactive Assistant থেকে Proactive Assistant-এ রূপান্তরিত হবে।

Advanced Memory

বর্তমান Memory সীমিত।

ভবিষ্যতে Memory System আরও উন্নত হবে।

যেমন—

- Long-term Memory
- Episodic Memory
- Semantic Memory
- Working Memory
- Conversation Memory
- Knowledge Memory
- User Preference Memory
- Project Memory

System বুঝতে পারবে—

কোন তথ্য সাময়িক,

কোন তথ্য দীর্ঘমেয়াদী,

কোন তথ্য ভুলে যাওয়া উচিত,

এবং কোন তথ্য ভবিষ্যতে ব্যবহার করা দরকার।

Advanced Planning

বর্তমান Planning সীমিত পর্যায়ে থাকবে।

ভবিষ্যতে Planning Engine আরও শক্তিশালী হবে।

যেমন—

- Goal Breakdown
- Recursive Planning
- Multi-Agent Planning
- Dynamic Replanning
- Alternative Strategy Generation
- Risk-aware Planning
- Cost-aware Planning
- Time-aware Planning

এর ফলে বড় ও জটিল কাজও ছোট ছোট ধাপে ভাগ করে দক্ষতার সাথে সম্পন্ন করা সম্ভব হবে।

Smarter Agent Ecosystem

বর্তমানে Agent System সীমিত সংখ্যক Agent নিয়ে শুরু হবে।

ভবিষ্যতে বিভিন্ন বিশেষায়িত Agent যুক্ত হবে।

যেমন—

- Coding Agent
- Research Agent
- Writing Agent
- Planning Agent
- Automation Agent
- File Agent
- Web Agent
- Vision Agent
- Data Analysis Agent
- DevOps Agent
- Security Agent

এরা একে অপরের সাথে সহযোগিতা করে একই কাজ সম্পন্ন করতে পারবে।

Rich Tool Ecosystem

বর্তমানে কয়েকটি Tool দিয়ে শুরু হবে।

ভবিষ্যতে শত শত Tool যুক্ত করা সম্ভব হবে।

যেমন—

- Browser
- Terminal
- Python Runtime
- Database
- Email
- Calendar
- GitHub
- Google Drive
- Notion
- Slack
- Discord
- Cloud Storage
- Spreadsheet
- PDF Processing

- OCR
- Image Processing
- Audio Processing
- Video Processing

Tool Registry-এর মাধ্যমে এগুলো সহজে যুক্ত বা অপসারণ করা যাবে।

Plugin Marketplace

বর্তমানে Plugin System শুধুমাত্র Architecture পর্যায়ে থাকবে।

ভবিষ্যতে Third-party Developer-রা নিজস্ব Plugin তৈরি করতে পারবেন।

এর মাধ্যমে—

- নতুন Tool
- নতুন Workflow
- নতুন Agent
- নতুন Capability

সহজেই যুক্ত করা যাবে।

IRON BOLT Core পরিবর্তন না করেই নতুন Feature যোগ করা সম্ভব হবে।

MCP Ecosystem

Model Context Protocol (MCP) ভবিষ্যতে একটি গুরুত্বপূর্ণ অংশ হবে।

এর মাধ্যমে IRON BOLT বিভিন্ন External Service-এর সাথে Standard Protocol ব্যবহার করে যোগাযোগ করবে।

যেমন—

- Cloud Service
- IDE
- Enterprise System
- Business Software
- Internal Company Tools

একটি Standard Interface-এর মাধ্যমে বহু Service ব্যবহার করা সম্ভব হবে।

Local + Cloud Hybrid Architecture

বর্তমানে মূল লক্ষ্য Local Execution।

ভবিষ্যতে Hybrid Architecture যুক্ত হবে।

যেখানে—

- কিছু কাজ Local Machine-এ চলবে।
- কিছু কাজ Cloud-এ চলবে।
- প্রয়োজন অনুযায়ী Provider নির্বাচন হবে।
- Resource অনুযায়ী Execution পরিবর্তিত হবে।

এতে Performance এবং Cost—দুটিই উন্নত হবে।

Multi-Provider Intelligence

বর্তমানে একটি Provider ব্যবহার করা সম্ভব।

ভবিষ্যতে একই Task-এর জন্য একাধিক Provider একসাথে কাজ করতে পারবে।

যেমন—

- Planning এক Model করবে।
- Coding অন্য Model করবে।
- Verification আরেক Model করবে।
- Final Review অন্য Model করবে।

Provider Router স্বয়ংক্রিয়ভাবে সর্বোত্তম Model নির্বাচন করতে পারবে।

Self-Improving System

ভবিষ্যতে IRON BOLT নিজের Performance বিশ্লেষণ করতে পারবে।

যেমন—

- Response Quality
- Tool Success Rate
- Workflow Success
- Error Pattern
- Execution Time
- User Feedback

এসব বিশ্লেষণ করে ভবিষ্যতের সিদ্ধান্ত আরও উন্নত করতে পারবে।

Enterprise Readiness

ভবিষ্যতের লক্ষ্য শুধুমাত্র ব্যক্তিগত ব্যবহারকারী নয়।

Enterprise পর্যায়েও ব্যবহারযোগ্য একটি Platform তৈরি করা।

যেখানে থাকবে—

- Team Workspace
- Shared Memory
- Shared Knowledge
- Organization Policy
- Audit Log
- Role Management
- Team Permission
- Workspace Isolation
- Enterprise Security

API & SDK (optional)

বর্তমান Iron Bolt-এ Public API নেই।

ভবিষ্যতে যুক্ত হবে—

- REST API
- Python SDK
- JavaScript SDK
- CLI

- Webhook Support
- Streaming API

এর মাধ্যমে অন্য Application সহজেই IRON BOLT ব্যবহার করতে পারবে।

Cross Platform ExPlatform (optional)

ভবিষ্যতে বিভিন্ন Platform-এর জন্য আলাদা Interface থাকবে।

যেমন—

- Desktop Application
- Web Application
- Mobile Application
- Command Line Interface (CLI)
- IDE Extension

সব Platform একই Core Architecture ব্যবহার করবে।

Vision Capabilities (optional)

ভবিষ্যতে শুধুমাত্র Text নয়।

System বুঝতে পারবে—

- Image
- PDF
- Diagram
- Screenshot
- Chart
- Table
- Audio
- Video

এবং প্রয়োজনে এগুলোর উপর ভিত্তি করে সিদ্ধান্ত নিতে পারবে।

Automation Platform (optional)

IRON BOLT ভবিষ্যতে শুধুমাত্র Assistant থাকবে না।

এটি Automation Platform হিসেবেও কাজ করবে।

যেখানে—

একটি Goal দিলে System নিজেই—

- পরিকল্পনা করবে,
- Tool নির্বাচন করবে,
- Workflow চালাবে,
- ফলাফল যাচাই করবে,
- প্রয়োজন হলে পুনরায় চেষ্টা করবে,
- এবং শেষ পর্যন্ত সম্পূর্ণ কাজ সম্পন্ন করবে।

Global Architecture Stability

পুরো Project এমনভাবে তৈরি করা হচ্ছে যাতে ভবিষ্যতে নতুন Feature যুক্ত হলেও Core Architecture পরিবর্তন না করতে হয়।

প্রতিটি নতুন Component শুধুমাত্র Extension হিসেবে যুক্ত হবে।

এতে—

- Maintainability বৃদ্ধি পাবে।
- Scalability বৃদ্ধি পাবে।
- Testing সহজ হবে।
- Development দ্রুত হবে।
- Production Stability বজায় থাকবে।

Final Vision

IRON BOLT-এর চূড়ান্ত লক্ষ্য শুধুমাত্র একটি Chatbot তৈরি করা নয়।

লক্ষ্য হলো একটি Modular, Intelligent, Scalable এবং Production-grade AI Platform তৈরি করা, যা ভবিষ্যতের AI Ecosystem-এর সাথে সহজে অভিযোজিত হতে পারে।

বর্তমান Architecture সেই ভবিষ্যতের ভিত্তি।

আজ যে Foundation তৈরি করা হচ্ছে, ভবিষ্যতের প্রতিটি নতুন Capability সেই Foundation-এর উপরই দাঁড়াবে।

এই Roadmap-এর প্রতিটি অধ্যায় সেই দীর্ঘমেয়াদী লক্ষ্যকে সামনে রেখে পরিকল্পনা করা হয়েছে, যাতে IRON BOLT ধাপে ধাপে একটি শক্তিশালী, নির্ভরযোগ্য এবং সম্প্রসারণযোগ্য AI Platform-এ পরিণত হতে পারে।

End of the Book of the Iron Bolt

এই Book of Iron Bolt-এ IRON BOLT-এর Vision, বর্তমান অবস্থা, Architecture, Development Philosophy, প্রতিটি প্রধান Component, Execution Flow এবং ভবিষ্যৎ সম্প্রসারণের সম্পূর্ণ নকশা তুলে ধরা হয়েছে।

Development-এর প্রতিটি Phase এই Blueprint অনুসরণ করে বাস্তবায়ন করা হবে।

এর ফলে Project-এর প্রতিটি নতুন Feature একই Architecture, একই Design Principle এবং একই Development Philosophy অনুসরণ করবে।

এই নথিই IRON BOLT-এর Official Development Blueprint হিসেবে কাজ করবে এবং ভবিষ্যতের সকল উন্নয়নের মূল রেফারেন্স ডকুমেন্ট হিসেবে ব্যবহৃত হবে।

Chapter 18: Conclusion

একটি যাত্রার শেষ নয়, বরং শুরু

এই Book of Iron Bolt শুধুমাত্র কিছু Feature-এর তালিকা নয়।

এটি IRON BOLT-এর সম্পূর্ণ Development Blueprint।

এই Document পড়ে একজন নতুন Developer বুঝতে পারবে—

- IRON BOLT কী।
- কেন এটি তৈরি করা হচ্ছে।
- Architecture কীভাবে কাজ করে।
- প্রতিটি Component-এর দায়িত্ব কী।
- কোন Component কার সাথে সংযুক্ত।
- কোন Feature কখন তৈরি হবে।
- বর্তমান Implementation কোথায় আছে।

- ভবিষ্যতে Project কোন দিকে এগোবে।

অর্থাৎ এই Roadmap Project-এর "Single Source of Truth" হিসেবে কাজ করবে।

IRON BOLT-এর মূল দর্শন

IRON BOLT-এর উদ্দেশ্য শুধুমাত্র আরেকটি AI Chatbot তৈরি করা নয়।

লক্ষ্য হলো এমন একটি AI Platform তৈরি করা যা—

- Modular
- Scalable
- Secure
- Extensible
- Provider Independent
- Tool Driven
- Memory Aware
- Agent Based
- Workflow Enabled

হবে।

প্রতিটি নতুন Feature এই মূল দর্শনের সাথে সামঞ্জস্য রেখে তৈরি করা হবে।

যদি কোনো Feature এই Architecture-এর সাথে মানানসই না হয়, তাহলে Feature-টি পুনরায় Design করা হবে।

Architecture কখনও Feature-এর জন্য ভাঙা হবে না।

Development Philosophy পুনরায় স্মরণ

IRON BOLT-এর প্রতিটি Development নিচের নীতিগুলো অনুসরণ করবে।

Quality First

দ্রুত Feature যোগ করার পরিবর্তে স্থিতিশীল Feature তৈরি করা হবে।

Simplicity First

যেখানে সহজ Design যথেষ্ট, সেখানে অপ্রয়োজনীয় Complexity যোগ করা হবে না।

Modular Design

প্রতিটি Module স্বাধীনভাবে কাজ করতে পারবে।

একটি Module পরিবর্তন করলে পুরো Project ভেঙে যাবে না।

Test Before Trust

যে Feature Test করা হয়নি, সেটিকে Production Ready ধরা হবে না।

Backward Compatibility

সম্ভব হলে নতুন পরিবর্তন পুরোনো System নষ্ট করবে না।

Documentation First

Code-এর পাশাপাশি Documentation সবসময় Update রাখা হবে।

বর্তমান অবস্থা

বর্তমানে IRON BOLT Foundation Stage-এ রয়েছে।

এখন পর্যন্ত তৈরি হয়েছে—

- Core Foundation
- Configuration System
- Provider Layer
- Provider Registry
- Provider Manager
- Startup Validation
- Logger
- State Management

- Router
- Interface Layer
- Base Provider
- OpenAI Provider
- Gemini Provider
- Groq Provider
- Testing Infrastructure

এই Foundation-এর উপর ভবিষ্যতের প্রতিটি System নির্মাণ করা হবে।

ভবিষ্যতের লক্ষ্য

Roadmap অনুযায়ী ধাপে ধাপে যুক্ত হবে—

- Advanced Memory
- Intelligent Agent System
- Tool Execution
- Workflow Engine
- Planning & Reasoning
- Prompt Management
- Plugin Ecosystem
- MCP Support
- Security Layer
- Permission System
- Multi-Agent Collaboration
- Cloud Deployment
- API Layer
- SDK
- Web Interface
- Desktop Client
- Mobile Client
- Enterprise Features

প্রতিটি Feature বর্তমান Foundation-এর উপর ভিত্তি করেই তৈরি হবে।

কেন এই **Architecture** দীর্ঘমেয়াদে গুরুত্বপূর্ণ

অনেক Project শুরুতে দ্রুত তৈরি হলেও পরে বড় হওয়ার সাথে সাথে জটিল হয়ে যায়।

IRON BOLT শুরু থেকেই দীর্ঘমেয়াদি Architecture মাথায় রেখে ডিজাইন করা হয়েছে।

এর ফলে ভবিষ্যতে—

- নতুন Provider যোগ করা সহজ হবে।
- নতুন Tool যুক্ত করা সহজ হবে।
- নতুন Memory Backend ব্যবহার করা যাবে।
- নতুন Agent তৈরি করা যাবে।
- নতুন Workflow যুক্ত করা যাবে।
- নতুন AI Model ব্যবহার করা যাবে।
- নতুন Platform-এ Deploy করা যাবে।

মূল Foundation পরিবর্তন না করেই System ধীরে ধীরে বড় করা সম্ভব হবে।

Community এবং Contribution

ভবিষ্যতে IRON BOLT Open Source Community-এর জন্য উন্মুক্ত করা হতে পারে।

সেই লক্ষ্য মাথায় রেখে—

- পরিষ্কার Folder Structure
- Consistent Naming
- Modular Code
- Comprehensive Documentation
- Automated Testing
- Clear Contribution Guidelines

ধাপে ধাপে যুক্ত করা হবে।

যাতে নতুন Contributor খুব সহজেই Project বুঝে কাজ শুরু করতে পারেন।

এই Book of Iron Bolt-এর ব্যবহার

এই Document ব্যবহার করা যাবে—

- Project Planning
- Development Tracking
- Architecture Reference
- Team Onboarding

- Contributor Guide
- Feature Prioritization
- Long-term Decision Making

এর ফলে Project-এর প্রতিটি সিদ্ধান্ত একই Vision অনুসরণ করবে।

শেষ কথা

IRON BOLT কোনো একদিনে সম্পূর্ণ হবে না।

এটি ধাপে ধাপে উন্নত হবে।

প্রতিটি Phase আগের Phase-এর উপর ভিত্তি করে তৈরি হবে।

প্রতিটি নতুন Capability আগের Architecture-এর সাথে সামঞ্জস্য রেখে যুক্ত হবে।

এই Book of Iron Bolt-এর উদ্দেশ্য শুধু ভবিষ্যতের Feature নির্ধারণ করা নয়, বরং এমন একটি শক্তিশালী ভিত্তি তৈরি করা যার উপর বহু বছর ধরে নির্ভরযোগ্য, সম্প্রসারণযোগ্য এবং Production-Ready AI Platform গড়ে তোলা সম্ভব।

এই নথি সেই দীর্ঘ যাত্রার ভিত্তিপ্রস্তর। ভবিষ্যতে Project যতই বড় হোক, IRON BOLT-এর মূল দর্শন একই থাকবে—পরিষ্কার Architecture, শক্তিশালী Foundation, নিরাপদ Execution এবং দীর্ঘমেয়াদি স্থায়িত্বের উপর দাঁড়িয়ে একটি আধুনিক AI Platform নির্মাণ।